

CIS 6270

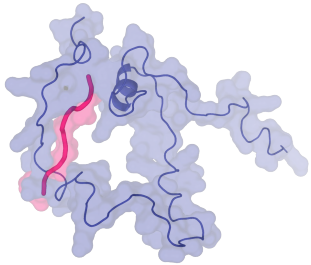
Module 5: Discrete Flow Matching

Lecture 5.3: Dirichlet, Fisher-Rao, and Gumbel-Softmax Flows

Tue, Nov 3, 2026

Pranam Chatterjee, PhD
Assistant Professor of Bioengineering
Assistant Professor of Computer and Information Science
University of Pennsylvania

Penn Engineering
University of Pennsylvania



Put a distance on the tokens

Put a distance on the tokens

Choose a distance d between tokens and concentrate on x_1:

$$p_t(y \mid x_1) = \mathrm{softmax}_y\big(-\beta_t\, d(y, x_1)\big), \qquad \beta_0 = 0,\ \beta_1 = \infty$$

Put a distance on the tokens

Choose a distance d between tokens and concentrate on x_1:

$$\boxed{p_t(y \mid x_1) = \mathrm{softmax}_y\big(-\beta_t\, d(y, x_1)\big), \qquad \beta_0 = 0,\ \beta_1 = \infty}$$

The conditional probability of the token y is a softmax over tokens of negative β_t times the distance $d(y, x_1)$ from y to the data token, where β_t grows from zero at t = 0 to infinity at t = 1.

Put a distance on the tokens

Choose a distance d between tokens and concentrate on x_1:

$$\boxed{p_t(y \mid x_1) = \mathrm{softmax}_y\big(-\beta_t\, d(y, x_1)\big), \qquad \beta_0 = 0,\ \beta_1 = \infty}$$

The conditional probability of the token y is a softmax over tokens of negative β_t times the distance $d(y, x_1)$ from y to the data token, where β_t grows from zero at t = 0 to infinity at t = 1.

$$u_t^\star(y, x \mid x_1) = p_t(y \mid x_1)\ \dot\beta_t \left[\, d(x, x_1) - d(y, x_1) \,\right]_+$$

Put a distance on the tokens

Choose a distance d between tokens and concentrate on x_1:

$$\boxed{p_t(y \mid x_1) = \mathrm{softmax}_y\big(-\beta_t\, d(y, x_1)\big), \qquad \beta_0 = 0,\ \beta_1 = \infty}$$

The conditional probability of the token y is a softmax over tokens of negative β_t times the distance $d(y, x_1)$ from y to the data token, where β_t grows from zero at t = 0 to infinity at t = 1.

$$u_t^\star(y, x \mid x_1) = p_t(y \mid x_1)\ \dot\beta_t \left[\, d(x, x_1) - d(y, x_1) \,\right]_+$$

The kinetic-optimal velocity from x to y equals the probability of y times the rate $\dot\beta_t$ times the positive part of the distance from x to the data token minus the distance from y to the data token.

Put a distance on the tokens

Choose a distance d between tokens and concentrate on x_1:

$$\boxed{p_t(y \mid x_1) = \mathrm{softmax}_y\big(-\beta_t\, d(y, x_1)\big), \qquad \beta_0 = 0,\ \beta_1 = \infty}$$

The conditional probability of the token y is a softmax over tokens of negative β_t times the distance $d(y, x_1)$ from y to the data token, where β_t grows from zero at t = 0 to infinity at t = 1.

$$u_t^\star(y, x \mid x_1) = p_t(y \mid x_1)\ \dot\beta_t \left[\, d(x, x_1) - d(y, x_1) \,\right]_+$$

The kinetic-optimal velocity from x to y equals the probability of y times the rate $\dot\beta_t$ times the positive part of the distance from x to the data token minus the distance from y to the data token.

Probability only moves to tokens strictly closer to x_1, faster when the step in distance is larger.

Let’s build a metric path for DNA

Let’s build a metric path for DNA

Transitions swap a purine for a purine, A and G, or a pyrimidine for a pyrimidine, C and T. They are the more common mutations.

Let’s build a metric path for DNA

Transitions swap a purine for a purine, A and G, or a pyrimidine for a pyrimidine, C and T. They are the more common mutations.

Let d = 1 for a transition and d = 2 for a transversion. With x_1 = G, d(·, G) = (1, 2, 0, 2) for (A, C, G, T).

Let’s build a metric path for DNA

Transitions swap a purine for a purine, A and G, or a pyrimidine for a pyrimidine, C and T. They are the more common mutations.

Let d = 1 for a transition and d = 2 for a transversion. With x_1 = G, d(·, G) = (1, 2, 0, 2) for (A, C, G, T).

At t = 0.5, with $\beta_t = t/(1 - t)$,

$$\beta = 1 : \qquad p_t(\cdot \mid \mathrm{G}) = (0.2245,\ 0.0826,\ 0.6103,\ 0.0826)$$

Let’s build a metric path for DNA

Transitions swap a purine for a purine, A and G, or a pyrimidine for a pyrimidine, C and T. They are the more common mutations.

Let d = 1 for a transition and d = 2 for a transversion. With x_1 = G, d(·, G) = (1, 2, 0, 2) for (A, C, G, T).

At t = 0.5, with $\beta_t = t/(1 - t)$,

$$\beta = 1: \qquad p_t(\cdot \mid \mathrm{G}) = (0.2245,\ 0.0826,\ 0.6103,\ 0.0826)$$

With β = 1, the softmax of negative distances gives probabilities 0.2245 for A, 0.0826 for C, 0.6103 for G, and 0.0826 for T.

Let’s build a metric path for DNA

Transitions swap a purine for a purine, A and G, or a pyrimidine for a pyrimidine, C and T. They are the more common mutations.

Let d = 1 for a transition and d = 2 for a transversion. With x_1 = G, d(·, G) = (1, 2, 0, 2) for (A, C, G, T).

At t = 0.5, with $\beta_t = t/(1 - t)$,

$$\beta = 1: \qquad p_t(\cdot \mid \mathrm{G}) = (0.2245,\ 0.0826,\ 0.6103,\ 0.0826)$$

With $\beta = 4$, a C moves toward A and toward G:

$$u_t^\star(\mathrm{A}, \mathrm{C}) = 0.2245 \times 4 \times (2 - 1) = 0.898, \qquad u_t^\star(\mathrm{G}, \mathrm{C}) = 0.6103 \times 4 \times 2 = 4.882$$

Let’s build a metric path for DNA

Transitions swap a purine for a purine, A and G, or a pyrimidine for a pyrimidine, C and T. They are the more common mutations.

Let d = 1 for a transition and d = 2 for a transversion. With x_1 = G, d(·, G) = (1, 2, 0, 2) for (A, C, G, T).

At t = 0.5, with $\beta_t = t/(1 - t)$,

$$\beta = 1: \qquad p_t(\cdot \mid \mathrm{G}) = (0.2245,\ 0.0826,\ 0.6103,\ 0.0826)$$

With $\beta = 4$, a C moves toward A and toward G:

$$u_t^\star(\mathrm{A}, \mathrm{C}) = 0.2245 \times 4 \times (2 - 1) = 0.898, \qquad u_t^\star(\mathrm{G}, \mathrm{C}) = 0.6103 \times 4 \times 2 = 4.882$$

The rate from C to A is 0.2245 times 4 times the distance gain of one, or 0.898, and the rate from C to G is 0.6103 times 4 times the gain of two, or 4.882.

Let’s build a metric path for DNA

Transitions swap a purine for a purine, A and G, or a pyrimidine for a pyrimidine, C and T. They are the more common mutations.

Let d = 1 for a transition and d = 2 for a transversion. With x_1 = G, d(·, G) = (1, 2, 0, 2) for (A, C, G, T).

At t = 0.5, with $\beta_t = t/(1-t)$,

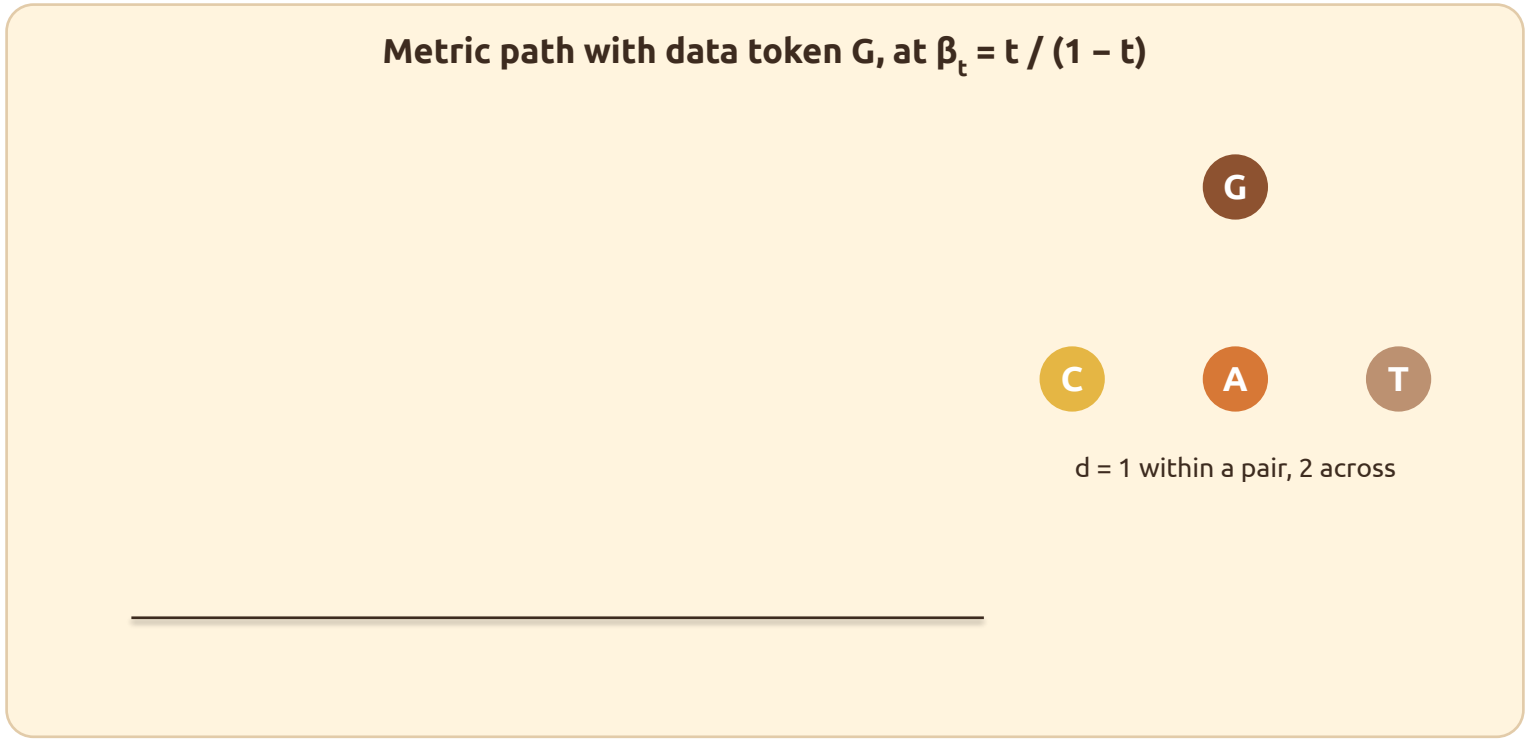
$$\beta = 1: \qquad p_t(\cdot \mid \mathrm{G}) = (0.2245,\ 0.0826,\ 0.6103,\ 0.0826)$$

With $\beta = 4$, a C moves toward A and toward G:

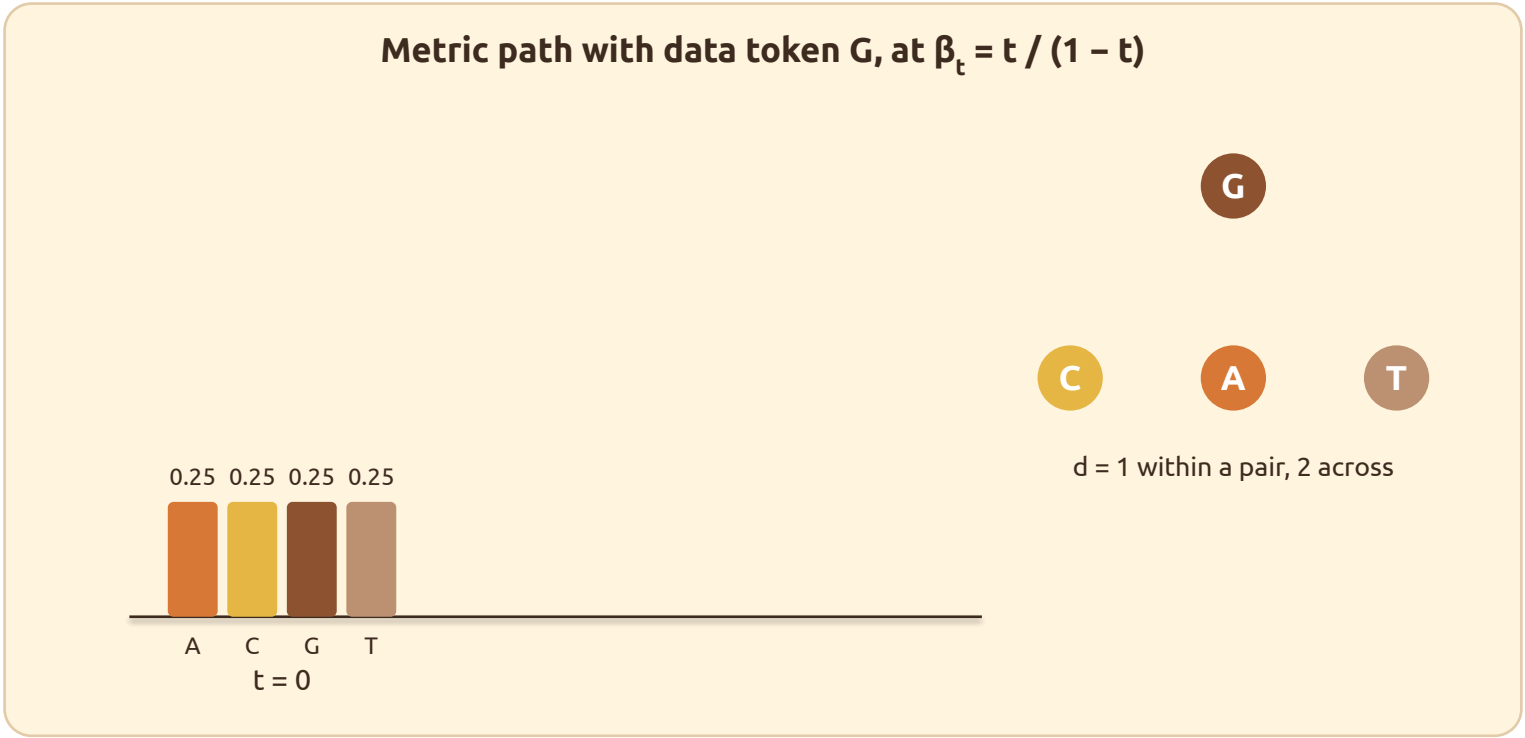
$$u_t^\star(\mathrm{A}, \mathrm{C}) = 0.2245 \times 4 \times (2-1) = 0.898, \qquad u_t^\star(\mathrm{G}, \mathrm{C}) = 0.6103 \times 4 \times 2 = 4.882$$

A pyrimidine can reach G directly or through the purine A. We use the token distances to distinguish smaller changes from larger ones.

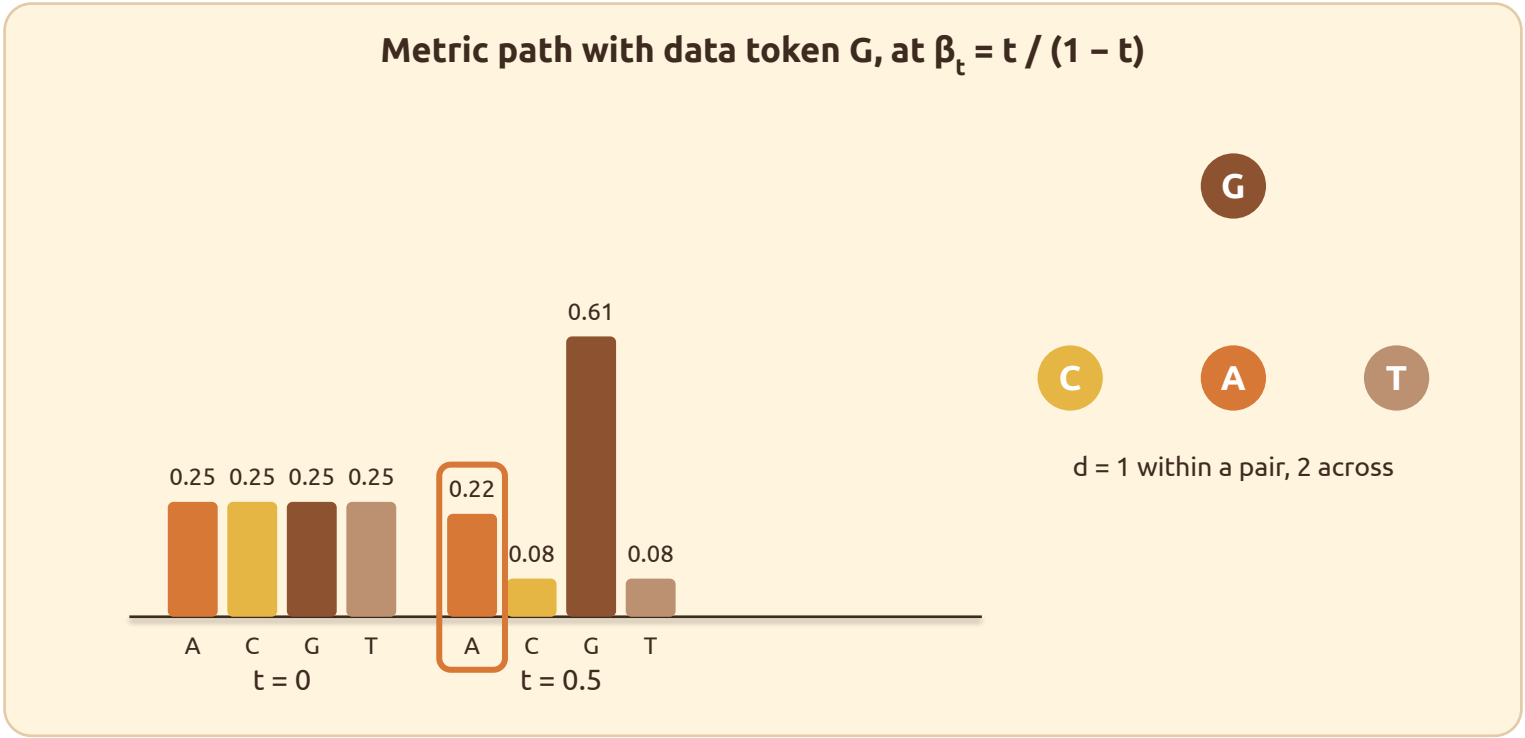
Now watch the metric path concentrate on G



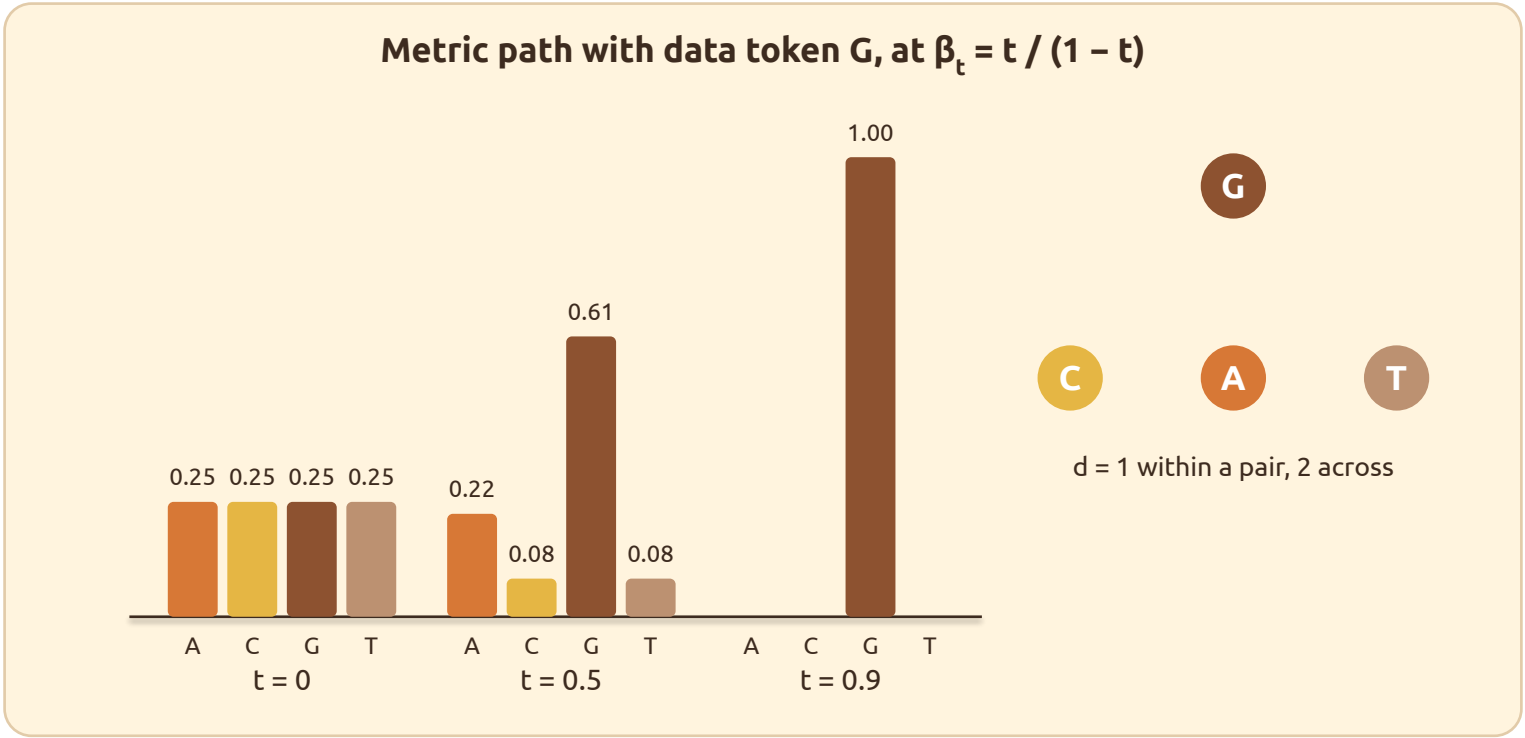
At t = 0, every base is equally likely



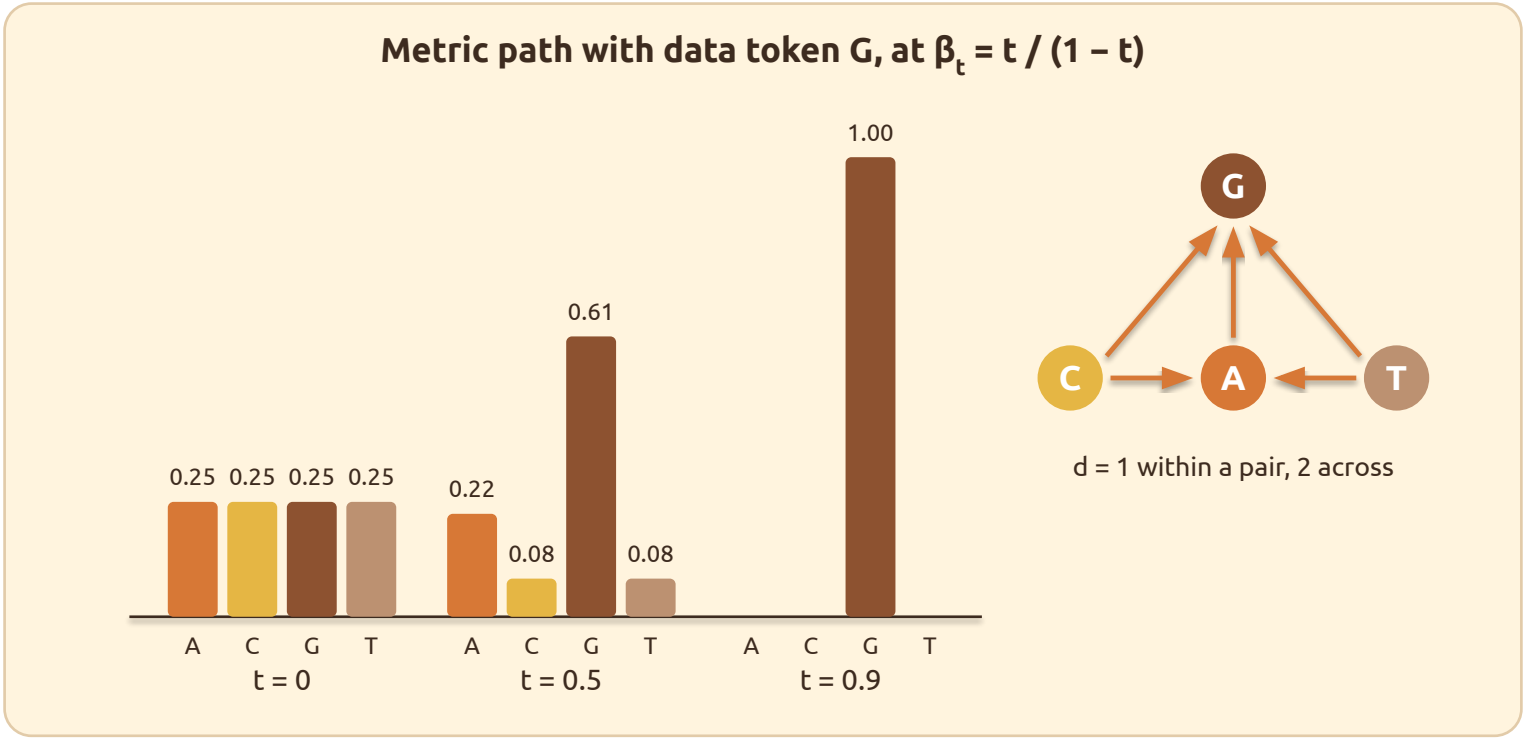
At t = 0.5, A has more probability than C or T because it is one transition from G



At t = 0.9, nearly all probability is on G



The arrows show the only allowed flows: toward bases closer to G



Now minimize the energy over the path as well

Now minimize the energy over the path as well

In the same paper, they also minimize this energy over the path itself, with fixed endpoints p and q.

Now minimize the energy over the path as well

In the same paper, they also minimize this energy over the path itself, with fixed endpoints p and q.

At the optimal velocity, the energy only depends on $\sqrt{p_t}$:

$$\int_0^1 \sum_x \frac{\dot{p}_t(x)^2}{p_t(x)}\,dt = 4\int_0^1 \sum_x \left(\frac{d}{dt}\sqrt{p_t(x)}\right)^2 dt$$

Now minimize the energy over the path as well

In the same paper, they also minimize this energy over the path itself, with fixed endpoints p and q.

At the optimal velocity, the energy only depends on $\sqrt{p_t}$:

$$\int_0^1 \sum_x \frac{\dot{p}_t(x)^2}{p_t(x)}\,dt = 4\int_0^1 \sum_x \left(\frac{d}{dt}\sqrt{p_t(x)}\right)^2 dt$$

The energy, the integral over time of the sum over states of the squared rate of change of $p_t(x)$ divided by $p_t(x)$, equals four times the integral of the squared speed of the square-root vector $\sqrt{p_t}$.

Now minimize the energy over the path as well

In the same paper, they also minimize this energy over the path itself, with fixed endpoints p and q.

At the optimal velocity, the energy only depends on $\sqrt{p_t}$:

$$\int_0^1 \sum_x \frac{\dot{p}_t(x)^2}{p_t(x)}\,dt = 4\int_0^1 \sum_x \left(\frac{d}{dt}\sqrt{p_t(x)}\right)^2 dt$$

The energy, the integral over time of the sum over states of the squared rate of change of $p_t(x)$ divided by $p_t(x)$, equals four times the integral of the squared speed of the square-root vector $\sqrt{p_t}$.

The vector $\sqrt{p_t}$ has unit length, so it lies on a sphere. The shortest path between two points on that sphere is a great-circle arc.

Now minimize the energy over the path as well

In the same paper, they also minimize this energy over the path itself, with fixed endpoints p and q.

At the optimal velocity, the energy only depends on $\sqrt{p_t}$:

$$\int_0^1 \sum_x \frac{\dot{p}_t(x)^2}{p_t(x)}\,dt = 4\int_0^1 \sum_x \left(\frac{d}{dt}\sqrt{p_t(x)}\right)^2 dt$$

The energy, the integral over time of the sum over states of the squared rate of change of $p_t(x)$ divided by $p_t(x)$, equals four times the integral of the squared speed of the square-root vector $\sqrt{p_t}$.

The least-energy probability path follows the shortest arc between its square-root endpoints.

Read the schedule off that arc

Read the schedule off that arc

For a point-mass target, the arc is a mixture path with a token-dependent schedule:

$$\kappa_t(x_1) = 1 - \frac{\sin^2\big((1-t)\,\Omega\big)}{\sin^2\Omega}, \qquad \Omega = \arccos\sqrt{p(x_1)}$$

Read the schedule off that arc

For a point-mass target, the arc is a mixture path with a token-dependent schedule:

$$\kappa_t(x_1) = 1 - \frac{\sin^2\big((1-t)\,\Omega\big)}{\sin^2\Omega}, \qquad \Omega = \arccos\sqrt{p(x_1)}$$

The schedule $\kappa_t(x_1)$ equals one minus the squared sine of one minus t times Ω, divided by the squared sine of Ω, where the angle Ω is the arccosine of the square root of the source probability of the data token x_1.

Read the schedule off that arc

For a point-mass target, the arc is a mixture path with a token-dependent schedule:

$$\kappa_t(x_1) = 1 - \frac{\sin^2\big((1-t)\,\Omega\big)}{\sin^2\Omega}, \qquad \Omega = \arccos\sqrt{p(x_1)}$$

The schedule $\kappa_t(x_1)$ equals one minus the squared sine of one minus t times Ω, divided by the squared sine of Ω, where the angle Ω is the arccosine of the square root of the source probability of the data token x_1.

A token that is already likely under the source starts with a small angle, so its schedule rises early.

Read the schedule off that arc

For a point-mass target, the arc is a mixture path with a token-dependent schedule:

$$\kappa_t(x_1) = 1 - \frac{\sin^2\big((1-t)\,\Omega\big)}{\sin^2\Omega}, \qquad \Omega = \arccos\sqrt{p(x_1)}$$

The schedule $\kappa_t(x_1)$ equals one minus the squared sine of one minus t times Ω, divided by the squared sine of Ω, where the angle Ω is the arccosine of the square root of the source probability of the data token x_1.

A token that is already likely under the source starts with a small angle, so its schedule rises early.

For the mask source, $p(x_1) = 0$ gives $\Omega = \pi/2$ and $\kappa_t = \sin^2(\pi t/2)$.

Next, we look for a likelihood bound

Next, we look for a likelihood bound

In the same paper, they derive an evidence lower bound for any path and velocity, with a closed form for mixture paths.

Next, we look for a likelihood bound

In the same paper, they derive an evidence lower bound for any path and velocity, with a closed form for mixture paths.

For the mask source with one schedule for every token, it reduces to

$$\log p_1^\theta(x_1) \;\geq\; \int_0^1 \frac{\dot{\kappa}_t}{1-\kappa_t}\, \mathbb{E}_{x_t}\Big[\sum_{i:\, x_t^i = m} \log p_{1|t}^\theta(x_1^i \mid x_t) \Big]\, dt$$

Next, we look for a likelihood bound

In the same paper, they derive an evidence lower bound for any path and velocity, with a closed form for mixture paths.

For the mask source with one schedule for every token, it reduces to

$$\log p_1^\theta(x_1) \;\geq\; \int_0^1 \frac{\dot{\kappa}_t}{1-\kappa_t}\, \mathbb{E}_{x_t}\Big[\sum_{i:\, x_t^i = m} \log p_{1|t}^\theta(x_1^i \mid x_t) \Big]\, dt$$

The model's log-likelihood of the data sequence x_1 is at least the integral over time of $\dot{\kappa}_t$ divided by one minus κ_t, times the expectation over the noisy sequence x_t of the sum over masked positions of the log probability the denoiser $p^\theta_{1|t}$ assigns to the true data token.

Next, we look for a likelihood bound

In the same paper, they derive an evidence lower bound for any path and velocity, with a closed form for mixture paths.

For the mask source with one schedule for every token, it reduces to

$$\log p_1^\theta(x_1) \;\geq\; \int_0^1 \frac{\dot{\kappa}_t}{1-\kappa_t}\, \mathbb{E}_{x_t}\Big[\sum_{i:\, x_t^i = m} \log p_{1|t}^\theta(x_1^i \mid x_t) \Big]\, dt$$

The model's log-likelihood of the data sequence x_1 is at least the integral over time of $\dot{\kappa}_t$ divided by one minus κ_t, times the expectation over the noisy sequence x_t of the sum over masked positions of the log probability the denoiser $p^\theta_{1|t}$ assigns to the true data token.

We recover MDLM's likelihood bound using weighted cross-entropy.

Next, we look for a likelihood bound

In the same paper, they derive an evidence lower bound for any path and velocity, with a closed form for mixture paths.

For the mask source with one schedule for every token, it reduces to

$$\log p_1^\theta(x_1) \;\geq\; \int_0^1 \frac{\dot{\kappa}_t}{1-\kappa_t}\, \mathbb{E}_{x_t}\Big[\sum_{i:\, x_t^i = m} \log p_{1|t}^\theta(x_1^i \mid x_t) \Big]\, dt$$

The model's log-likelihood of the data sequence x_1 is at least the integral over time of $\dot{\kappa}_t$ divided by one minus κ_t, times the expectation over the noisy sequence x_t of the sum over masked positions of the log probability the denoiser $p^\theta_{1|t}$ assigns to the true data token.

We recover MDLM's likelihood bound using weighted cross-entropy. Unweighted cross-entropy has the same minimizer.

Take a single masked base, with the linear schedule

Take a single masked base, with the linear schedule

For a single masked base with $\kappa_t = t$, the weight is $1/(1 - t)$, while the masking probability is $1 - t$.

Take a single masked base, with the linear schedule

For a single masked base with $\kappa_t = t$, the weight is $1/(1 - t)$, while the masking probability is $1 - t$.

Their product is 1, so the integral of a constant log $p^{\theta}(G)$ over $[0, 1]$ is that log probability itself.

Take a single masked base, with the linear schedule

For a single masked base with $\kappa_t = t$, the weight is $1/(1 - t)$, while the masking probability is $1 - t$.

Their product is 1, so the integral of a constant log $p^\theta(G)$ over $[0, 1]$ is that log probability itself.

With $p^\theta(G) = 0.6$, the bound comes to $\log 0.6 = -0.511$, the negative cross-entropy of that one position.

Kinetic-optimal flow on A and G

Toy masses on A and G are (0.8, 0.2), with derivatives (−0.1, +0.1). Use the kinetic-optimal rates for the reciprocal-probability edge weights.

Kinetic-optimal flow on A and G

Toy masses on A and G are (0.8, 0.2), with derivatives (−0.1, +0.1). Use the kinetic-optimal rates for the reciprocal-probability edge weights.

$$u^*(\mathrm{G},\mathrm{A}) = 0.2\left[\frac{0.1}{0.2} - \frac{-0.1}{0.8}\right]_+ = 0.125$$

Kinetic-optimal flow on A and G

Toy masses on A and G are (0.8, 0.2), with derivatives (−0.1, +0.1). Use the kinetic-optimal rates for the reciprocal-probability edge weights.

$$u^*(\mathrm{G},\mathrm{A}) = 0.2\left[\frac{0.1}{0.2} - \frac{-0.1}{0.8}\right]_+ = 0.125$$

G grows at relative rate 0.5 while A shrinks at relative rate 0.125. Their difference sets the jump rate.

Kinetic-optimal flow on A and G

Toy masses on A and G are (0.8, 0.2), with derivatives (−0.1, +0.1). Use the kinetic-optimal rates for the reciprocal-probability edge weights.

$$u^*(\mathrm{G},\mathrm{A}) = 0.2\left[\frac{0.1}{0.2} - \frac{-0.1}{0.8}\right]_+ = 0.125$$

G grows at relative rate 0.5 while A shrinks at relative rate 0.125. Their difference sets the jump rate.

$$j(\mathrm{G},\mathrm{A}) = (0.8)(0.125) = 0.1, \qquad p_{t+0.1}(\mathrm{G}) \approx 0.2 + 0.1(0.1) = 0.21$$

Kinetic-optimal flow on A and G

Toy masses on A and G are (0.8, 0.2), with derivatives (−0.1, +0.1). Use the kinetic-optimal rates for the reciprocal-probability edge weights.

$$u^*(\mathrm{G},\mathrm{A}) = 0.2\left[\frac{0.1}{0.2} - \frac{-0.1}{0.8}\right]_+ = 0.125$$

G grows at relative rate 0.5 while A shrinks at relative rate 0.125. Their difference sets the jump rate.

$$j(\mathrm{G},\mathrm{A}) = (0.8)(0.125) = 0.1, \qquad p_{t+0.1}(\mathrm{G}) \approx 0.2 + 0.1(0.1) = 0.21$$

The flux is 0.1. A short first-order update moves the G mass from 0.20 to 0.21.

Kinetic-optimal flow on A and G

Toy masses on A and G are (0.8, 0.2), with derivatives (−0.1, +0.1). Use the kinetic-optimal rates for the reciprocal-probability edge weights.

$$u^*(\mathrm{G},\mathrm{A}) = 0.2\left[\frac{0.1}{0.2} - \frac{-0.1}{0.8}\right]_+ = 0.125$$

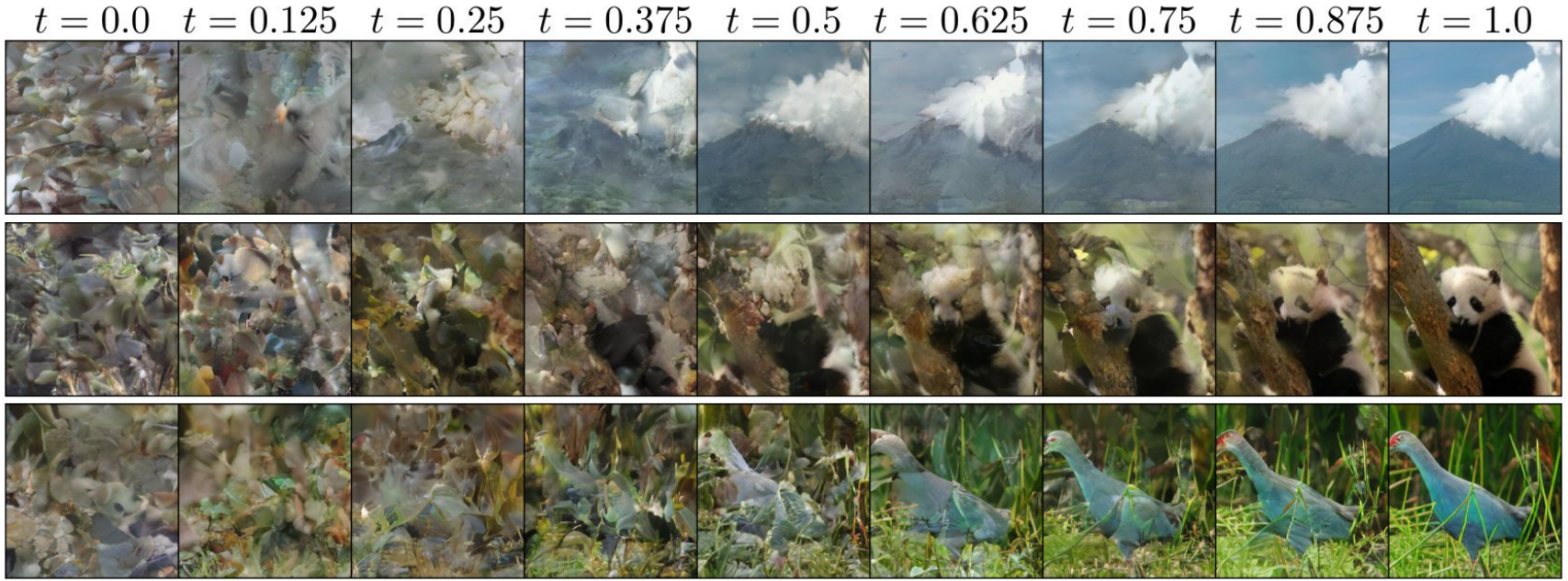
G grows at relative rate 0.5 while A shrinks at relative rate 0.125. Their difference sets the jump rate.

$$j(\mathrm{G},\mathrm{A}) = (0.8)(0.125) = 0.1, \qquad p_{t+0.1}(\mathrm{G}) \approx 0.2 + 0.1(0.1) = 0.21$$

The flux is 0.1. A short first-order update moves the G mass from 0.20 to 0.21.

The rate describes one particle. The flux tells us how much total probability moves.

On image tokens, a metric path runs from noise to an image



Their metric-induced path on image tokens, from t = 0 to t = 1.

Shaul,..,Chen. *ICLR* (2025)

Now look at the FID numbers in Table 3 of the paper

Table 3: Face-blurred ImageNet-256 with the Llama-B architecture (111M parameters).
* denotes our reimplementation.

Method	NFE	FID
LlamaGen (AR) (Sun et al., 2024)	256	5.46
LlamaGen (AR)* (Sun et al., 2024)	256	4.81
DFM - Mask* (Gat et al., 2024)	100	5.72
DFM - Metric (Ours)	100	**4.50**

Shaul,..,Chen. *ICLR* (2025)

On face-blurred ImageNet at 256 by 256, the masked path reaches an FID of 5.72

Table 3: Face-blurred ImageNet-256 with the Llama-B architecture (111M parameters).
* denotes our reimplementation.

Method	NFE	FID
LlamaGen (AR) (Sun et al., 2024)	256	5.46
LlamaGen (AR)* (Sun et al., 2024)	256	4.81
DFM - Mask* (Gat et al., 2024)	100	5.72
DFM - Metric (Ours)	100	**4.50**

Shaul,..,Chen. *ICLR* (2025)

With the metric path, FID improves to 4.50 in 100 steps

Table 3: Face-blurred ImageNet-256 with the Llama-B architecture (111M parameters).
* denotes our reimplementation.

Method	NFE	FID
LlamaGen (AR) (Sun et al., 2024)	256	5.46
LlamaGen (AR)* (Sun et al., 2024)	256	4.81
DFM - Mask* (Gat et al., 2024)	100	5.72
DFM - Metric (Ours)	100	**4.50**

Shaul,..,Chen. *ICLR* (2025)

The autoregressive LlamaGen reaches 4.81 in 256 steps

Table 3: Face-blurred ImageNet-256 with the Llama-B architecture (111M parameters).
* denotes our reimplementation.

Method	NFE	FID
LlamaGen (AR) (Sun et al., 2024)	256	5.46
LlamaGen (AR)* (Sun et al., 2024)	256	4.81
DFM - Mask* (Gat et al., 2024)	100	5.72
DFM - Metric (Ours)	100	**4.50**

Shaul,..,Chen. *ICLR* (2025)

We choose the probability path just as we chose the corruption process in discrete diffusion

Table 3: Face-blurred ImageNet-256 with the Llama-B architecture (111M parameters).
* denotes our reimplementation.

Method	NFE	FID
LlamaGen (AR) (Sun et al., 2024)	256	5.46
LlamaGen (AR)* (Sun et al., 2024)	256	4.81
DFM - Mask* (Gat et al., 2024)	100	5.72
DFM - Metric (Ours)	100	**4.50**

Shaul,..,Chen. *ICLR* (2025)

Could the flow insert and delete tokens so the sequence length can change too?

Edit Flows

Edit Flows: Variable Length Discrete Flow Matching with Sequence-Level Edit Operations

Marton Havasi FAIR at Meta
Brian Karrer FAIR at Meta
Itai Gat FAIR at Meta
Ricky T. Q. Chen FAIR at Meta

Havasi,..,Chen. *NeurIPS* (2025)

So far, every path has had a fixed length

So far, every path has had a fixed length

A substitution changes a token, but it never adds or removes one.

So far, every path has had a fixed length

A substitution changes a token, but it never adds or removes one.

If only substitutions have positive rates, the length never changes, so p_t stays on sequences of one length for all t.

So far, every path has had a fixed length

A substitution changes a token, but it never adds or removes one.

If only substitutions have positive rates, the length never changes, so p_t stays on sequences of one length for all t.

DNA, proteins, molecules, and code vary in length. With a mask model, we must choose the length before sampling.

In Edit Flows we also allow insertions and deletions

In Edit Flows we also allow insertions and deletions

In Edit Flows, there are three edits at each position i:

$$\mathrm{ins}(x, i, a), \qquad \mathrm{del}(x, i), \qquad \mathrm{sub}(x, i, a)$$

In Edit Flows we also allow insertions and deletions

In Edit Flows, there are three edits at each position i:

$$\mathrm{ins}(x, i, a), \qquad \mathrm{del}(x, i), \qquad \mathrm{sub}(x, i, a)$$

An insertion places the token a to the right of position i, a deletion removes the token at position i, and a substitution replaces the token at position i with a.

In Edit Flows we also allow insertions and deletions

In Edit Flows, there are three edits at each position i:

$$\mathrm{ins}(x, i, a), \qquad \mathrm{del}(x, i), \qquad \mathrm{sub}(x, i, a)$$

An insertion places the token a to the right of position i, a deletion removes the token at position i, and a substitution replaces the token at position i with a.

$$u_t(\mathrm{ins}(x,i,a) \mid x) = \lambda_{t,i}^{\mathrm{ins}}(x)\, Q_{t,i}^{\mathrm{ins}}(a \mid x), \quad u_t(\mathrm{del}(x,i) \mid x) = \lambda_{t,i}^{\mathrm{del}}(x), \quad u_t(\mathrm{sub}(x,i,a) \mid x) = \lambda_{t,i}^{\mathrm{sub}}(x)\, Q_{t,i}^{\mathrm{sub}}(a \mid x)$$

In Edit Flows we also allow insertions and deletions

In Edit Flows, there are three edits at each position i:

$$\mathrm{ins}(x, i, a), \qquad \mathrm{del}(x, i), \qquad \mathrm{sub}(x, i, a)$$

An insertion places the token a to the right of position i, a deletion removes the token at position i, and a substitution replaces the token at position i with a.

$$u_t(\mathrm{ins}(x,i,a) \mid x) = \lambda^{\mathrm{ins}}_{t,i}(x)\, Q^{\mathrm{ins}}_{t,i}(a \mid x), \quad u_t(\mathrm{del}(x,i) \mid x) = \lambda^{\mathrm{del}}_{t,i}(x), \quad u_t(\mathrm{sub}(x,i,a) \mid x) = \lambda^{\mathrm{sub}}_{t,i}(x)\, Q^{\mathrm{sub}}_{t,i}(a \mid x)$$

Each insertion rate is an intensity $\lambda^{\mathrm{ins}}_{t,i}(x)$ times a token distribution $Q^{\mathrm{ins}}_{t,i}(a \mid x)$, each deletion rate is an intensity $\lambda^{\mathrm{del}}_{t,i}(x)$, and each substitution rate is an intensity $\lambda^{\mathrm{sub}}_{t,i}(x)$ times a token distribution $Q^{\mathrm{sub}}_{t,i}(a \mid x)$.

In Edit Flows we also allow insertions and deletions

In Edit Flows, there are three edits at each position i:

$$\mathrm{ins}(x, i, a), \qquad \mathrm{del}(x, i), \qquad \mathrm{sub}(x, i, a)$$

An insertion places the token a to the right of position i, a deletion removes the token at position i, and a substitution replaces the token at position i with a.

$$u_t(\mathrm{ins}(x,i,a) \mid x) = \lambda^{\mathrm{ins}}_{t,i}(x)\, Q^{\mathrm{ins}}_{t,i}(a \mid x), \quad u_t(\mathrm{del}(x,i) \mid x) = \lambda^{\mathrm{del}}_{t,i}(x), \quad u_t(\mathrm{sub}(x,i,a) \mid x) = \lambda^{\mathrm{sub}}_{t,i}(x)\, Q^{\mathrm{sub}}_{t,i}(a \mid x)$$

Each insertion rate is an intensity $\lambda^{\mathrm{ins}}_{t,i}(x)$ times a token distribution $Q^{\mathrm{ins}}_{t,i}(a \mid x)$, each deletion rate is an intensity $\lambda^{\mathrm{del}}_{t,i}(x)$, and each substitution rate is an intensity $\lambda^{\mathrm{sub}}_{t,i}(x)$ times a token distribution $Q^{\mathrm{sub}}_{t,i}(a \mid x)$.

We predict three intensities and two token distributions per position.

In Edit Flows we also allow insertions and deletions

In Edit Flows, there are three edits at each position i:

$$\mathrm{ins}(x, i, a), \qquad \mathrm{del}(x, i), \qquad \mathrm{sub}(x, i, a)$$

An insertion places the token a to the right of position i, a deletion removes the token at position i, and a substitution replaces the token at position i with a.

$$u_t(\mathrm{ins}(x,i,a) \mid x) = \lambda^{\mathrm{ins}}_{t,i}(x)\, Q^{\mathrm{ins}}_{t,i}(a \mid x), \quad u_t(\mathrm{del}(x,i) \mid x) = \lambda^{\mathrm{del}}_{t,i}(x), \quad u_t(\mathrm{sub}(x,i,a) \mid x) = \lambda^{\mathrm{sub}}_{t,i}(x)\, Q^{\mathrm{sub}}_{t,i}(a \mid x)$$

Each insertion rate is an intensity $\lambda^{\mathrm{ins}}_{t,i}(x)$ times a token distribution $Q^{\mathrm{ins}}_{t,i}(a \mid x)$, each deletion rate is an intensity $\lambda^{\mathrm{del}}_{t,i}(x)$, and each substitution rate is an intensity $\lambda^{\mathrm{sub}}_{t,i}(x)$ times a token distribution $Q^{\mathrm{sub}}_{t,i}(a \mid x)$.

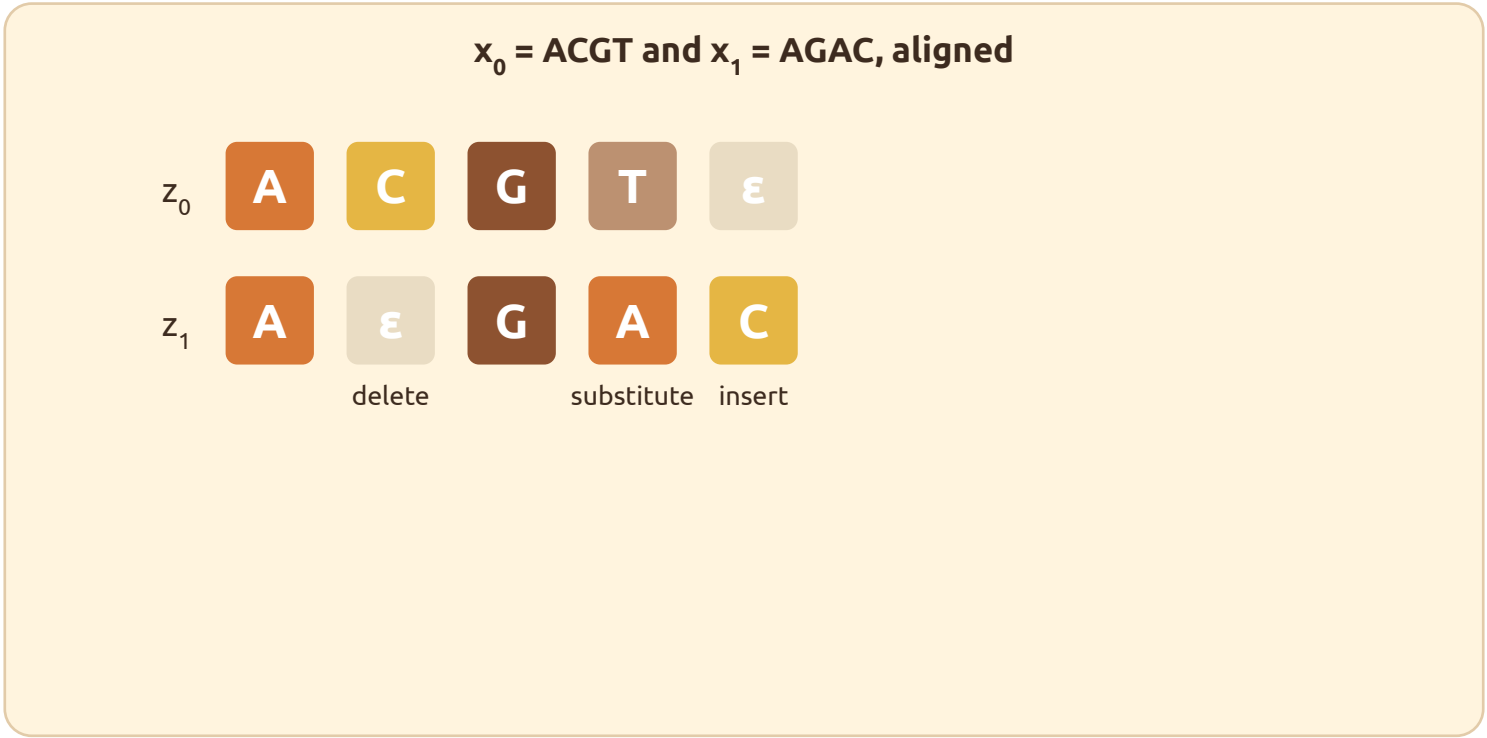
We predict three intensities and two token distributions per position. We set the diagonal to minus the sum of all intensities.

Now align two sequences of different lengths

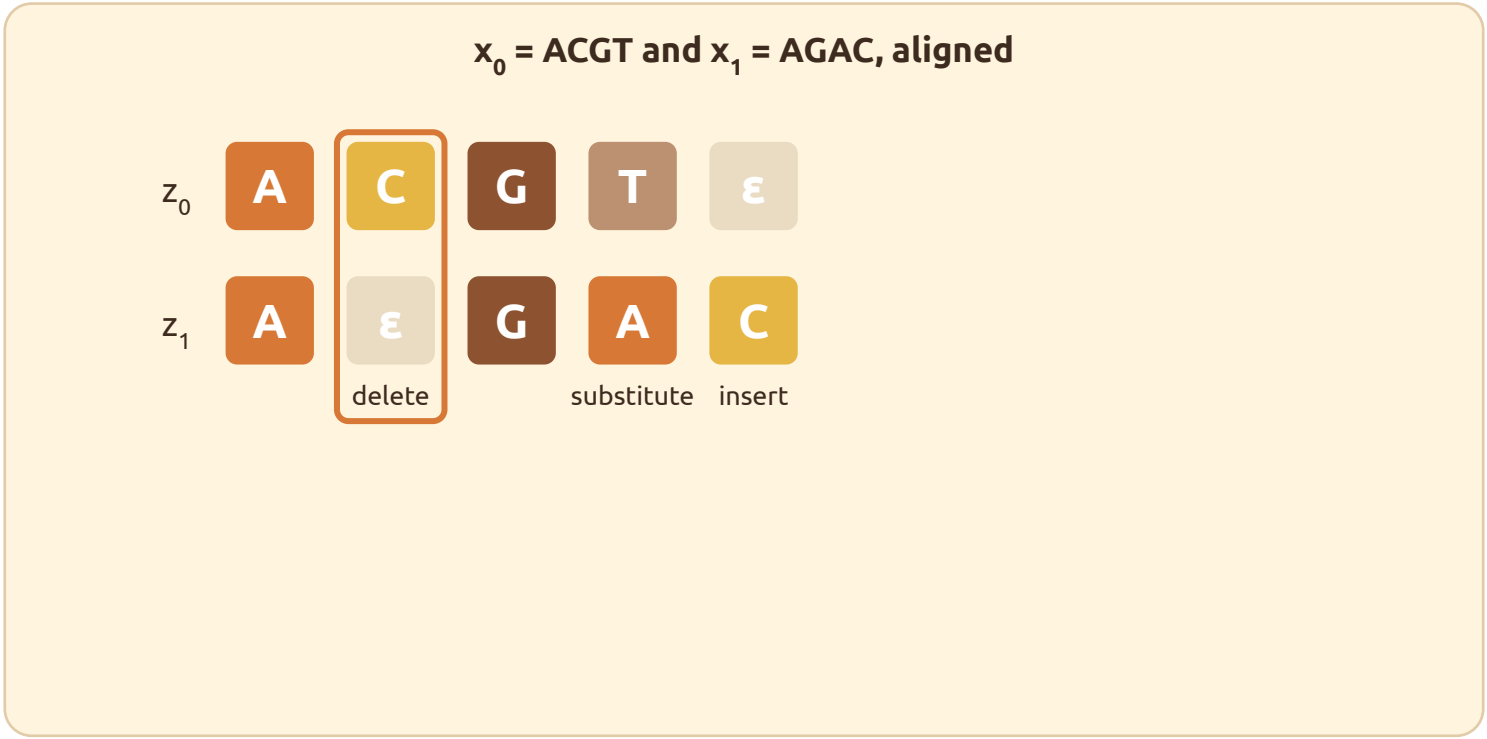
x_0 = ACGT and x_1 = AGAC, aligned

delete substitute insert

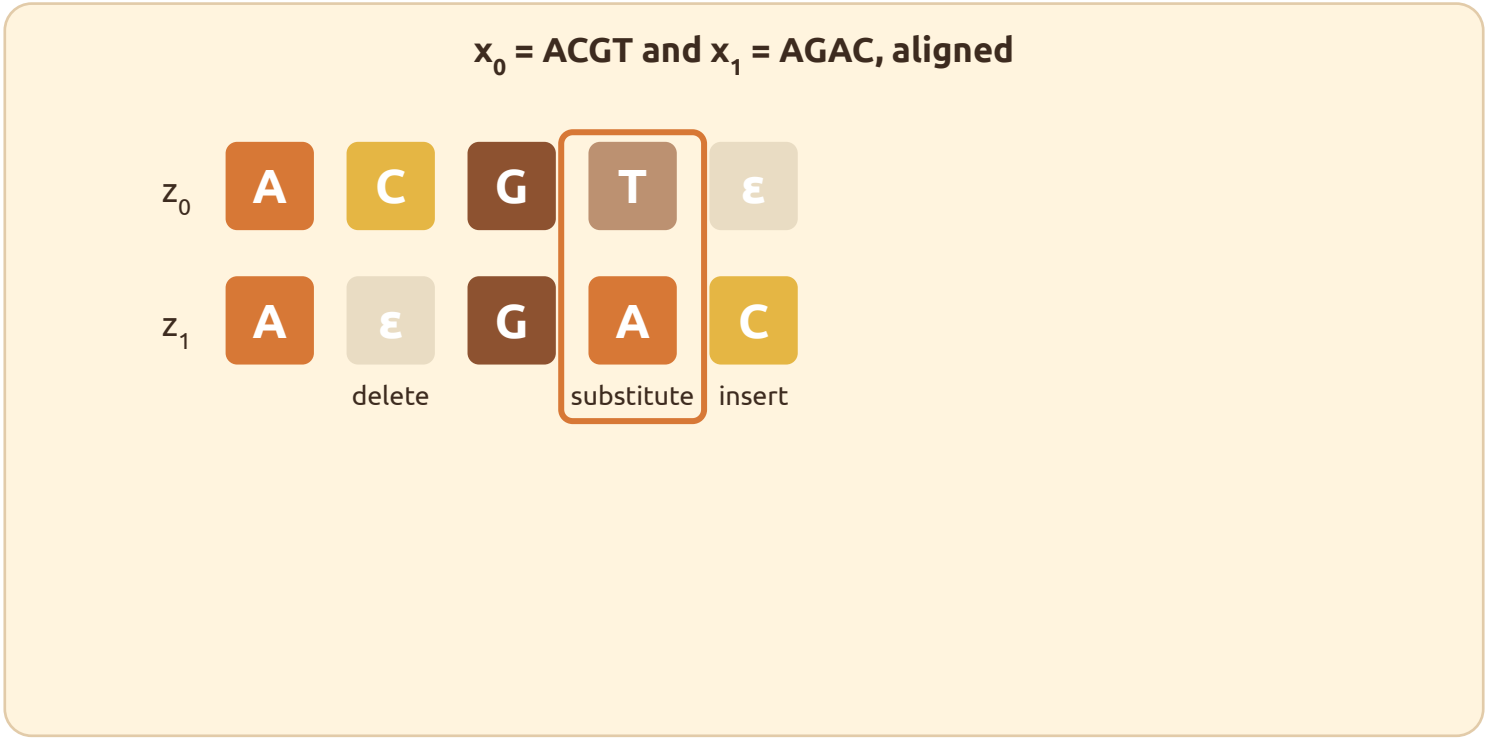
Pad both sequences with a blank ε so that the columns line up



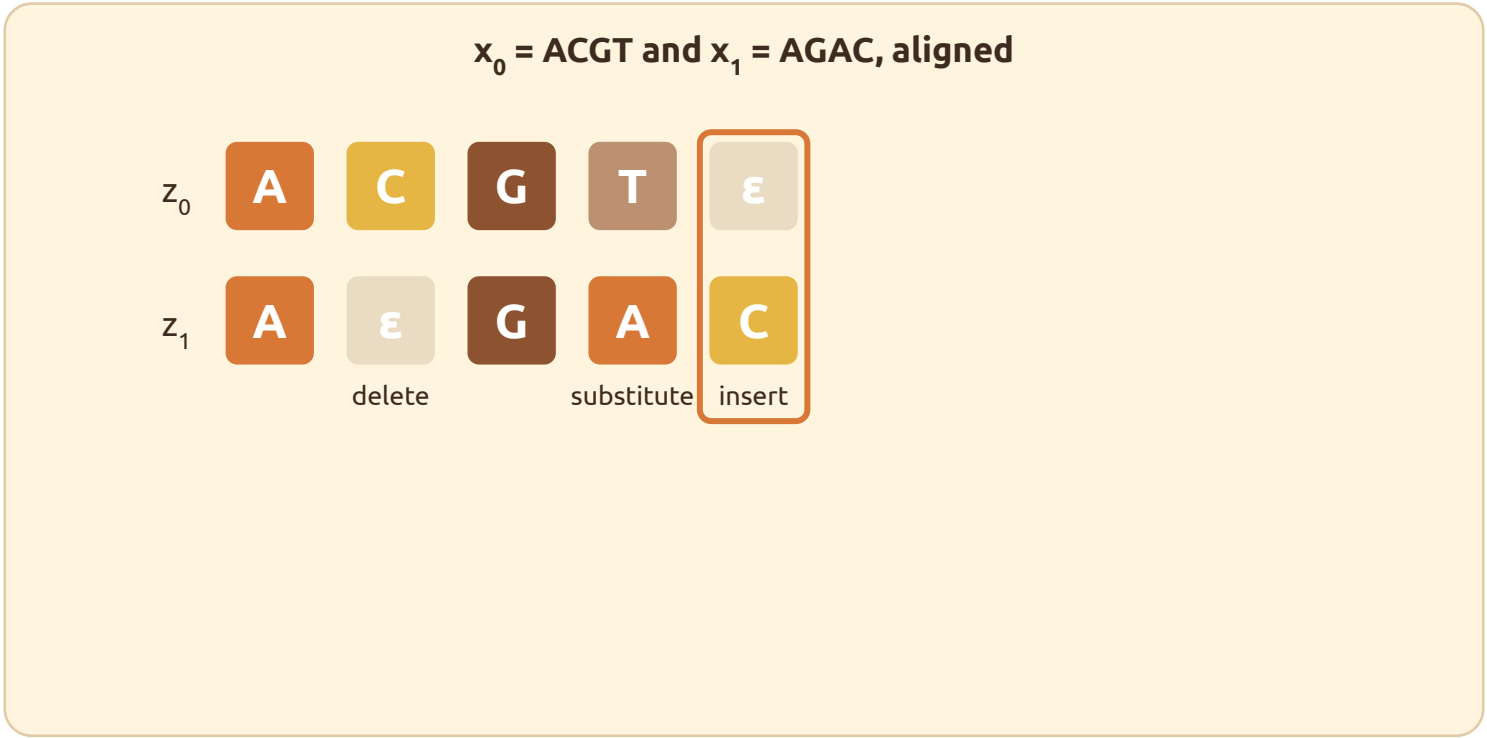
A blank in the target column is a deletion



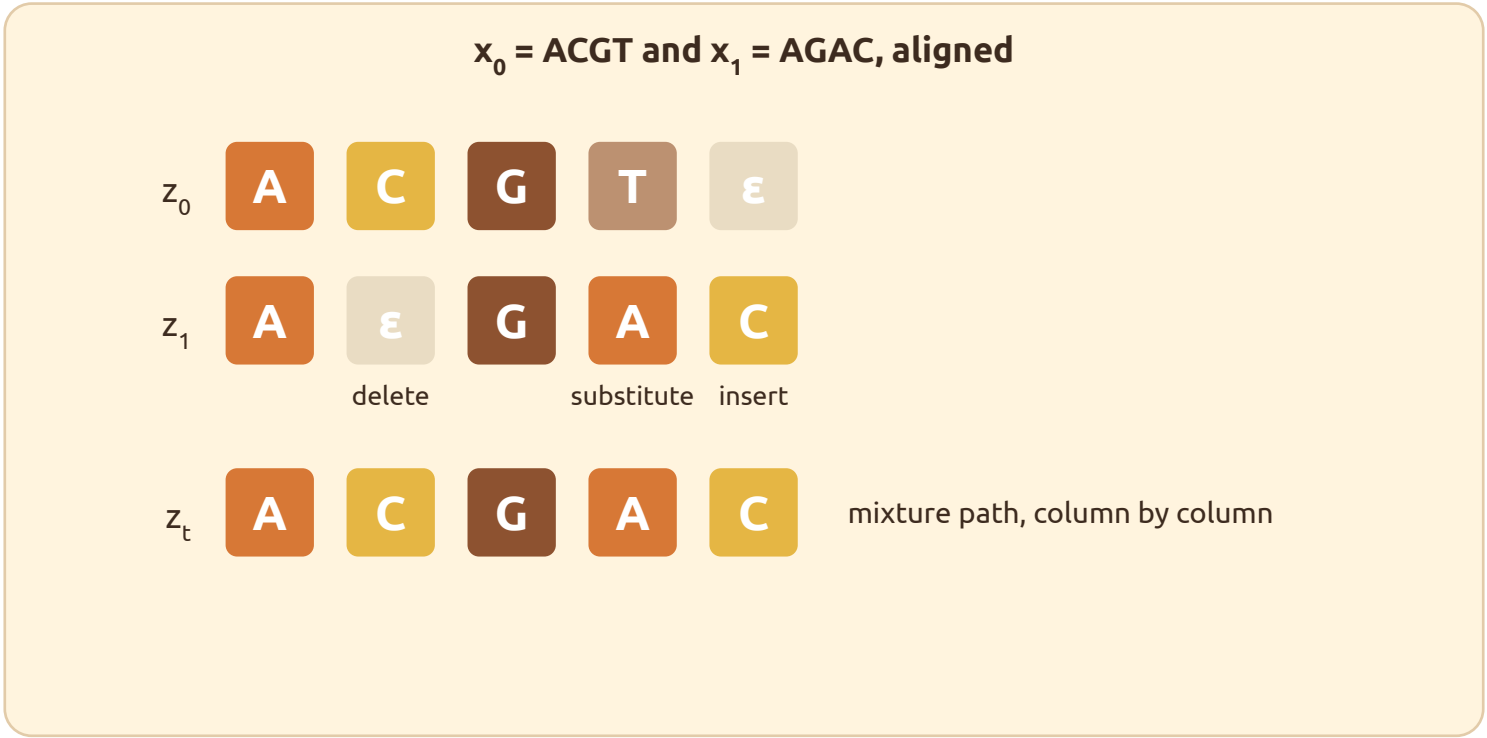
Different tokens in one column are a substitution



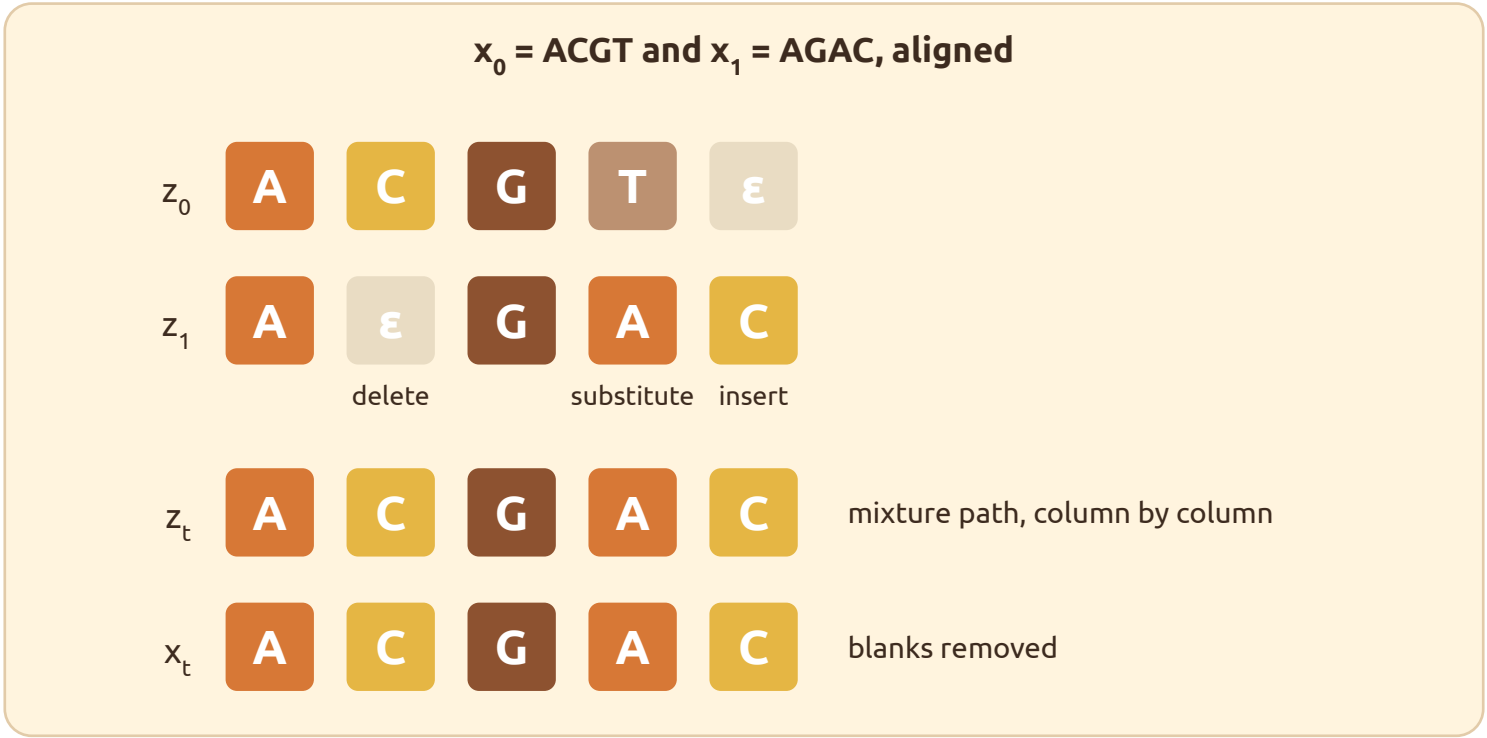
A blank in the source column is an insertion



On the aligned columns, each column follows our mixture path



Remove the blanks from z_t to get the sequence x_t



We average over the alignments we cannot see

We average over the alignments we cannot see

Several alignments can represent the same sequence pair. We cannot determine the current alignment from the noisy sequence x_t alone.

We average over the alignments we cannot see

Several alignments can represent the same sequence pair. We cannot determine the current alignment from the noisy sequence x_t alone.

By their Theorem 3.1, we marginalize the auxiliary alignment, as we marginalized x_1:

$$u_t(x \mid x_t) = \sum_z \mathbb{E}_{p_t(z_t|x_t)} \left[u_t(x, z \mid x_t, z_t) \right]$$

We average over the alignments we cannot see

Several alignments can represent the same sequence pair. We cannot determine the current alignment from the noisy sequence x_t alone.

By their Theorem 3.1, we marginalize the auxiliary alignment, as we marginalized x_1:

$$u_t(x \mid x_t) = \sum_z \mathbb{E}_{p_t(z_t|x_t)} \left[u_t(x, z \mid x_t, z_t) \right]$$

The marginal rate from x_t to x equals the sum over aligned states z of the expected rate on the joint space, taken over the posterior of the current aligned state z_t given the visible sequence x_t.

We average over the alignments we cannot see

Several alignments can represent the same sequence pair. We cannot determine the current alignment from the noisy sequence x_t alone.

By their Theorem 3.1, we marginalize the auxiliary alignment, as we marginalized x_1:

$$u_t(x \mid x_t) = \sum_z \mathbb{E}_{p_t(z_t|x_t)} \left[u_t(x, z \mid x_t, z_t) \right]$$

The marginal rate from x_t to x equals the sum over aligned states z of the expected rate on the joint space, taken over the posterior of the current aligned state z_t given the visible sequence x_t.

It is the posterior-averaging theorem again, with an unseen alignment in place of the unseen data sequence.

We average over the alignments we cannot see

Several alignments can represent the same sequence pair. We cannot determine the current alignment from the noisy sequence x_t alone.

By their Theorem 3.1, we marginalize the auxiliary alignment, as we marginalized x_1:

$$u_t(x \mid x_t) = \sum_z \mathbb{E}_{p_t(z_t|x_t)} \left[u_t(x, z \mid x_t, z_t) \right]$$

The marginal rate from x_t to x equals the sum over aligned states z of the expected rate on the joint space, taken over the posterior of the current aligned state z_t given the visible sequence x_t.

It is the posterior-averaging theorem again, with an unseen alignment in place of the unseen data sequence.

If we train on single sampled alignments, the gradients are the same as for the averaged rate.

We average over the alignments we cannot see

Several alignments can represent the same sequence pair. We cannot determine the current alignment from the noisy sequence x_t alone.

By their Theorem 3.1, we marginalize the auxiliary alignment, as we marginalized x_1:

$$u_t(x \mid x_t) = \sum_z \mathbb{E}_{p_t(z_t|x_t)} \left[u_t(x, z \mid x_t, z_t) \right]$$

The marginal rate from x_t to x equals the sum over aligned states z of the expected rate on the joint space, taken over the posterior of the current aligned state z_t given the visible sequence x_t.

It is the posterior-averaging theorem again, with an unseen alignment in place of the unseen data sequence.

A visible edit can correspond to several alignments. We average their rates using the posterior probabilities of those alignments.

Next, we write the training loss

Next, we write the training loss

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\sum_{x \neq x_t} u_t^\theta(x \mid x_t) \; - \; \sum_{i=1}^{N} \mathbb{1}\big[z_1^i \neq z_t^i\big] \, \frac{\dot{\kappa}_t}{1-\kappa_t} \, \log u_t^\theta\big(x(z_t, i, z_1^i) \mid x_t\big) \Big]$$

Next, we write the training loss

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\sum_{x \neq x_t} u_t^\theta(x \mid x_t) \; - \; \sum_{i=1}^{N} \mathbb{1}\big[z_1^i \neq z_t^i\big] \, \frac{\dot{\kappa}_t}{1-\kappa_t} \, \log u_t^\theta\big(x(z_t, i, z_1^i) \mid x_t\big) \Big]$$

The loss is the expected total rate that the model assigns to leaving x_t, minus the sum over unfinished aligned columns i of $\dot{\kappa}_t$ divided by one minus κ_t, times the log of the model rate of the edit that sets column i to its target.

Next, we write the training loss

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\sum_{x \neq x_t} u_t^\theta(x \mid x_t) \; - \; \sum_{i=1}^{N} \mathbb{1}\big[z_1^i \neq z_t^i\big] \, \frac{\dot{\kappa}_t}{1 - \kappa_t} \, \log u_t^\theta\big(x(z_t, i, z_1^i) \mid x_t\big) \Big]$$

The loss is the expected total rate that the model assigns to leaving x_t, minus the sum over unfinished aligned columns i of $\dot{\kappa}_t$ divided by one minus κ_t, times the log of the model rate of the edit that sets column i to its target.

The first term sums the model's intensities. The second is a weighted cross-entropy over edits that move x_t toward x_1.

Next, we write the training loss

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\sum_{x \neq x_t} u_t^\theta(x \mid x_t) \; - \; \sum_{i=1}^{N} \mathbb{1}\big[z_1^i \neq z_t^i\big] \, \frac{\dot{\kappa}_t}{1 - \kappa_t} \, \log u_t^\theta\big(x(z_t, i, z_1^i) \mid x_t\big) \Big]$$

The loss is the expected total rate that the model assigns to leaving x_t, minus the sum over unfinished aligned columns i of $\dot{\kappa}_t$ divided by one minus κ_t, times the log of the model rate of the edit that sets column i to its target.

The first term sums the model's intensities. The second is a weighted cross-entropy over edits that move x_t toward x_1.

With substitutions only and the mask source, the loss reduces to masked flow matching.

Next, we write the training loss

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\sum_{x \neq x_t} u_t^\theta(x \mid x_t) \; - \; \sum_{i=1}^{N} \mathbb{1}\big[z_1^i \neq z_t^i\big] \, \frac{\dot{\kappa}_t}{1 - \kappa_t} \, \log u_t^\theta\big(x(z_t, i, z_1^i) \mid x_t\big) \Big]$$

The loss is the expected total rate that the model assigns to leaving x_t, minus the sum over unfinished aligned columns i of $\dot{\kappa}_t$ divided by one minus κ_t, times the log of the model rate of the edit that sets column i to its target.

The first term sums the model's intensities. The second is a weighted cross-entropy over edits that move x_t toward x_1.

We train both when to edit and what to change by rewarding the required edits and penalizing excessive editing.

Now take one step of the sampler

Now take one step of the sampler

At each position, insert with probability $h\lambda^{ins}_{t,i}$, and delete or substitute with probability $h(\lambda^{del}_{t,i} + \lambda^{sub}_{t,i})$.

Now take one step of the sampler

At each position, insert with probability $h\lambda^{ins}_{t,i}$, and delete or substitute with probability $h(\lambda^{del}_{t,i} + \lambda^{sub}_{t,i})$.

We cannot both delete and substitute the same token, so we choose between those operations in proportion to their intensities.

Now take one step of the sampler

At each position, insert with probability $h\lambda^{ins}_{t,i}$, and delete or substitute with probability $h(\lambda^{del}_{t,i} + \lambda^{sub}_{t,i})$.

We cannot both delete and substitute the same token, so we choose between those operations in proportion to their intensities.

We draw new tokens from Q^{ins} or Q^{sub}, then apply the selected edits together.

Now take one step of the sampler

At each position, insert with probability $h\lambda^{ins}_{t,i}$, and delete or substitute with probability $h(\lambda^{del}_{t,i} + \lambda^{sub}_{t,i})$.

We cannot both delete and substitute the same token, so we choose between those operations in proportion to their intensities.

We draw new tokens from Q^{ins} or Q^{sub}, then apply the selected edits together.

For x_t = ACG, let $\lambda^{del} = 0.9$ and $\lambda^{sub} = 0.3$ at C. A step of $h = 0.05$ edits C with probability 0.06. Conditional on an edit, deletion has probability 0.75.

Edit Flows: what happens to ACG?

At the C in a toy ACG sequence, deletion has rate 0.9 and substitution has rate 0.3. Take a first-order step of 0.05, with no insertions. A substitution chooses T with probability 0.8 and A otherwise.

Edit Flows: what happens to ACG?

At the C in a toy ACG sequence, deletion has rate 0.9 and substitution has rate 0.3. Take a first-order step of 0.05, with no insertions. A substitution chooses T with probability 0.8 and A otherwise.

$$p(\text{delete}) = 0.05(0.9) = 0.045, \quad p(\text{substitute}) = 0.05(0.3) = 0.015, \quad p(\text{keep}) = 0.94$$

Edit Flows: what happens to ACG?

At the C in a toy ACG sequence, deletion has rate 0.9 and substitution has rate 0.3. Take a first-order step of 0.05, with no insertions. A substitution chooses T with probability 0.8 and A otherwise.

$$p(\text{delete}) = 0.05(0.9) = 0.045, \quad p(\text{substitute}) = 0.05(0.3) = 0.015, \quad p(\text{keep}) = 0.94$$

Deletion produces AG. A substitution edits the C without changing the sequence length.

Edit Flows: what happens to ACG?

At the C in a toy ACG sequence, deletion has rate 0.9 and substitution has rate 0.3. Take a first-order step of 0.05, with no insertions. A substitution chooses T with probability 0.8 and A otherwise.

$$p(\text{delete}) = 0.05(0.9) = 0.045, \quad p(\text{substitute}) = 0.05(0.3) = 0.015, \quad p(\text{keep}) = 0.94$$

Deletion produces AG. A substitution edits the C without changing the sequence length.

$$p(\text{ATG}) = p(\text{substitute})\, Q^{\text{sub}}(\text{T}) = (0.015)(0.8) = 0.012$$

Edit Flows: what happens to ACG?

At the C in a toy ACG sequence, deletion has rate 0.9 and substitution has rate 0.3. Take a first-order step of 0.05, with no insertions. A substitution chooses T with probability 0.8 and A otherwise.

$$p(\text{delete}) = 0.05(0.9) = 0.045, \quad p(\text{substitute}) = 0.05(0.3) = 0.015, \quad p(\text{keep}) = 0.94$$

Deletion produces AG. A substitution edits the C without changing the sequence length.

$$p(\text{ATG}) = p(\text{substitute})\, Q^{\text{sub}}(\text{T}) = (0.015)(0.8) = 0.012$$

The probability of ATG is 1.2%, while the probability of AG is 4.5%. ACG remains unchanged with probability 94%.

Edit Flows: what happens to ACG?

At the C in a toy ACG sequence, deletion has rate 0.9 and substitution has rate 0.3. Take a first-order step of 0.05, with no insertions. A substitution chooses T with probability 0.8 and A otherwise.

$$p(\text{delete}) = 0.05(0.9) = 0.045, \quad p(\text{substitute}) = 0.05(0.3) = 0.015, \quad p(\text{keep}) = 0.94$$

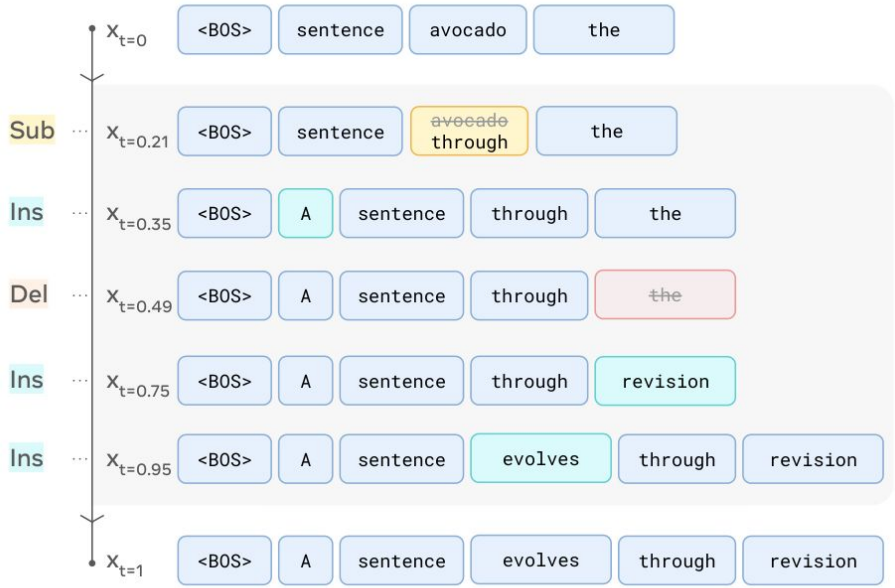
Deletion produces AG. A substitution edits the C without changing the sequence length.

$$p(\text{ATG}) = p(\text{substitute})\, Q^{\text{sub}}(\text{T}) = (0.015)(0.8) = 0.012$$

The probability of ATG is 1.2%, while the probability of AG is 4.5%. ACG remains unchanged with probability 94%.

We use the intensity to determine whether to edit. If the edit needs a new token, we draw it from the token distribution.

Each step is one insertion, deletion, or substitution



Generation from a short random sequence, with each edit labeled.

Havasi,..,Chen. *NeurIPS* (2025)

Now compare Edit Flows with a mask model on code

	Method	HumanEval		HumanEval+		MBPP	
		Pass@1	Pass@10	Pass@1	Pass@10	Pass@1	Pass@10
	Autoregressive (Gat et al., 2024)	14.3	21.3			17.0	34.3
	Llama3 Autoregressive†	17.0	34.7	14.0	28.6	25.6	45.4
Non-AR	Mask DFM (Gat et al., 2024)	6.7	13.4			6.7	20.6
Non-AR	Mask DFM (Oracle Length) (Gat et al., 2024)	11.6	18.3			13.1	28.4
Non-AR	Mask DFM†	9.1	17.6	7.9	13.4	6.2	25.0
Non-AR	Uniform X_0 + Edit Flow (**Ours**)	9.7	24.3	9.7	19.5	9.4	33.4
Non-AR	Edit Flow (**Ours**)	12.8	24.3	10.4	20.7	10.0	36.4
Non-AR	Localized Edit Flow (**Ours**)	14.0	22.6	10.4	18.9	14.8	34.0

Table 3: Code generation benchmarks using Llama3 1.3B parameter models trained on the CodeLlama (Roziere et al., 2023) datamix. †Superscript denotes our own implementation. We highlight the best non-autoregressive models, where colors show the best and second best among each metric.

Havasi,..,Chen. *NeurIPS* (2025)

A mask model of the same size solves 9.1% of HumanEval problems on the first try

	Method	HumanEval		HumanEval+		MBPP	
		Pass@1	Pass@10	Pass@1	Pass@10	Pass@1	Pass@10
	Autoregressive (Gat et al., 2024)	14.3	21.3			17.0	34.3
	Llama3 Autoregressive†	17.0	34.7	14.0	28.6	25.6	45.4
Non-AR	Mask DFM (Gat et al., 2024)	6.7	13.4			6.7	20.6
Non-AR	Mask DFM (Oracle Length) (Gat et al., 2024)	11.6	18.3			13.1	28.4
Non-AR	Mask DFM†	9.1	17.6	7.9	13.4	6.2	25.0
Non-AR	Uniform X_0 + Edit Flow (**Ours**)	9.7	24.3	9.7	19.5	9.4	33.4
Non-AR	Edit Flow (**Ours**)	12.8	24.3	10.4	20.7	10.0	36.4
Non-AR	Localized Edit Flow (**Ours**)	14.0	22.6	10.4	18.9	14.8	34.0

Table 3: Code generation benchmarks using Llama3 1.3B parameter models trained on the CodeLlama (Roziere et al., 2023) datamix. †Superscript denotes our own implementation. We highlight the best non-autoregressive models, where colors show the best and second best among each metric.

Havasi,..,Chen. *NeurIPS* (2025)

Edit Flows solve 12.8%, more than the mask model given the true length

	Method	HumanEval		HumanEval+		MBPP	
		Pass@1	Pass@10	Pass@1	Pass@10	Pass@1	Pass@10
	Autoregressive (Gat et al., 2024)	14.3	21.3			17.0	34.3
	Llama3 Autoregressive†	17.0	34.7	14.0	28.6	25.6	45.4
Non-AR	Mask DFM (Gat et al., 2024)	6.7	13.4			6.7	20.6
Non-AR	Mask DFM (Oracle Length) (Gat et al., 2024)	11.6	18.3			13.1	28.4
Non-AR	Mask DFM†	9.1	17.6	7.9	13.4	6.2	25.0
Non-AR	Uniform X_0 + Edit Flow (**Ours**)	9.7	24.3	9.7	19.5	9.4	33.4
Non-AR	Edit Flow (**Ours**)	12.8	24.3	10.4	20.7	10.0	36.4
Non-AR	Localized Edit Flow (**Ours**)	14.0	22.6	10.4	18.9	14.8	34.0

Table 3: Code generation benchmarks using Llama3 1.3B parameter models trained on the CodeLlama (Roziere et al., 2023) datamix. †Superscript denotes our own implementation. We highlight the best non-autoregressive models, where colors show the best and second best among each metric.

Havasi,..,Chen. *NeurIPS* (2025)

Letting insertions favor neighbors of existing tokens raises MBPP from 10.0% to 14.8%

	Method	HumanEval		HumanEval+		MBPP	
		Pass@1	Pass@10	Pass@1	Pass@10	Pass@1	Pass@10
	Autoregressive (Gat et al., 2024)	14.3	21.3			17.0	34.3
	Llama3 Autoregressive†	17.0	34.7	14.0	28.6	25.6	45.4
Non-AR	Mask DFM (Gat et al., 2024)	6.7	13.4			6.7	20.6
Non-AR	Mask DFM (Oracle Length) (Gat et al., 2024)	11.6	18.3			13.1	28.4
Non-AR	Mask DFM†	9.1	17.6	7.9	13.4	6.2	25.0
Non-AR	Uniform X_0 + Edit Flow (**Ours**)	9.7	24.3	9.7	19.5	9.4	33.4
Non-AR	Edit Flow (**Ours**)	12.8	24.3	10.4	20.7	10.0	36.4
Non-AR	Localized Edit Flow (**Ours**)	14.0	22.6	10.4	18.9	14.8	34.0

Table 3: Code generation benchmarks using Llama3 1.3B parameter models trained on the CodeLlama (Roziere et al., 2023) datamix. †Superscript denotes our own implementation. We highlight the best non-autoregressive models, where colors show the best and second best among each metric.

Havasi,..,Chen. *NeurIPS* (2025)

We can use the same rate-based construction on sequences of varying length

	Method	HumanEval		HumanEval+		MBPP	
		Pass@1	Pass@10	Pass@1	Pass@10	Pass@1	Pass@10
	Autoregressive (Gat et al., 2024)	14.3	21.3			17.0	34.3
	Llama3 Autoregressive†	17.0	34.7	14.0	28.6	25.6	45.4
Non-AR	Mask DFM (Gat et al., 2024)	6.7	13.4			6.7	20.6
Non-AR	Mask DFM (Oracle Length) (Gat et al., 2024)	11.6	18.3			13.1	28.4
Non-AR	Mask DFM†	9.1	17.6	7.9	13.4	6.2	25.0
Non-AR	Uniform X_0 + Edit Flow (**Ours**)	9.7	24.3	9.7	19.5	9.4	33.4
Non-AR	Edit Flow (**Ours**)	12.8	24.3	10.4	20.7	10.0	36.4
Non-AR	Localized Edit Flow (**Ours**)	14.0	22.6	10.4	18.9	14.8	34.0

Table 3: Code generation benchmarks using Llama3 1.3B parameter models trained on the CodeLlama (Roziere et al., 2023) datamix. †Superscript denotes our own implementation. We highlight the best non-autoregressive models, where colors show the best and second best among each metric.

Havasi,..,Chen. *NeurIPS* (2025)

Summary: velocities, paths, and the denoiser

Summary: velocities, paths, and the denoiser

Velocities are rates: $u_t(y, x) \geq 0 \ (y \neq x), \quad \sum_y u_t(y, x) = 0$

Summary: velocities, paths, and the denoiser

Velocities are rates: $u_t(y, x) \geq 0 \ (y \neq x), \quad \sum_y u_t(y, x) = 0$

A probability velocity has nonnegative rates toward other states. Including the diagonal, its rates from each state sum to zero.

Summary: velocities, paths, and the denoiser

Velocities are rates: $$u_t(y, x) \geq 0 \ (y \neq x), \quad \sum_y u_t(y, x) = 0$$

They generate a path: $$\frac{d}{dt} p_t(y) = \sum_x u_t(y, x)\, p_t(x)$$

Summary: velocities, paths, and the denoiser

Velocities are rates: $$u_t(y, x) \geq 0 \ (y \neq x), \quad \sum_y u_t(y, x) = 0$$

They generate a path: $$\frac{d}{dt} p_t(y) = \sum_x u_t(y, x)\, p_t(x)$$

A velocity generates p_t when the rate of change of $p_t(y)$ equals the sum over x of $u_t(y, x)$ times $p_t(x)$.

Summary: velocities, paths, and the denoiser

Velocities are rates: $$u_t(y, x) \geq 0 \ (y \neq x), \quad \sum_y u_t(y, x) = 0$$

They generate a path: $$\frac{d}{dt} p_t(y) = \sum_x u_t(y, x)\, p_t(x)$$

Mixture paths: $$p_t(x^i \mid x_0, x_1) = (1 - \kappa_t)\, \delta_{x_0^i} + \kappa_t\, \delta_{x_1^i}$$

Summary: velocities, paths, and the denoiser

Velocities are rates: $$u_t(y, x) \geq 0 \ (y \neq x), \quad \sum_y u_t(y, x) = 0$$

They generate a path: $$\frac{d}{dt} p_t(y) = \sum_x u_t(y, x)\, p_t(x)$$

Mixture paths: $$p_t(x^i \mid x_0, x_1) = (1 - \kappa_t)\, \delta_{x_0^i} + \kappa_t\, \delta_{x_1^i}$$

Each position shows its source token with probability one minus κ_t and its data token with probability κ_t.

Summary: training, sampling, and correctors

Summary: training, sampling, and correctors

Posterior average: $$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i} \right]$$

Summary: training, sampling, and correctors

Posterior average: $$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i} \right]$$

The marginal velocity is the schedule rate times the denoiser distribution minus the point mass on the current token.

Summary: training, sampling, and correctors

Posterior average:

$$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i} \right]$$

Training:

$$\mathcal{L} = -\,\mathbb{E} \sum_i \log p^{\theta}_{1|t}(X_1^i \mid X_t)$$

Summary: training, sampling, and correctors

Posterior average:

$$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i} \right]$$

Training:

$$\mathcal{L} = -\,\mathbb{E} \sum_i \log p^{\theta}_{1|t}(X_1^i \mid X_t)$$

We train the denoiser by cross-entropy on the true data token at every position.

Summary: training, sampling, and correctors

Posterior average:

$$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i} \right]$$

Training:

$$\mathcal{L} = -\,\mathbb{E} \sum_i \log p^\theta_{1|t}(X_1^i \mid X_t)$$

Correctors:

$$\bar{u}_t = \hat{u}_t + \beta_t \left(\hat{u}_t - \breve{u}_t \right)$$

Summary: training, sampling, and correctors

Posterior average: $$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i} \right]$$

Training: $$\mathcal{L} = -\,\mathbb{E} \sum_i \log p^\theta_{1|t}(X_1^i \mid X_t)$$

Correctors: $$\bar{u}_t = \hat{u}_t + \beta_t \left(\hat{u}_t - \check{u}_t \right)$$

The corrector velocity $\bar{u}_t$ equals the forward velocity $\hat{u}_t$ plus β_t times the difference between the forward velocity $\hat{u}_t$ and the backward velocity $\check{u}_t$.

Continuous representations of discrete tokens

Continuous representations of discrete tokens

So far, each position has held one of four bases. We described its changes through jump rates.

Continuous representations of discrete tokens

So far, each position has held one of four bases. We described its changes through jump rates.

The master equation, the posterior average, and the corrector all work on that discrete set.

Continuous representations of discrete tokens

So far, each position has held one of four bases. We described its changes through jump rates.

The master equation, the posterior average, and the corrector all work on that discrete set.

With continuous flow matching, we can use ODE solvers, classifier guidance, and distillation to one step.

Now we relax a token into a point we can move continuously

Now we relax a token into a point we can move continuously

A base becomes a probability vector over A, C, G, and T.

Now we relax a token into a point we can move continuously

A base becomes a probability vector over A, C, G, and T.

Certainty about A corresponds to the corner (1, 0, 0, 0), while complete uncertainty corresponds to the center (0.25, 0.25, 0.25, 0.25).

Now we relax a token into a point we can move continuously

A base becomes a probability vector over A, C, G, and T.

Certainty about A corresponds to the corner (1, 0, 0, 0), while complete uncertainty corresponds to the center (0.25, 0.25, 0.25, 0.25).

Generation becomes a continuous curve from the center to a corner, driven by an ordinary velocity field.

Now we relax a token into a point we can move continuously

A base becomes a probability vector over A, C, G, and T.

Certainty about A corresponds to the corner (1, 0, 0, 0), while complete uncertainty corresponds to the center (0.25, 0.25, 0.25, 0.25).

Generation becomes a continuous curve from the center to a corner, driven by an ordinary velocity field.

At the end we read off the token by taking the largest coordinate.

The simplex is the set of probability vectors

The simplex is the set of probability vectors

With K letters in the alphabet, $$S_K = \{ x \in \mathbb{R}^K \ : \ \sum_k x_k = 1, \ x_k \geq 0 \}$$

The simplex is the set of probability vectors

With K letters in the alphabet, $$S_K = \{ x \in \mathbb{R}^K \ : \ \sum_k x_k = 1, \ x_k \geq 0 \}$$

The simplex S_K is the set of vectors x in K-dimensional space whose coordinates sum to one and are all nonnegative.

The simplex is the set of probability vectors

With K letters in the alphabet, $$S_K = \{ x \in \mathbb{R}^K \ : \ \sum_k x_k = 1, \ x_k \geq 0 \}$$

The simplex S_K is the set of vectors x in K-dimensional space whose coordinates sum to one and are all nonnegative.

For DNA, K = 4 and S_4 is a solid tetrahedron with corners A, C, G, and T.

The simplex is the set of probability vectors

With K letters in the alphabet, $$S_K = \{ x \in \mathbb{R}^K \ : \ \sum_k x_k = 1, \ x_k \geq 0 \}$$

The simplex S_K is the set of vectors x in K-dimensional space whose coordinates sum to one and are all nonnegative.

For DNA, K = 4 and S_4 is a solid tetrahedron with corners A, C, G, and T.

The corners are the one-hot vectors: $e_k \in S_K, \qquad (e_k)_j = \delta_{jk}$

The simplex is the set of probability vectors

With K letters in the alphabet, $$S_K = \{\, x \in \mathbb{R}^K \;:\; \textstyle\sum_k x_k = 1,\; x_k \geq 0 \,\}$$

The simplex S_K is the set of vectors x in K-dimensional space whose coordinates sum to one and are all nonnegative.

For DNA, K = 4 and S_4 is a solid tetrahedron with corners A, C, G, and T.

The corners are the one-hot vectors: $$e_k \in S_K, \qquad (e_k)_j = \delta_{jk}$$

The corner e_k is the vector whose k-th coordinate is one and whose other coordinates are zero.

The simplex is the set of probability vectors

With K letters in the alphabet, $$S_K = \{\, x \in \mathbb{R}^K \;:\; \textstyle\sum_k x_k = 1,\; x_k \geq 0 \,\}$$

The simplex S_K is the set of vectors x in K-dimensional space whose coordinates sum to one and are all nonnegative.

For DNA, K = 4 and S_4 is a solid tetrahedron with corners A, C, G, and T.

The corners are the one-hot vectors: $e_k \in S_K, \qquad (e_k)_j = \delta_{jk}$

The sum constraint removes one degree of freedom, so S_K is a flat piece of a K – 1 dimensional plane.

Data tokens occupy the simplex corners

Data tokens occupy the simplex corners

Read a categorical distribution over letters on the simplex: $$q(x) = \sum_{k=1}^{K} q_k \, \delta(x - e_k)$$

Data tokens occupy the simplex corners

Read a categorical distribution over letters on the simplex: $$q(x) = \sum_{k=1}^{K} q_k \, \delta(x - e_k)$$

The data distribution q(x) is the sum over letters k of the probability q_k times a point mass at the corner e_k.

Data tokens occupy the simplex corners

Read a categorical distribution over letters on the simplex: $$q(x) = \sum_{k=1}^{K} q_k \, \delta(x - e_k)$$

The data distribution q(x) is the sum over letters k of the probability q_k times a point mass at the corner e_k.

We keep data tokens at the corners, so we lose no information in the relaxation.

Data tokens occupy the simplex corners

Read a categorical distribution over letters on the simplex: $$q(x) = \sum_{k=1}^{K} q_k \, \delta(x - e_k)$$

The data distribution q(x) is the sum over letters k of the probability q_k times a point mass at the corner e_k.

We keep data tokens at the corners, so we lose no information in the relaxation.

A sequence of N positions is a point in the product S_K^N, one simplex per position.

A path now has to run from the center to a corner

Read a categorical distribution over letters on the simplex: $$q(x) = \sum_{k=1}^{K} q_k \, \delta(x - e_k)$$

The data distribution q(x) is the sum over letters k of the probability q_k times a point mass at the corner e_k.

We keep data tokens at the corners, so we lose no information in the relaxation.

A sequence of N positions is a point in the product S_K^N, one simplex per position.

A straight-line path on the simplex

A straight-line path on the simplex

Interpolate a source point and a corner: $\psi_t(x_0 \mid x_1) = (1 - t)\, x_0 + t\, x_1$

A straight-line path on the simplex

Interpolate a source point and a corner: $$\psi_t(x_0 \mid x_1) = (1 - t)\, x_0 + t\, x_1$$

The straight-line interpolant ψ_t equals one minus t times the source point x_0, plus t times the data corner x_1.

A straight-line path on the simplex

Interpolate a source point and a corner: $\psi_t(x_0 \mid x_1) = (1 - t)\, x_0 + t\, x_1$

The straight-line interpolant ψ_t equals one minus t times the source point x_0, plus t times the data corner x_1.

Its velocity is the one we know: $u_t(x \mid x_1) = \dfrac{x_1 - x}{1 - t}$

A straight-line path on the simplex

Interpolate a source point and a corner: $\psi_t(x_0 \mid x_1) = (1 - t)\, x_0 + t\, x_1$

The straight-line interpolant ψ_t equals one minus t times the source point x_0, plus t times the data corner x_1.

Its velocity is the one we know: $u_t(x \mid x_1) = \dfrac{x_1 - x}{1 - t}$

The conditional velocity at the point x equals the displacement x_1 – x that remains to the corner, divided by the time 1 – t that remains.

A straight-line path on the simplex

Interpolate a source point and a corner: $\psi_t(x_0 \mid x_1) = (1 - t)\, x_0 + t\, x_1$

The straight-line interpolant ψ_t equals one minus t times the source point x_0, plus t times the data corner x_1.

Its velocity is the one we know: $u_t(x \mid x_1) = \dfrac{x_1 - x}{1 - t}$

The conditional velocity at the point x equals the displacement x_1 – x that remains to the corner, divided by the time 1 – t that remains.

The entire path stays inside the simplex because every point is a convex combination of two simplex points.

Draw the source point uniformly on the simplex

Draw the source point uniformly on the simplex

Draw x_0 uniformly on the simplex. The interpolant shrinks the whole simplex toward the corner e_i by the factor $1 - t$.

Draw the source point uniformly on the simplex

Draw x_0 uniformly on the simplex. The interpolant shrinks the whole simplex toward the corner e_i by the factor 1 – t.

$$p_t(x \mid x_1 = e_i) = \begin{cases} \left[(1-t)^{K-1}\,\Gamma(K)\right]^{-1} & x_i \geq t \\ 0 & x_i < t \end{cases}$$

Draw the source point uniformly on the simplex

Draw x_0 uniformly on the simplex. The interpolant shrinks the whole simplex toward the corner e_i by the factor 1 – t.

$$p_t(x \mid x_1 = e_i) = \begin{cases} \left[(1-t)^{K-1}\,\Gamma(K)\right]^{-1} & x_i \geq t \\ 0 & x_i < t \end{cases}$$

The conditional density is constant on the region where the coordinate x_i is at least t, with value one over the quantity one minus t raised to the K – 1, times Γ(K), and it is zero where x_i is below t.

Draw the source point uniformly on the simplex

Draw x_0 uniformly on the simplex. The interpolant shrinks the whole simplex toward the corner e_i by the factor 1 – t.

$$p_t(x \mid x_1 = e_i) = \begin{cases} \left[(1-t)^{K-1}\,\Gamma(K)\right]^{-1} & x_i \geq t \\ 0 & x_i < t \end{cases}$$

The conditional density is constant on the region where the coordinate x_i is at least t, with value one over the quantity one minus t raised to the K – 1, times Γ(K), and it is zero where x_i is below t.

The support is the smaller simplex $\{x_i \geq t\}$. We normalize its constant density to keep the total probability equal to one.

Draw the source point uniformly on the simplex

Draw x_0 uniformly on the simplex. The interpolant shrinks the whole simplex toward the corner e_i by the factor 1 – t.

$$p_t(x \mid x_1 = e_i) = \begin{cases} \left[(1-t)^{K-1}\,\Gamma(K)\right]^{-1} & x_i \geq t \\ 0 & x_i < t \end{cases}$$

The conditional density is constant on the region where the coordinate x_i is at least t, with value one over the quantity one minus t raised to the K – 1, times Γ(K), and it is zero where x_i is below t.

The support is the smaller simplex $\{x_i \geq t\}$. We normalize its constant density to keep the total probability equal to one.

A point x with $x_i < t$ could not have come from corner e_i, so the posterior over corners gives e_i probability zero.

Could a smoother probability path keep alternative tokens available for longer?

Dirichlet Flow Matching

Dirichlet Flow Matching with Applications to DNA Sequence Design

Hannes Stark [*1] **Bowen Jing** [*1] **Chenyu Wang** [1] **Gabriele Corso** [1]
Bonnie Berger [1 2] **Regina Barzilay** [1] **Tommi Jaakkola** [1]

Stark,..,Jaakkola. *ICML* (2024)

How many data corners remain possible?

How many data corners remain possible?

Proposition 1: with the straight-line path, the posterior over corners has support on at most $k - 1$ corners once $t > 1/k$.

How many data corners remain possible?

Proposition 1: with the straight-line path, the posterior over corners has support on at most k – 1 corners once t > 1/k.

Suppose k corners are still possible at time t > 1/k:

$$\mathbf{1}^\top x = \sum_{j=1}^{K} x_j \;\geq\; \sum_{j=1}^{k} x_j \;\geq\; k\,t \;>\; k \cdot \frac{1}{k} = 1$$

How many data corners remain possible?

Proposition 1: with the straight-line path, the posterior over corners has support on at most k – 1 corners once t > 1/k.

Suppose k corners are still possible at time t > 1/k:

$$\mathbf{1}^\top x = \sum_{j=1}^{K} x_j \;\geq\; \sum_{j=1}^{k} x_j \;\geq\; k\,t \;>\; k \cdot \frac{1}{k} = 1$$

The sum of all coordinates of x is at least the sum of the k coordinates that are still possible, each of which is at least t, so the total exceeds k times one over k, which is one.

How many data corners remain possible?

Proposition 1: with the straight-line path, the posterior over corners has support on at most k – 1 corners once t > 1/k.

Suppose k corners are still possible at time t > 1/k:

$$\mathbf{1}^\top x = \sum_{j=1}^{K} x_j \;\geq\; \sum_{j=1}^{k} x_j \;\geq\; k\,t \;>\; k \cdot \frac{1}{k} = 1$$

The sum of all coordinates of x is at least the sum of the k coordinates that are still possible, each of which is at least t, so the total exceeds k times one over k, which is one.

A simplex point sums to exactly one, so k corners cannot all stay possible. The assumption fails.

Let’s put numbers on that for DNA

Let’s put numbers on that for DNA

For DNA, one base is eliminated at t = 1/4, another at t = 1/3, and a third at t = 1/2.

Let’s put numbers on that for DNA

For DNA, one base is eliminated at t = 1/4, another at t = 1/3, and a third at t = 1/2.

At t = 0.3 with x = (0.34, 0.33, 0.33, 0.00), only A, C, and G remain possible, which is the bound for k = 4.

Let’s put numbers on that for DNA

For DNA, one base is eliminated at t = 1/4, another at t = 1/3, and a third at t = 1/2.

At t = 0.3 with x = (0.34, 0.33, 0.33, 0.00), only A, C, and G remain possible, which is the bound for k = 4.

At t = 0.6 with x = (0.70, 0.30, 0, 0), only A remains, so the posterior is the argmax.

Let’s put numbers on that for DNA

For DNA, one base is eliminated at t = 1/4, another at t = 1/3, and a third at t = 1/2.

At t = 0.3 with x = (0.34, 0.33, 0.33, 0.00), only A, C, and G remain possible, which is the bound for k = 4.

At t = 0.6 with x = (0.70, 0.30, 0, 0), only A remains, so the posterior is the argmax.

The target distribution is discontinuous in both time and space. Numerical integration is therefore sensitive to the step size.

Recall the Dirichlet distribution

Recall the Dirichlet distribution

Take a density on the simplex with one parameter per letter:

$$\mathrm{Dir}(x; \alpha_1, \ldots, \alpha_K) = \frac{1}{\mathcal{B}(\alpha)} \prod_{k=1}^{K} x_k^{\alpha_k - 1}$$

Recall the Dirichlet distribution

Take a density on the simplex with one parameter per letter:

$$\mathrm{Dir}(x; \alpha_1, \ldots, \alpha_K) = \frac{1}{\mathcal{B}(\alpha)} \prod_{k=1}^{K} x_k^{\alpha_k - 1}$$

The Dirichlet density at the point x is one over the multivariate beta function of α, times the product over letters of x_k raised to the power $\alpha_k - 1$.

Recall the Dirichlet distribution

Take a density on the simplex with one parameter per letter:

$$\mathrm{Dir}(x; \alpha_1, \ldots, \alpha_K) = \frac{1}{\mathcal{B}(\alpha)} \prod_{k=1}^{K} x_k^{\alpha_k - 1}$$

The Dirichlet density at the point x is one over the multivariate beta function of α, times the product over letters of x_k raised to the power $\alpha_k - 1$.

Setting every concentration to one gives the uniform distribution on the simplex.

Recall the Dirichlet distribution

Take a density on the simplex with one parameter per letter:

$$\mathrm{Dir}(x; \alpha_1, \ldots, \alpha_K) = \frac{1}{\mathcal{B}(\alpha)} \prod_{k=1}^{K} x_k^{\alpha_k - 1}$$

The Dirichlet density at the point x is one over the multivariate beta function of α, times the product over letters of x_k raised to the power $\alpha_k - 1$.

Setting every concentration to one gives the uniform distribution on the simplex.

Increasing one concentration places more probability near the corresponding corner.

Now replace the straight line with a Dirichlet family

Now replace the straight line with a Dirichlet family

Replace the straight line with a family of Dirichlet distributions:

$$\boxed{p_t(x \mid x_1 = e_i) = \mathrm{Dir}\big(x;\ \alpha = \mathbf{1} + t\, e_i\big)}$$

Now replace the straight line with a Dirichlet family

Replace the straight line with a family of Dirichlet distributions:

$$\boxed{p_t(x \mid x_1 = e_i) = \mathrm{Dir}\big(x;\ \alpha = \mathbf{1} + t\, e_i\big)}$$

The conditional path at time t is the Dirichlet distribution over the simplex whose concentration parameter is the all-ones vector plus t times the one-hot vector at the data corner e_i.

Now replace the straight line with a Dirichlet family

Replace the straight line with a family of Dirichlet distributions:

$$\boxed{p_t(x \mid x_1 = e_i) = \mathrm{Dir}\big(x;\ \alpha = \mathbf{1} + t\, e_i\big)}$$

The conditional path at time t is the Dirichlet distribution over the simplex whose concentration parameter is the all-ones vector plus t times the one-hot vector at the data corner e_i.

The i-th concentration is 1 + t, while the others remain at 1. As t increases, we place more probability near the data corner.

Now replace the straight line with a Dirichlet family

Replace the straight line with a family of Dirichlet distributions:

$$\boxed{p_t(x \mid x_1 = e_i) = \mathrm{Dir}\big(x;\ \alpha = \mathbf{1} + t\, e_i\big)}$$

The conditional path at time t is the Dirichlet distribution over the simplex whose concentration parameter is the all-ones vector plus t times the one-hot vector at the data corner e_i.

The i-th concentration is 1 + t, while the others remain at 1. As t increases, we place more probability near the data corner.

At t = 0, we start generation from the uniform distribution on the simplex.

Substitute that concentration into the density

Substitute that concentration into the density

Substitute $\alpha = 1 + t\, e_i$ into the density:

$$p_t(x \mid x_1 = e_i) = \frac{\Gamma(t+K)}{\Gamma(t+1)}\, x_i^t$$

Substitute that concentration into the density

Substitute $\alpha = 1 + t\, e_i$ into the density:

$$p_t(x \mid x_1 = e_i) = \frac{\Gamma(t+K)}{\Gamma(t+1)}\, x_i^t$$

The conditional density equals $\Gamma(t + K)$ divided by $\Gamma(t + 1)$, times the data coordinate x_i raised to the power t.

Substitute that concentration into the density

Substitute $\alpha = 1 + t\, e_i$ into the density:

$$p_t(x \mid x_1 = e_i) = \frac{\Gamma(t+K)}{\Gamma(t+1)}\, x_i^t$$

The conditional density equals $\Gamma(t + K)$ divided by $\Gamma(t + 1)$, times the data coordinate x_i raised to the power t.

Only the i-th exponent $\alpha_k - 1$ is nonzero, with value t. The beta function reduces to $\Gamma(t + 1)$ divided by $\Gamma(t + K)$.

Substitute that concentration into the density

Substitute $\alpha = 1 + t\, e_i$ into the density:

$$p_t(x \mid x_1 = e_i) = \frac{\Gamma(t+K)}{\Gamma(t+1)}\, x_i^t$$

The conditional density equals $\Gamma(t + K)$ divided by $\Gamma(t + 1)$, times the data coordinate x_i raised to the power t.

Only the i-th exponent $\alpha_k - 1$ is nonzero, with value t. The beta function reduces to $\Gamma(t + 1)$ divided by $\Gamma(t + K)$.

At $t = 0$ the density is the constant $\Gamma(K)$, which is the uniform distribution on the simplex. For DNA that constant is 6.

Substitute that concentration into the density

Substitute $\alpha = 1 + t\, e_i$ into the density:

$$p_t(x \mid x_1 = e_i) = \frac{\Gamma(t+K)}{\Gamma(t+1)}\, x_i^t$$

The conditional density equals $\Gamma(t + K)$ divided by $\Gamma(t + 1)$, times the data coordinate x_i raised to the power t.

Only the i-th exponent $\alpha_k - 1$ is nonzero, with value t. The beta function reduces to $\Gamma(t + 1)$ divided by $\Gamma(t + K)$.

At $t = 0$ the density is the constant $\Gamma(K)$, which is the uniform distribution on the simplex. For DNA that constant is 6.

For every finite t the density is positive wherever $x_i > 0$, so no corner is ever ruled out.

Substitute that concentration into the density

Substitute $\alpha = 1 + t\, e_i$ into the density:

$$p_t(x \mid x_1 = e_i) = \frac{\Gamma(t+K)}{\Gamma(t+1)}\, x_i^t$$

The conditional density equals $\Gamma(t + K)$ divided by $\Gamma(t + 1)$, times the data coordinate x_i raised to the power t.

Only the i-th exponent $\alpha_k - 1$ is nonzero, with value t. The beta function reduces to $\Gamma(t + 1)$ divided by $\Gamma(t + K)$.

At $t = 0$ the density is the constant $\Gamma(K)$, which is the uniform distribution on the simplex. For DNA that constant is 6.

Every corner remains possible at finite time, so a small change in the state need not rule out a token abruptly.

Time runs over the whole half-line

Time runs over the whole half-line

The Dirichlet concentration 1 + t reaches a point mass at the corner only as t grows without bound.

Time runs over the whole half-line

The Dirichlet concentration 1 + t reaches a point mass at the corner only as t grows without bound.

$$t \in [0, \infty), \qquad t \sim \mathrm{Exp}(1) \text{ in training}, \qquad t_{\max} = 8 \text{ at inference}$$

Time runs over the whole half-line

The Dirichlet concentration 1 + t reaches a point mass at the corner only as t grows without bound.

$$t \in [0, \infty), \qquad t \sim \mathrm{Exp}(1) \text{ in training}, \qquad t_{\max} = 8 \text{ at inference}$$

Time ranges over the nonnegative half-line. During training, we draw t from an exponential distribution with rate one. During inference, we integrate up to t = 8.

Time runs over the whole half-line

The Dirichlet concentration 1 + t reaches a point mass at the corner only as t grows without bound.

$$t \in [0, \infty), \qquad t \sim \mathrm{Exp}(1) \text{ in training}, \qquad t_{\max} = 8 \text{ at inference}$$

Time ranges over the nonnegative half-line. During training, we draw t from an exponential distribution with rate one. During inference, we integrate up to t = 8.

We still use x_1 for the data sample. Its subscript no longer denotes time one.

Time runs over the whole half-line

The Dirichlet concentration 1 + t reaches a point mass at the corner only as t grows without bound.

$$t \in [0, \infty), \qquad t \sim \mathrm{Exp}(1) \text{ in training}, \qquad t_{\max} = 8 \text{ at inference}$$

Time ranges over the nonnegative half-line. During training, we draw t from an exponential distribution with rate one. During inference, we integrate up to t = 8.

We still use x_1 for the data sample. Its subscript no longer denotes time one.

In Dirichlet Flow Matching we take 100 Euler steps up to t = 8 and then read the token off the final posterior.

Next, we need the velocity for this path

Next, we need the velocity for this path

Keep the direction and rescale the speed:

$$\boxed{u_t(x \mid x_1 = e_i) = C(x_i, t)\,(e_i - x)}$$

Next, we need the velocity for this path

Keep the direction and rescale the speed:

$$u_t(x \mid x_1 = e_i) = C(x_i, t)\,(e_i - x)$$

The conditional velocity at x equals a scalar speed $C(x_i, t)$, which depends on the data coordinate and the time, times the direction e_i – x from the current point to the data corner.

Next, we need the velocity for this path

Keep the direction and rescale the speed:

$$u_t(x \mid x_1 = e_i) = C(x_i, t)\,(e_i - x)$$

The conditional velocity at x equals a scalar speed $C(x_i, t)$, which depends on the data coordinate and the time, times the direction e_i – x from the current point to the data corner.

The straight-line path is the special case C = 1/(1 – t), a function of time alone.

Next, we need the velocity for this path

Keep the direction and rescale the speed:

$$u_t(x \mid x_1 = e_i) = C(x_i, t)\,(e_i - x)$$

The conditional velocity at x equals a scalar speed $C(x_i, t)$, which depends on the data coordinate and the time, times the direction $e_i - x$ from the current point to the data corner.

The straight-line path is the special case $C = 1/(1 - t)$, a function of time alone.

The direction $e_i - x$ has coordinates summing to zero, so the velocity is tangent to the simplex and the point stays on it.

Now cut the simplex with a plane

Now cut the simplex with a plane

Cut the simplex at $x_i = b$. The region S_1 contains the points with $x_i < b$.

Now cut the simplex with a plane

Cut the simplex at $x_i = b$. The region S_1 contains the points with $x_i < b$.

Probability crossing that plane must equal the rate at which S_1 loses probability.

Now cut the simplex with a plane

Cut the simplex at $x_i = b$. The region S_1 contains the points with $x_i < b$.

Probability crossing that plane must equal the rate at which S_1 loses probability.

The mass below the cut is a Beta CDF:

$$p_t(S_1) = \frac{\mathcal{B}(b;\, t+1,\, K-1)}{\mathcal{B}(t+1,\, K-1)} = I_b(t+1,\, K-1)$$

Now cut the simplex with a plane

Cut the simplex at $x_i = b$. The region S_1 contains the points with $x_i < b$.

Probability crossing that plane must equal the rate at which S_1 loses probability.

The mass below the cut is a Beta CDF:

$$p_t(S_1) = \frac{\mathcal{B}(b;\, t+1,\, K-1)}{\mathcal{B}(t+1,\, K-1)} = I_b(t+1,\, K-1)$$

The probability of the region S_1 equals the incomplete beta function at b with parameters t + 1 and K – 1, divided by the complete beta function, which is the cumulative distribution function of a Beta distribution evaluated at b.

Now cut the simplex with a plane

Cut the simplex at $x_i = b$. The region S_1 contains the points with $x_i < b$.

Probability crossing that plane must equal the rate at which S_1 loses probability.

The mass below the cut is a Beta CDF:

$$p_t(S_1) = \frac{\mathcal{B}(b;\, t+1,\, K-1)}{\mathcal{B}(t+1,\, K-1)} = I_b(t+1,\, K-1)$$

The probability of the region S_1 equals the incomplete beta function at b with parameters t + 1 and K – 1, divided by the complete beta function, which is the cumulative distribution function of a Beta distribution evaluated at b.

Under the Dirichlet path, x_i follows a Beta(t + 1, K – 1) distribution. We can therefore compute the required flux using this single coordinate.

Match the flux at the cut

Match the flux at the cut

$$C(x_i, t) = -\tilde{I}_{x_i}(t+1,\ K-1)\ \frac{\mathcal{B}(t+1,\ K-1)}{(1-x_i)^{K-1}\, x_i^t}, \qquad \tilde{I}_x(a,b) = \frac{\partial}{\partial a} I_x(a,b)$$

Match the flux at the cut

$$C(x_i, t) = -\tilde{I}_{x_i}(t+1,\ K-1)\ \frac{\mathcal{B}(t+1,\ K-1)}{(1-x_i)^{K-1}\, x_i^t}, \qquad \tilde{I}_x(a,b) = \frac{\partial}{\partial a} I_x(a,b)$$

The speed equals minus the derivative of the regularized incomplete beta function with respect to its first parameter, evaluated at x_i, times the complete beta function of t + 1 and K – 1, divided by one minus x_i raised to the K – 1, times x_i raised to the t.

Match the flux at the cut

$$C(x_i, t) = -\tilde{I}_{x_i}(t+1,\ K-1)\ \frac{\mathcal{B}(t+1,\ K-1)}{(1-x_i)^{K-1}\, x_i^t}, \qquad \tilde{I}_x(a,b) = \frac{\partial}{\partial a} I_x(a,b)$$

The speed equals minus the derivative of the regularized incomplete beta function with respect to its first parameter, evaluated at x_i, times the complete beta function of t + 1 and K – 1, divided by one minus x_i raised to the K – 1, times x_i raised to the t.

Increasing the first Beta parameter shifts probability toward larger coordinates. The derivative is therefore negative, giving a positive speed.

Match the flux at the cut

$$C(x_i, t) = -\tilde{I}_{x_i}(t+1,\ K-1)\ \frac{\mathcal{B}(t+1,\ K-1)}{(1-x_i)^{K-1}\, x_i^{t}}, \qquad \tilde{I}_x(a,b) = \frac{\partial}{\partial a} I_x(a,b)$$

The speed equals minus the derivative of the regularized incomplete beta function with respect to its first parameter, evaluated at x_i, times the complete beta function of t + 1 and K – 1, divided by one minus x_i raised to the K – 1, times x_i raised to the t.

Increasing the first Beta parameter shifts probability toward larger coordinates. The derivative is therefore negative, giving a positive speed.

There is no closed form for that derivative, so we compute it numerically at every inference step.

Match the flux at the cut

$$C(x_i, t) = -\tilde{I}_{x_i}(t+1,\ K-1)\ \frac{\mathcal{B}(t+1,\ K-1)}{(1-x_i)^{K-1}\, x_i^{t}}, \qquad \tilde{I}_x(a,b) = \frac{\partial}{\partial a} I_x(a,b)$$

The speed equals minus the derivative of the regularized incomplete beta function with respect to its first parameter, evaluated at x_i, times the complete beta function of t + 1 and K – 1, divided by one minus x_i raised to the K – 1, times x_i raised to the t.

Increasing the first Beta parameter shifts probability toward larger coordinates. The derivative is therefore negative, giving a positive speed.

There is no closed form for that derivative, so we compute it numerically at every inference step.

This velocity and this path satisfy the continuity equation, which is their Proposition 2.

Match the flux at the cut

$$C(x_i, t) = -\tilde{I}_{x_i}(t+1,\ K-1)\ \frac{\mathcal{B}(t+1,\ K-1)}{(1-x_i)^{K-1}\, x_i^t}, \qquad \tilde{I}_x(a,b) = \frac{\partial}{\partial a} I_x(a,b)$$

The speed equals minus the derivative of the regularized incomplete beta function with respect to its first parameter, evaluated at x_i, times the complete beta function of t + 1 and K – 1, divided by one minus x_i raised to the K – 1, times x_i raised to the t.

Increasing the first Beta parameter shifts probability toward larger coordinates. The derivative is therefore negative, giving a positive speed.

There is no closed form for that derivative, so we compute it numerically at every inference step.

The speed carries exactly the required amount of probability across each cut of the simplex.

Now take just two letters

Now take just two letters

With two letters, $I_x(a, 1) = x^a$, so we can evaluate the speed in closed form:

$$K = 2: \qquad C(x_1, t) = \frac{-x_1 \ln x_1}{(t+1)(1-x_1)}$$

Now take just two letters

With two letters, $I_x(a, 1) = x^a$, so we can evaluate the speed in closed form:

$$K = 2: \qquad C(x_1, t) = \frac{-x_1 \ln x_1}{(t+1)(1-x_1)}$$

With two letters, the speed equals minus x_1 times the natural logarithm of x_1, divided by t + 1 times one minus x_1.

Now take just two letters

With two letters, $I_x(a, 1) = x^a$, so we can evaluate the speed in closed form:

$$K = 2: \qquad C(x_1, t) = \frac{-\,x_1 \ln x_1}{(t+1)\,(1-x_1)}$$

With two letters, the speed equals minus x_1 times the natural logarithm of x_1, divided by t + 1 times one minus x_1.

The logarithm of a number below one is negative, so the speed is positive on the whole interval.

Now take just two letters

With two letters, $I_x(a, 1) = x^a$, so we can evaluate the speed in closed form:

$$K = 2 : \qquad C(x_1, t) = \frac{-\, x_1 \ln x_1}{(t+1)\,(1 - x_1)}$$

With two letters, the speed equals minus x_1 times the natural logarithm of x_1, divided by t + 1 times one minus x_1.

The logarithm of a number below one is negative, so the speed is positive on the whole interval.

At $x_1 = 0.3$ and t = 1 it gives 0.258, which matches the general formula to six decimals.

We can check the continuity equation by hand

We can check the continuity equation by hand

With $p_t(x) = (1 + t)x^t$ and $u = C(x, t)(1 - x)$:

$$\partial_t p + \partial_x (p\, u) = \left[x^t + (1+t)x^t \ln x\right] - \left[(t+1)x^t \ln x + x^t\right] = 0$$

We can check the continuity equation by hand

With $p_t(x) = (1 + t)x^t$ and $u = C(x, t)(1 - x)$:

$$\partial_t p + \partial_x (p\,u) = \left[x^t + (1+t)x^t \ln x\right] - \left[(t+1)x^t \ln x + x^t\right] = 0$$

The time derivative of the density is x to the power t plus one plus t times x to the power t times log x, the space derivative of the density times the velocity is t plus one times x to the power t times log x plus x to the power t, and the two cancel.

We can check the continuity equation by hand

With $p_t(x) = (1 + t)x^t$ and $u = C(x, t)(1 - x)$:

$$\partial_t p + \partial_x (p\,u) = \left[x^t + (1+t)x^t \ln x\right] - \left[(t+1)x^t \ln x + x^t\right] = 0$$

The time derivative of the density is x to the power t plus one plus t times x to the power t times log x, the space derivative of the density times the velocity is t plus one times x to the power t times log x plus x to the power t, and the two cancel.

The product p u is minus x to the power t plus one times log x, whose derivative gives the second bracket.

We can check the continuity equation by hand

With $p_t(x) = (1 + t)x^t$ and $u = C(x, t)(1 - x)$:

$$\partial_t p + \partial_x (p\,u) = \left[x^t + (1+t)x^t \ln x\right] - \left[(t+1)x^t \ln x + x^t\right] = 0$$

The time derivative of the density is x to the power t plus one plus t times x to the power t times log x, the space derivative of the density times the velocity is t plus one times x to the power t times log x plus x to the power t, and the two cancel.

The product p u is minus x to the power t plus one times log x, whose derivative gives the second bracket.

So this path and this velocity satisfy the continuity equation exactly, which is Proposition 2 for K = 2.

Let’s run the speed on a DNA base

Let’s run the speed on a DNA base

Take K = 4, t = 1, and the relaxed base x = (0.40, 0.30, 0.20, 0.10) for (A, C, G, T).

Let’s run the speed on a DNA base

Take K = 4, t = 1, and the relaxed base x = (0.40, 0.30, 0.20, 0.10) for (A, C, G, T).

The four speeds are

$$C(0.40, 1) = 0.232, \quad C(0.30, 1) = 0.199, \quad C(0.20, 1) = 0.159, \quad C(0.10, 1) = 0.104$$

Let’s run the speed on a DNA base

Take K = 4, t = 1, and the relaxed base x = (0.40, 0.30, 0.20, 0.10) for (A, C, G, T).

The four speeds are

$$C(0.40, 1) = 0.232, \quad C(0.30, 1) = 0.199, \quad C(0.20, 1) = 0.159, \quad C(0.10, 1) = 0.104$$

The speeds are 0.232 toward A, 0.199 toward C, 0.159 toward G, and 0.104 toward T.

Let’s run the speed on a DNA base

Take K = 4, t = 1, and the relaxed base x = (0.40, 0.30, 0.20, 0.10) for (A, C, G, T).

The four speeds are

$$C(0.40, 1) = 0.232, \quad C(0.30, 1) = 0.199, \quad C(0.20, 1) = 0.159, \quad C(0.10, 1) = 0.104$$

A coordinate that is already large has a larger speed, so the flow concentrates gradually.

Let’s run the speed on a DNA base

Take K = 4, t = 1, and the relaxed base x = (0.40, 0.30, 0.20, 0.10) for (A, C, G, T).

The four speeds are

$$C(0.40, 1) = 0.232, \quad C(0.30, 1) = 0.199, \quad C(0.20, 1) = 0.159, \quad C(0.10, 1) = 0.104$$

A coordinate that is already large has a larger speed, so the flow concentrates gradually.

$$u_t(x \mid e_A) = 0.232 \times (0.60,\ -0.30,\ -0.20,\ -0.10) = (0.139,\ -0.070,\ -0.046,\ -0.023)$$

Let’s run the speed on a DNA base

Take K = 4, t = 1, and the relaxed base x = (0.40, 0.30, 0.20, 0.10) for (A, C, G, T).

The four speeds are

$$C(0.40, 1) = 0.232, \quad C(0.30, 1) = 0.199, \quad C(0.20, 1) = 0.159, \quad C(0.10, 1) = 0.104$$

A coordinate that is already large has a larger speed, so the flow concentrates gradually.

$$u_t(x \mid e_A) = 0.232 \times (0.60,\ -0.30,\ -0.20,\ -0.10) = (0.139,\ -0.070,\ -0.046,\ -0.023)$$

The velocity toward A is 0.232 times the direction to the A corner, giving 0.139 for A, −0.070 for C, −0.046 for G, and −0.023 for T. These components sum to zero.

The posterior average is now a sum over corners

The posterior average is now a sum over corners

The marginal velocity averages the conditional velocities:

$$v_t(x) = \sum_{k=1}^{K} u_t(x \mid x_1 = e_k)\, p_{1|t}(e_k \mid x)$$

The posterior average is now a sum over corners

The marginal velocity averages the conditional velocities:

$$v_t(x) = \sum_{k=1}^{K} u_t(x \mid x_1 = e_k)\, p_{1|t}(e_k \mid x)$$

The marginal velocity at x equals the sum over corners e_k of the conditional velocity toward that corner, weighted by the posterior probability $p_{1|t}(e_k \mid x)$ that the data token is k.

The posterior average is now a sum over corners

The marginal velocity averages the conditional velocities:

$$v_t(x) = \sum_{k=1}^{K} u_t(x \mid x_1 = e_k)\, p_{1|t}(e_k \mid x)$$

The marginal velocity at x equals the sum over corners e_k of the conditional velocity toward that corner, weighted by the posterior probability $p_{1|t}(e_k \mid x)$ that the data token is k.

We average over the K possible data corners, so the posterior integral reduces to a sum.

The posterior average is now a sum over corners

The marginal velocity averages the conditional velocities:

$$v_t(x) = \sum_{k=1}^{K} u_t(x \mid x_1 = e_k)\, p_{1|t}(e_k \mid x)$$

The marginal velocity at x equals the sum over corners e_k of the conditional velocity toward that corner, weighted by the posterior probability $p_{1|t}(e_k \mid x)$ that the data token is k.

We average over the K possible data corners, so the posterior integral reduces to a sum.

A weighted average of tangent vectors remains tangent. We therefore need no additional tangency constraint on the learned field.

The posterior average is now a sum over corners

The marginal velocity averages the conditional velocities:

$$v_t(x) = \sum_{k=1}^{K} u_t(x \mid x_1 = e_k)\, p_{1|t}(e_k \mid x)$$

The marginal velocity at x equals the sum over corners e_k of the conditional velocity toward that corner, weighted by the posterior probability $p_{1|t}(e_k \mid x)$ that the data token is k.

We average over the K possible data corners, so the posterior integral reduces to a sum.

We weight the velocity toward each corner by the denoiser probability of that token.

We train the same denoiser with the same loss

We train the same denoiser with the same loss

Cross-entropy on the data token:

$$\boxed{\mathcal{L}(\theta) = -\,\mathbb{E}_{t,\,x_1,\,x}\big[\,\log p^{\theta}_{1|t}(x_1 \mid x)\big]}$$

We train the same denoiser with the same loss

Cross-entropy on the data token:

$$\mathcal{L}(\theta) = -\,\mathbb{E}_{t,\,x_1,\,x}\big[\,\log p^{\theta}_{1|t}(x_1 \mid x)\big]$$

The loss is minus the expectation, over the time t, the data token x_1, and the noisy point x, of the log probability the denoiser assigns to the data token.

We train the same denoiser with the same loss

Cross-entropy on the data token:

$$\mathcal{L}(\theta) = -\,\mathbb{E}_{t,\,x_1,\,x}\big[\,\log p^{\theta}_{1|t}(x_1 \mid x)\big]$$

The loss is minus the expectation, over the time t, the data token x_1, and the noisy point x, of the log probability the denoiser assigns to the data token.

We recover the true posterior by minimizing cross-entropy. Averaging with that posterior gives the true marginal velocity.

We train the same denoiser with the same loss

Cross-entropy on the data token:

$$\mathcal{L}(\theta) = -\,\mathbb{E}_{t,\,x_1,\,x}\big[\,\log p^{\theta}_{1|t}(x_1 \mid x)\big]$$

The loss is minus the expectation, over the time t, the data token x_1, and the noisy point x, of the log probability the denoiser assigns to the data token.

We recover the true posterior by minimizing cross-entropy. Averaging with that posterior gives the true marginal velocity.

We evaluate the conditional velocity only at inference.

We train the same denoiser with the same loss

Cross-entropy on the data token: $$\mathcal{L}(\theta) = -\,\mathbb{E}_{t,\,x_1,\,x}\big[\,\log p^{\theta}_{1|t}(x_1 \mid x)\big]$$

The loss is minus the expectation, over the time t, the data token x_1, and the noisy point x, of the log probability the denoiser assigns to the data token.

We recover the true posterior by minimizing cross-entropy. Averaging with that posterior gives the true marginal velocity.

We evaluate the conditional velocity only at inference.

We pass all N positions to the network and predict logits at each position. The denoiser can therefore represent dependencies between positions.

We can also compute the score of the path

We can also compute the score of the path

Differentiate the log density Γ-terms plus t log x_i:

$$s_t(x \mid x_1 = e_i) = \nabla_x \, t \log x_i = \frac{t}{x_i} \, e_i$$

We can also compute the score of the path

Differentiate the log density Γ-terms plus t log x_i: $$s_t(x \mid x_1 = e_i) = \nabla_x \, t \log x_i = \frac{t}{x_i} e_i$$

The conditional score is the gradient of t times the log of x_i, which is t divided by x_i in the i-th coordinate and zero elsewhere.

We can also compute the score of the path

Differentiate the log density Γ-terms plus t log x_i:

$$s_t(x \mid x_1 = e_i) = \nabla_x \, t \log x_i = \frac{t}{x_i} \, e_i$$

The conditional score is the gradient of t times the log of x_i, which is t divided by x_i in the i-th coordinate and zero elsewhere.

Average over corners with the denoiser:

$$\boxed{\hat{s} = D \, \hat{p}, \qquad D_{jk} = \delta_{jk} \, \frac{t}{x_j}}$$

We can also compute the score of the path

Differentiate the log density Γ-terms plus t log x_i:

$$s_t(x \mid x_1 = e_i) = \nabla_x \, t \log x_i = \frac{t}{x_i} \, e_i$$

The conditional score is the gradient of t times the log of x_i, which is t divided by x_i in the i-th coordinate and zero elsewhere.

Average over corners with the denoiser:

$$\hat{s} = D \, \hat{p}, \qquad D_{jk} = \delta_{jk} \frac{t}{x_j}$$

The score of the marginal path equals a diagonal matrix D, whose j-th entry is t divided by x_j, times the denoiser vector p-hat.

We can also compute the score of the path

Differentiate the log density Γ-terms plus t log x_i: $$s_t(x \mid x_1 = e_i) = \nabla_x \, t \log x_i = \frac{t}{x_i} \, e_i$$

The conditional score is the gradient of t times the log of x_i, which is t divided by x_i in the i-th coordinate and zero elsewhere.

Average over corners with the denoiser: $$\boxed{\hat{s} = D \, \hat{p}, \qquad D_{jk} = \delta_{jk} \frac{t}{x_j}}$$

The score of the marginal path equals a diagonal matrix D, whose j-th entry is t divided by x_j, times the denoiser vector p-hat.

The velocity is also linear in the denoiser: v-hat = U p-hat, where U contains the conditional velocities. Substituting the score relation gives v-hat = U D^{-1} s-hat.

Next, we use that linear map for guidance

Next, we use that linear map for guidance

In classifier-free guidance, we combine a conditional and an unconditional field:

$$\boxed{\hat{v}_{\mathrm{CFG}}(x, t, y) = \gamma\, \hat{v}(x, t, y) + (1 - \gamma)\, \hat{v}(x, t, \varnothing)}$$

Next, we use that linear map for guidance

In classifier-free guidance, we combine a conditional and an unconditional field:

$$\boxed{\hat{v}_{\mathrm{CFG}}(x, t, y) = \gamma\, \hat{v}(x, t, y) + (1 - \gamma)\, \hat{v}(x, t, \varnothing)}$$

The guided velocity equals γ times the velocity from the label-conditioned denoiser, plus one minus γ times the velocity from the unconditional denoiser.

Next, we use that linear map for guidance

In classifier-free guidance, we combine a conditional and an unconditional field:

$$\boxed{\hat{v}_{\mathrm{CFG}}(x, t, y) = \gamma\, \hat{v}(x, t, y) + (1 - \gamma)\, \hat{v}(x, t, \varnothing)}$$

The guided velocity equals γ times the velocity from the label-conditioned denoiser, plus one minus γ times the velocity from the unconditional denoiser.

Because both are linear in the denoiser, in Dirichlet Flow Matching we combine the two probability vectors, then project back onto the simplex when γ > 1 makes an entry negative.

Next, we use that linear map for guidance

In classifier-free guidance, we combine a conditional and an unconditional field:

$$\boxed{\hat{v}_{\mathrm{CFG}}(x, t, y) = \gamma\, \hat{v}(x, t, y) + (1 - \gamma)\, \hat{v}(x, t, \varnothing)}$$

The guided velocity equals γ times the velocity from the label-conditioned denoiser, plus one minus γ times the velocity from the unconditional denoiser.

Because both are linear in the denoiser, in Dirichlet Flow Matching we combine the two probability vectors, then project back onto the simplex when γ > 1 makes an entry negative.

For classifier guidance, we add the noisy classifier gradient to the score. We then project onto the tangent plane before recovering the denoiser.

Next, we use that linear map for guidance

In classifier-free guidance, we combine a conditional and an unconditional field:

$$\boxed{\hat{v}_{\mathrm{CFG}}(x, t, y) = \gamma\, \hat{v}(x, t, y) + (1 - \gamma)\, \hat{v}(x, t, \varnothing)}$$

The guided velocity equals γ times the velocity from the label-conditioned denoiser, plus one minus γ times the velocity from the unconditional denoiser.

Because both are linear in the denoiser, in Dirichlet Flow Matching we combine the two probability vectors, then project back onto the simplex when γ > 1 makes an entry negative.

For classifier guidance, we add the noisy classifier gradient to the score. We then project onto the tangent plane before recovering the denoiser.

With γ near 2 or 3, guidance improves even unconditional samples.

Recall that sampling here integrates an ODE

Recall that sampling here integrates an ODE

Sampling here integrates an ODE, so the map from the source point to the generated sequence is deterministic.

Recall that sampling here integrates an ODE

Sampling here integrates an ODE, so the map from the source point to the generated sequence is deterministic.

We run the teacher for 100 steps to pair each source point with its output. We then train a student to predict that output directly.

Recall that sampling here integrates an ODE

Sampling here integrates an ODE, so the map from the source point to the generated sequence is deterministic.

We run the teacher for 100 steps to pair each source point with its output. We then train a student to predict that output directly.

The student needs one function evaluation. For discrete-noise diffusion, the output remains random given the source, so we do not have the same deterministic training pairs.

With the relaxation, distillation is possible

Sampling here integrates an ODE, so the map from the source point to the generated sequence is deterministic.

We run the teacher for 100 steps to pair each source point with its output. We then train a student to predict that output directly.

The student needs one function evaluation. For discrete-noise diffusion, the output remains random given the source, so we do not have the same deterministic training pairs.

Dirichlet Flow Matching: one simplex step

Consider the two-letter alphabet A and G. The current point is (0.3, 0.7), with clean endpoint A at time 1. Use the two-letter closed form for the speed.

Dirichlet Flow Matching: one simplex step

Consider the two-letter alphabet A and G. The current point is (0.3, 0.7), with clean endpoint A at time 1. Use the two-letter closed form for the speed.

$$C(0.3, 1) = \frac{-0.3 \log(0.3)}{(1+1)(1-0.3)} \approx 0.2580$$

Dirichlet Flow Matching: one simplex step

Consider the two-letter alphabet A and G. The current point is (0.3, 0.7), with clean endpoint A at time 1. Use the two-letter closed form for the speed.

$$C(0.3, 1) = \frac{-0.3 \log(0.3)}{(1+1)(1-0.3)} \approx 0.2580$$

Multiplying this speed by the direction (0.7, −0.7) gives velocity (0.1806, −0.1806).

Dirichlet Flow Matching: one simplex step

Consider the two-letter alphabet A and G. The current point is (0.3, 0.7), with clean endpoint A at time 1. Use the two-letter closed form for the speed.

$$C(0.3, 1) = \frac{-0.3 \log(0.3)}{(1+1)(1-0.3)} \approx 0.2580$$

Multiplying this speed by the direction (0.7, −0.7) gives velocity (0.1806, −0.1806).

$$x_{1.1} \approx (0.3, 0.7) + 0.1(0.1806, -0.1806) = (0.3181, 0.6819)$$

Dirichlet Flow Matching: one simplex step

Consider the two-letter alphabet A and G. The current point is (0.3, 0.7), with clean endpoint A at time 1. Use the two-letter closed form for the speed.

$$C(0.3, 1) = \frac{-0.3 \log(0.3)}{(1+1)(1-0.3)} \approx 0.2580$$

Multiplying this speed by the direction (0.7, −0.7) gives velocity (0.1806, −0.1806).

$$x_{1.1} \approx (0.3, 0.7) + 0.1(0.1806, -0.1806) = (0.3181, 0.6819)$$

One Euler step transfers about 1.81 percentage points from G to A. The coordinates still sum to one.

Dirichlet Flow Matching: one simplex step

Consider the two-letter alphabet A and G. The current point is (0.3, 0.7), with clean endpoint A at time 1. Use the two-letter closed form for the speed.

$$C(0.3, 1) = \frac{-0.3 \log(0.3)}{(1+1)(1-0.3)} \approx 0.2580$$

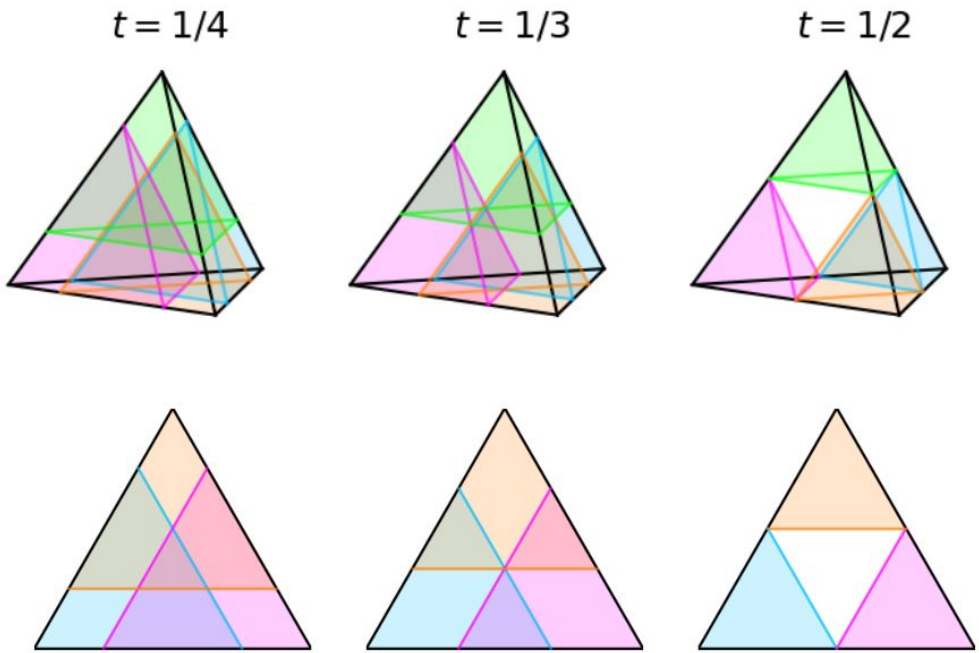
Multiplying this speed by the direction (0.7, −0.7) gives velocity (0.1806, −0.1806).

$$x_{1.1} \approx (0.3, 0.7) + 0.1(0.1806, -0.1806) = (0.3181, 0.6819)$$

One Euler step transfers about 1.81 percentage points from G to A. The coordinates still sum to one.

The velocity moves toward the clean corner at a speed that depends on the current belief.

Under a Dirichlet path, every corner still has support



Conditional paths for K = 4 and K = 3 at t = 1/4, 1/3, and 1/2. The overlap vanishes.

Stark,..,Jaakkola. *ICML* (2024)

Here is the promoter design result in Table 1 of the paper

METHOD	MSE	NFE
BIT DIFFUSION (BIT-ENCODING)*	.0414	100
BIT DIFFUSION (ONE-HOT ENCODING)*	.0395	100
D3PM-UNIFORM*	.0375	100
DDSM*	.0334	100
LANGUAGE MODEL	.0333	1024
LINEAR FM	.0281	100
DIRICHLET FM	**.0269**	100
DIRICHLET FM DISTILLED	.0278	**1**

Stark,..,Jaakkola. *ICML* (2024)

The straight-line path on the simplex reaches an error of 0.0281

METHOD	MSE	NFE
BIT DIFFUSION (BIT-ENCODING)*	.0414	100
BIT DIFFUSION (ONE-HOT ENCODING)*	.0395	100
D3PM-UNIFORM*	.0375	100
DDSM*	.0334	100
LANGUAGE MODEL	.0333	1024
LINEAR FM	.0281	100
DIRICHLET FM	**.0269**	100
DIRICHLET FM DISTILLED	.0278	**1**

Stark,..,Jaakkola. *ICML* (2024)

With the same 100 steps, the Dirichlet path improves the error to 0.0269

METHOD	MSE	NFE
BIT DIFFUSION (BIT-ENCODING)*	.0414	100
BIT DIFFUSION (ONE-HOT ENCODING)*	.0395	100
D3PM-UNIFORM*	.0375	100
DDSM*	.0334	100
LANGUAGE MODEL	.0333	1024
LINEAR FM	.0281	100
DIRICHLET FM	**.0269**	100
DIRICHLET FM DISTILLED	.0278	**1**

Stark,..,Jaakkola. *ICML* (2024)

Distilled into a single step, it keeps 0.0278, ahead of every diffusion baseline

METHOD	MSE	NFE
BIT DIFFUSION (BIT-ENCODING)*	.0414	100
BIT DIFFUSION (ONE-HOT ENCODING)*	.0395	100
D3PM-UNIFORM*	.0375	100
DDSM*	.0334	100
LANGUAGE MODEL	.0333	1024
LINEAR FM	.0281	100
DIRICHLET FM	**.0269**	100
DIRICHLET FM DISTILLED	.0278	**1**

Stark,..,Jaakkola. *ICML* (2024)

A language model needs 1024 steps for 0.0333

METHOD	MSE	NFE
BIT DIFFUSION (BIT-ENCODING)*	.0414	100
BIT DIFFUSION (ONE-HOT ENCODING)*	.0395	100
D3PM-UNIFORM*	.0375	100
DDSM*	.0334	100
LANGUAGE MODEL	.0333	1024
LINEAR FM	.0281	100
DIRICHLET FM	**.0269**	100
DIRICHLET FM DISTILLED	.0278	**1**

Stark,..,Jaakkola. *ICML* (2024)

Now look at the enhancer results in Table 2 of the paper

METHOD	MELANOMA FBD	FLY BRAIN FBD	NFE
RANDOM SEQUENCE	622.8	876	–
LANGUAGE MODEL	36.0	25.2	500
LINEAR FM	19.6	15.0	100
DIRICHLET FM	5.3	15.2	100
DIRICHLET FM DIST.	6.1	15.8	**1**
DIRICHLET FM CFG	**1.9**	**1.0**	200

Stark,..,Jaakkola. *ICML* (2024)

Without guidance, the Dirichlet path reaches a biological distance of 5.3 on melanoma enhancers

Method	Melanoma FBD	Fly Brain FBD	NFE
Random Sequence	622.8	876	–
Language Model	36.0	25.2	500
Linear FM	19.6	15.0	100
Dirichlet FM	5.3	15.2	100
Dirichlet FM dist.	6.1	15.8	**1**
Dirichlet FM CFG	**1.9**	**1.0**	200

Stark,..,Jaakkola. *ICML* (2024)

Classifier-free guidance reaches 1.9 on melanoma and 1.0 on fly brain

Method	Melanoma FBD	Fly Brain FBD	NFE
Random Sequence	622.8	876	–
Language Model	36.0	25.2	500
Linear FM	19.6	15.0	100
Dirichlet FM	5.3	15.2	100
Dirichlet FM dist.	6.1	15.8	**1**
Dirichlet FM CFG	**1.9**	**1.0**	200

Stark,..,Jaakkola. *ICML* (2024)

A language model reaches 36.0 and 25.2 on the same task

Method	Melanoma FBD	Fly Brain FBD	NFE
Random Sequence	622.8	876	–
Language Model	36.0	25.2	500
Linear FM	19.6	15.0	100
Dirichlet FM	5.3	15.2	100
Dirichlet FM dist.	6.1	15.8	**1**
Dirichlet FM CFG	**1.9**	**1.0**	200

Stark,..,Jaakkola. *ICML* (2024)

On fly brain without guidance, the Dirichlet path is level with the straight line, at 15.2 against 15.0

Method	Melanoma FBD	Fly Brain FBD	NFE
Random Sequence	622.8	876	–
Language Model	36.0	25.2	500
Linear FM	19.6	15.0	100
Dirichlet FM	5.3	15.2	100
Dirichlet FM dist.	6.1	15.8	**1**
Dirichlet FM CFG	**1.9**	**1.0**	200

Stark,..,Jaakkola. *ICML* (2024)

Should a small change in a rare token's probability count the same as one in a common token's?

Fisher-Flow

Fisher Flow Matching for Generative Modeling over Discrete Data

Oscar Davis[1*] **Samuel Kessler**[1] **Mircea Petrache**[2] **İsmail İlkan Ceylan**[1]

Michael Bronstein[1,3] **Avishek Joey Bose**[1]

[1]University of Oxford, [2]Pontificia Universidad Católica de Chile, [3]Aithyra

Davis,..,Bose. *NeurIPS* (2024)

Recall the ordinary straight-line distance on the simplex

Recall the ordinary straight-line distance on the simplex

In Dirichlet flow matching, we measured distance on the simplex with the straight-line distance.

Recall the ordinary straight-line distance on the simplex

In Dirichlet flow matching, we measured distance on the simplex with the straight-line distance.

Moving a base from (0.5, 0.5, 0, 0) to (0.4, 0.6, 0, 0) changes it a little.

Recall the ordinary straight-line distance on the simplex

In Dirichlet flow matching, we measured distance on the simplex with the straight-line distance.

Moving a base from (0.5, 0.5, 0, 0) to (0.4, 0.6, 0, 0) changes it a little.

Changing (0.01, 0.99, 0, 0) to (0.001, 0.999, 0, 0) reduces the first probability tenfold. Its straight-line distance is ten times smaller than in the preceding example.

In Fisher-Flow, we start from the KL divergence

In Fisher-Flow, we start from the KL divergence

Expand the KL divergence between two nearby distributions:

$$D_{\mathrm{KL}}\left(p_\theta \,\|\, p_{\theta+d\theta}\right) = \frac{1}{2}\sum_{j,k} g_{jk}(\theta)\, d\theta^j\, d\theta^k + o(|d\theta|^2)$$

In Fisher-Flow, we start from the KL divergence

Expand the KL divergence between two nearby distributions:

$$D_{\mathrm{KL}}\left(p_\theta \,\|\, p_{\theta+d\theta}\right) = \frac{1}{2}\sum_{j,k} g_{jk}(\theta)\, d\theta^j\, d\theta^k + o(|d\theta|^2)$$

The KL divergence between a distribution and a nearby one is one half the quadratic form of the matrix g evaluated at θ, applied to the parameter change, plus higher-order terms.

In Fisher-Flow, we start from the KL divergence

Expand the KL divergence between two nearby distributions:

$$D_{\mathrm{KL}}\left(p_\theta \,\|\, p_{\theta+d\theta}\right) = \frac{1}{2}\sum_{j,k} g_{jk}(\theta)\, d\theta^j\, d\theta^k + o(|d\theta|^2)$$

The KL divergence between a distribution and a nearby one is one half the quadratic form of the matrix g evaluated at θ, applied to the parameter change, plus higher-order terms.

The Fisher information matrix g determines the second-order approximation of KL.

In Fisher-Flow, we start from the KL divergence

Expand the KL divergence between two nearby distributions:

$$D_{\mathrm{KL}}\left(p_\theta \,\|\, p_{\theta+d\theta}\right) = \frac{1}{2}\sum_{j,k} g_{jk}(\theta)\, d\theta^j\, d\theta^k + o(|d\theta|^2)$$

The KL divergence between a distribution and a nearby one is one half the quadratic form of the matrix g evaluated at θ, applied to the parameter change, plus higher-order terms.

Changing a rare outcome's probability by a large fraction can matter even when the absolute change is small.

Compute that matrix for a categorical distribution

Compute that matrix for a categorical distribution

$$g_{jk}(p) = \mathbb{E}_p\left[\frac{\partial \log p}{\partial p^j}\,\frac{\partial \log p}{\partial p^k}\right] = \sum_i p^i\,\frac{\delta_{ij}}{p^j}\,\frac{\delta_{ik}}{p^k} = \frac{\delta_{jk}}{p^j}$$

Compute that matrix for a categorical distribution

$$g_{jk}(p) = \mathbb{E}_p\left[\frac{\partial \log p}{\partial p^j}\,\frac{\partial \log p}{\partial p^k}\right] = \sum_i p^i\,\frac{\delta_{ij}}{p^j}\,\frac{\delta_{ik}}{p^k} = \frac{\delta_{jk}}{p^j}$$

The Fisher matrix at p is the expectation of the product of two derivatives of the log probability, which collapses to one over p^j on the diagonal and zero off it.

Compute that matrix for a categorical distribution

$$g_{jk}(p) = \mathbb{E}_p\left[\frac{\partial \log p}{\partial p^j}\,\frac{\partial \log p}{\partial p^k}\right] = \sum_i p^i\,\frac{\delta_{ij}}{p^j}\,\frac{\delta_{ik}}{p^k} = \frac{\delta_{jk}}{p^j}$$

The Fisher matrix at p is the expectation of the product of two derivatives of the log probability, which collapses to one over p^j on the diagonal and zero off it.

The derivative of the log probability of outcome i with respect to p^j is one over p^j when i equals j, and zero otherwise.

Compute that matrix for a categorical distribution

$$g_{jk}(p) = \mathbb{E}_p\left[\frac{\partial \log p}{\partial p^j}\,\frac{\partial \log p}{\partial p^k}\right] = \sum_i p^i\,\frac{\delta_{ij}}{p^j}\,\frac{\delta_{ik}}{p^k} = \frac{\delta_{jk}}{p^j}$$

The Fisher matrix at p is the expectation of the product of two derivatives of the log probability, which collapses to one over p^j on the diagonal and zero off it.

The derivative of the log probability of outcome i with respect to p^j is one over p^j when i equals j, and zero otherwise.

After averaging over outcomes, only the diagonal terms remain.

Write the inner product of two tangent vectors at a point

Write the inner product of two tangent vectors at a point

The inner product of two tangent vectors at the point p:

$$\langle u, v \rangle_p = \sum_{i=1}^{K} \frac{u^i v^i}{p^i}$$

Write the inner product of two tangent vectors at a point

The inner product of two tangent vectors at the point p:

$$\langle u, v \rangle_p = \sum_{i=1}^{K} \frac{u^i v^i}{p^i}$$

The Fisher-Rao inner product of two tangent vectors u and v at the point p is the sum over letters of u^i times v^i divided by p^i.

Write the inner product of two tangent vectors at a point

The inner product of two tangent vectors at the point p:

$$\langle u, v \rangle_p = \sum_{i=1}^{K} \frac{u^i v^i}{p^i}$$

The Fisher-Rao inner product of two tangent vectors u and v at the point p is the sum over letters of u^i times v^i divided by p^i.

A change in a rare coordinate counts for more, because the division by a small p^i makes it large.

Suppose we relabel or merge the letters

Suppose we relabel or merge the letters

By Chentsov’s theorem, Fisher-Rao is the only metric, up to a constant, under which merging or relabeling outcomes is an isometry.

Suppose we relabel or merge the letters

By Chentsov’s theorem, Fisher-Rao is the only metric, up to a constant, under which merging or relabeling outcomes is an isometry.

Any other choice would make the geometry depend on how we happened to name the letters.

Suppose we relabel or merge the letters

By Chentsov’s theorem, Fisher-Rao is the only metric, up to a constant, under which merging or relabeling outcomes is an isometry.

Any other choice would make the geometry depend on how we happened to name the letters.

The metric diverges at the boundary, where some p^i is zero. We therefore apply a small amount of label smoothing at the data corners.

Take the square root of every coordinate

Take the square root of every coordinate

Take the square root of every coordinate:

$$\varphi(p) = \left(\sqrt{p^1},\ \ldots,\ \sqrt{p^K}\right), \qquad \varphi^{-1}(s) = \left(s_1^2,\ \ldots,\ s_K^2\right)$$

Take the square root of every coordinate

Take the square root of every coordinate:

$$\varphi(p) = \left(\sqrt{p^1},\ \ldots,\ \sqrt{p^K}\right), \qquad \varphi^{-1}(s) = \left(s_1^2,\ \ldots,\ s_K^2\right)$$

Under φ, we take the square root of each coordinate of p. Under the inverse map, we square each coordinate.

Take the square root of every coordinate

Take the square root of every coordinate:

$$\boxed{\varphi(p) = \left(\sqrt{p^1},\ \ldots,\ \sqrt{p^K}\right), \qquad \varphi^{-1}(s) = \left(s_1^2,\ \ldots,\ s_K^2\right)}$$

Under φ, we take the square root of each coordinate of p. Under the inverse map, we square each coordinate.

The simplex constraint becomes a norm constraint:

$$\sum_i p^i = 1 \quad \Longrightarrow \quad \left\|\varphi(p)\right\|_2^2 = \sum_i \left(\sqrt{p^i}\right)^2 = 1$$

Take the square root of every coordinate

Take the square root of every coordinate:

$$\boxed{\varphi(p) = \left(\sqrt{p^1},\ \ldots,\ \sqrt{p^K}\right), \qquad \varphi^{-1}(s) = \left(s_1^2,\ \ldots,\ s_K^2\right)}$$

Under φ, we take the square root of each coordinate of p. Under the inverse map, we square each coordinate.

The simplex constraint becomes a norm constraint:

$$\sum_i p^i = 1 \quad \Longrightarrow \quad \left\|\varphi(p)\right\|_2^2 = \sum_i \left(\sqrt{p^i}\right)^2 = 1$$

Because the coordinates of p sum to one, the squared length of the vector of square roots is also one, so φ(p) lies on the unit sphere.

Take the square root of every coordinate

Take the square root of every coordinate:

$$\varphi(p) = \left(\sqrt{p^1},\ \ldots,\ \sqrt{p^K}\right), \qquad \varphi^{-1}(s) = \left(s_1^2,\ \ldots,\ s_K^2\right)$$

Under φ, we take the square root of each coordinate of p. Under the inverse map, we square each coordinate.

The simplex constraint becomes a norm constraint:

$$\sum_i p^i = 1 \quad \Longrightarrow \quad \left\|\varphi(p)\right\|_2^2 = \sum_i \left(\sqrt{p^i}\right)^2 = 1$$

Because the coordinates of p sum to one, the squared length of the vector of square roots is also one, so φ(p) lies on the unit sphere.

Probabilities are nonnegative, so the image is the positive part of the sphere, one of its 2^K octants.

Differentiate the square root map

Differentiate the square root map

Differentiate the square root:

$$ds^i = \frac{dp^i}{2\sqrt{p^i}} \quad \Longrightarrow \quad \|ds\|_2^2 = \sum_i \frac{(dp^i)^2}{4\,p^i} = \tfrac{1}{4}\,\langle dp, dp\rangle_p$$

Differentiate the square root map

Differentiate the square root:

$$ds^i = \frac{dp^i}{2\sqrt{p^i}} \quad \Longrightarrow \quad \|ds\|_2^2 = \sum_i \frac{(dp^i)^2}{4\,p^i} = \tfrac{1}{4}\,\langle dp, dp\rangle_p$$

The change in a sphere coordinate is the change in the probability divided by twice its square root, so the squared Euclidean length on the sphere is one quarter of the Fisher-Rao length on the simplex.

Differentiate the square root map

Differentiate the square root:

$$ds^i = \frac{dp^i}{2\sqrt{p^i}} \quad \Longrightarrow \quad \|ds\|_2^2 = \sum_i \frac{(dp^i)^2}{4\,p^i} = \tfrac{1}{4}\,\langle dp, dp\rangle_p$$

The change in a sphere coordinate is the change in the probability divided by twice its square root, so the squared Euclidean length on the sphere is one quarter of the Fisher-Rao length on the simplex.

Scaled by two, the map is an isometry, so in Fisher-Flow the sphere has radius 2.

Differentiate the square root map

Differentiate the square root:

$$ds^i = \frac{dp^i}{2\sqrt{p^i}} \quad \Longrightarrow \quad \|ds\|_2^2 = \sum_i \frac{(dp^i)^2}{4\,p^i} = \tfrac{1}{4}\,\langle dp, dp\rangle_p$$

The change in a sphere coordinate is the change in the probability divided by twice its square root, so the squared Euclidean length on the sphere is one quarter of the Fisher-Rao length on the simplex.

Scaled by two, the map is an isometry, so in Fisher-Flow the sphere has radius 2.

The metric is now the plain Euclidean one restricted to a sphere, with no division by a probability.

Differentiate the square root map

Differentiate the square root:

$$ds^i = \frac{dp^i}{2\sqrt{p^i}} \quad \Longrightarrow \quad \|ds\|_2^2 = \sum_i \frac{(dp^i)^2}{4\,p^i} = \tfrac{1}{4}\,\langle dp, dp\rangle_p$$

The change in a sphere coordinate is the change in the probability divided by twice its square root, so the squared Euclidean length on the sphere is one quarter of the Fisher-Rao length on the simplex.

Scaled by two, the map is an isometry, so in Fisher-Flow the sphere has radius 2.

Taking square roots expands rare coordinates, making their changes more visible to ordinary geometric distance.

Start from two probability vectors p and q

Start from two probability vectors p and q

$$\boxed{d_{\mathrm{FR}}(p,q) = 2\arccos\left(\left\langle \sqrt{p},\ \sqrt{q} \right\rangle\right)}$$

Start from two probability vectors p and q

$$\boxed{d_{\mathrm{FR}}(p,q) = 2\arccos\left(\left\langle \sqrt{p},\ \sqrt{q} \right\rangle\right)}$$

The Fisher-Rao distance between p and q is twice the arccosine of the inner product of their square-root vectors.

Start from two probability vectors p and q

$$\boxed{d_{\mathrm{FR}}(p, q) = 2 \arccos\left(\left\langle \sqrt{p},\ \sqrt{q} \right\rangle\right)}$$

The Fisher-Rao distance between p and q is twice the arccosine of the inner product of their square-root vectors.

Square-root vectors of probability vectors have nonnegative entries, so the angle is at most a right angle and the distance is at most π.

Start from two probability vectors p and q

$$\boxed{d_{\mathrm{FR}}(p,q) = 2\arccos\left(\left\langle \sqrt{p},\ \sqrt{q} \right\rangle\right)}$$

The Fisher-Rao distance between p and q is twice the arccosine of the inner product of their square-root vectors.

Square-root vectors of probability vectors have nonnegative entries, so the angle is at most a right angle and the distance is at most π.

Two different letters are the farthest apart, at distance π, whichever letters they are.

Let’s put a DNA base on the sphere

Let’s put a DNA base on the sphere

Take the uniform base p_0 = (0.25, 0.25, 0.25, 0.25) and a smoothed A, p_1 = (0.91, 0.03, 0.03, 0.03).

Let’s put a DNA base on the sphere

Take the uniform base p_0 = (0.25, 0.25, 0.25, 0.25) and a smoothed A, p_1 = (0.91, 0.03, 0.03, 0.03).

Their square roots are

$$x_0 = (0.5,\ 0.5,\ 0.5,\ 0.5), \qquad x_1 = (0.954,\ 0.173,\ 0.173,\ 0.173)$$

Let’s put a DNA base on the sphere

Take the uniform base p_0 = (0.25, 0.25, 0.25, 0.25) and a smoothed A, p_1 = (0.91, 0.03, 0.03, 0.03).

Their square roots are

$$x_0 = (0.5,\ 0.5,\ 0.5,\ 0.5), \qquad x_1 = (0.954,\ 0.173,\ 0.173,\ 0.173)$$

The uniform base maps to coordinates (0.5, 0.5, 0.5, 0.5), while smoothed A maps to 0.954 for A and 0.173 for each other base.

Let’s put a DNA base on the sphere

Take the uniform base p_0 = (0.25, 0.25, 0.25, 0.25) and a smoothed A, p_1 = (0.91, 0.03, 0.03, 0.03).

Their square roots are

$$x_0 = (0.5,\ 0.5,\ 0.5,\ 0.5), \qquad x_1 = (0.954,\ 0.173,\ 0.173,\ 0.173)$$

Both have length one, as every square-root vector does.

Let’s put a DNA base on the sphere

Take the uniform base p_0 = (0.25, 0.25, 0.25, 0.25) and a smoothed A, p_1 = (0.91, 0.03, 0.03, 0.03).

Their square roots are

$$x_0 = (0.5,\ 0.5,\ 0.5,\ 0.5), \qquad x_1 = (0.954,\ 0.173,\ 0.173,\ 0.173)$$

Both have length one, as every square-root vector does.

The angle between them:

$$\langle x_0, x_1 \rangle = 0.737, \qquad \theta = \arccos(0.737) = 0.743, \qquad d_{\mathrm{FR}} = 1.485$$

Let’s put a DNA base on the sphere

Take the uniform base p_0 = (0.25, 0.25, 0.25, 0.25) and a smoothed A, p_1 = (0.91, 0.03, 0.03, 0.03).

Their square roots are

$$x_0 = (0.5,\ 0.5,\ 0.5,\ 0.5), \qquad x_1 = (0.954,\ 0.173,\ 0.173,\ 0.173)$$

Both have length one, as every square-root vector does.

The angle between them:

$$\langle x_0, x_1 \rangle = 0.737, \qquad \theta = \arccos(0.737) = 0.743, \qquad d_{\mathrm{FR}} = 1.485$$

The inner product is 0.737, giving an angle of 0.743 radians. The Fisher-Rao distance is twice that angle, or 1.485.

Geodesics on a sphere are great circles

Geodesics on a sphere are great circles

The conditional path is the constant-speed arc from x_0 to x_1:

$$\boxed{x_t = \exp_{x_0}\left(t \log_{x_0}(x_1)\right)}$$

Geodesics on a sphere are great circles

The conditional path is the constant-speed arc from x_0 to x_1:

$$x_t = \exp_{x_0}\left(t \log_{x_0}(x_1)\right)$$

The point at time t is the exponential map at x_0 applied to t times the logarithm map from x_0 to x_1, which is the point a fraction t of the way along the arc.

Geodesics on a sphere are great circles

The conditional path is the constant-speed arc from x_0 to x_1:

$$x_t = \exp_{x_0}\left(t \log_{x_0}(x_1)\right)$$

The point at time t is the exponential map at x_0 applied to t times the logarithm map from x_0 to x_1, which is the point a fraction t of the way along the arc.

The logarithm map gives the tangent vector at x_0 needed to reach x_1 in unit time. We use the exponential map to move along the sphere in that direction.

Geodesics on a sphere are great circles

The conditional path is the constant-speed arc from x_0 to x_1:

$$x_t = \exp_{x_0}\left(t \log_{x_0}(x_1)\right)$$

The point at time t is the exponential map at x_0 applied to t times the logarithm map from x_0 to x_1, which is the point a fraction t of the way along the arc.

The logarithm map gives the tangent vector at x_0 needed to reach x_1 in unit time. We use the exponential map to move along the sphere in that direction.

$$\exp_x(w) = \cos(\|w\|)\, x + \sin(\|w\|)\, \frac{w}{\|w\|}, \qquad \log_x(y) = \frac{\arccos\langle x, y\rangle}{\sqrt{1 - \langle x, y\rangle^2}} \left(y - \langle x, y\rangle\, x\right)$$

Geodesics on a sphere are great circles

The conditional path is the constant-speed arc from x_0 to x_1: $$\boxed{x_t = \exp_{x_0}\left(t \log_{x_0}(x_1)\right)}$$

The point at time t is the exponential map at x_0 applied to t times the logarithm map from x_0 to x_1, which is the point a fraction t of the way along the arc.

The logarithm map gives the tangent vector at x_0 needed to reach x_1 in unit time. We use the exponential map to move along the sphere in that direction.

$$\exp_x(w) = \cos(\|w\|)\, x + \sin(\|w\|)\, \frac{w}{\|w\|}, \qquad \log_x(y) = \frac{\arccos\langle x, y\rangle}{\sqrt{1 - \langle x, y\rangle^2}} \left(y - \langle x, y\rangle\, x\right)$$

The exponential map at x in the direction w is the cosine of the length of w times x, plus the sine of that length times the unit vector along w; the logarithm map from x to y is the angle between them, divided by the sine of that angle, times the component of y orthogonal to x.

Geodesics on a sphere are great circles

The conditional path is the constant-speed arc from x_0 to x_1: $$\boxed{x_t = \exp_{x_0}\left(t \log_{x_0}(x_1)\right)}$$

The point at time t is the exponential map at x_0 applied to t times the logarithm map from x_0 to x_1, which is the point a fraction t of the way along the arc.

The logarithm map gives the tangent vector at x_0 needed to reach x_1 in unit time. We use the exponential map to move along the sphere in that direction.

$$\exp_x(w) = \cos(\|w\|)\, x + \sin(\|w\|)\, \frac{w}{\|w\|}, \qquad \log_x(y) = \frac{\arccos\langle x, y\rangle}{\sqrt{1 - \langle x, y\rangle^2}} \left(y - \langle x, y\rangle\, x\right)$$

We use the logarithm map to compute a tangent direction, then the exponential map to move along the sphere.

Aim at the data and divide by the time left

Aim at the data and divide by the time left

At the current point, aim at the data and divide by the time left:

$$\boxed{u_t(x_t \mid x_1) = \frac{\log_{x_t}(x_1)}{1-t}}$$

Aim at the data and divide by the time left

At the current point, aim at the data and divide by the time left:

$$\boxed{u_t(x_t \mid x_1) = \frac{\log_{x_t}(x_1)}{1-t}}$$

The conditional velocity at the point x_t equals the logarithm map from x_t to the data point x_1, divided by one minus t.

Aim at the data and divide by the time left

At the current point, aim at the data and divide by the time left:

$$\boxed{u_t(x_t \mid x_1) = \frac{\log_{x_t}(x_1)}{1-t}}$$

The conditional velocity at the point x_t equals the logarithm map from x_t to the data point x_1, divided by one minus t.

The logarithm has length equal to the remaining arc, (1 – t) times the total, so the division leaves a constant speed.

Aim at the data and divide by the time left

At the current point, aim at the data and divide by the time left:

$$\boxed{u_t(x_t \mid x_1) = \frac{\log_{x_t}(x_1)}{1-t}}$$

The conditional velocity at the point x_t equals the logarithm map from x_t to the data point x_1, divided by one minus t.

The logarithm has length equal to the remaining arc, (1 – t) times the total, so the division leaves a constant speed.

For our base, that speed is 0.743 at every time, the same number as the half distance.

Aim at the data and divide by the time left

At the current point, aim at the data and divide by the time left:

$$\boxed{u_t(x_t \mid x_1) = \frac{\log_{x_t}(x_1)}{1-t}}$$

The conditional velocity at the point x_t equals the logarithm map from x_t to the data point x_1, divided by one minus t.

The logarithm has length equal to the remaining arc, (1 – t) times the total, so the division leaves a constant speed.

For our base, that speed is 0.743 at every time, the same number as the half distance.

For straight-line flow matching, we divide the remaining displacement by the remaining time. Here we use the remaining arc instead.

Let’s take one step along the arc

Let’s take one step along the arc

At t = 0.5 our base sits at x_t = (0.780, 0.361, 0.361, 0.361), which reads as the distribution (0.609, 0.130, 0.130, 0.130).

Let’s take one step along the arc

At t = 0.5 our base sits at x_t = (0.780, 0.361, 0.361, 0.361), which reads as the distribution (0.609, 0.130, 0.130, 0.130).

One exponential-map step: $$x_{t+\Delta} = \exp_{x_t}\left(\frac{\Delta}{1-t}\,\log_{x_t}(x_1)\right), \qquad \Delta = 0.1$$

Let’s take one step along the arc

At t = 0.5 our base sits at x_t = (0.780, 0.361, 0.361, 0.361), which reads as the distribution (0.609, 0.130, 0.130, 0.130).

One exponential-map step: $$x_{t+\Delta} = \exp_{x_t}\left(\frac{\Delta}{1-t}\,\log_{x_t}(x_1)\right), \qquad \Delta = 0.1$$

With a step Δ of 0.1, the next point is the exponential map at x_t applied to Δ divided by one minus t, times the logarithm map from x_t toward x_1.

Let’s take one step along the arc

At t = 0.5 our base sits at x_t = (0.780, 0.361, 0.361, 0.361), which reads as the distribution (0.609, 0.130, 0.130, 0.130).

One exponential-map step: $$x_{t+\Delta} = \exp_{x_t}\left(\frac{\Delta}{1-t}\,\log_{x_t}(x_1)\right), \qquad \Delta = 0.1$$

$$x_{0.6} = (0.824,\ 0.327,\ 0.327,\ 0.327) \implies p_{0.6} = (0.680,\ 0.107,\ 0.107,\ 0.107)$$

Let’s take one step along the arc

At t = 0.5 our base sits at x_t = (0.780, 0.361, 0.361, 0.361), which reads as the distribution (0.609, 0.130, 0.130, 0.130).

One exponential-map step: $$x_{t+\Delta} = \exp_{x_t}\left(\frac{\Delta}{1-t}\,\log_{x_t}(x_1)\right), \qquad \Delta = 0.1$$

$$x_{0.6} = (0.824,\ 0.327,\ 0.327,\ 0.327) \implies p_{0.6} = (0.680,\ 0.107,\ 0.107,\ 0.107)$$

The step lands at 0.824 in the A coordinate and 0.327 in the others, which squares to the distribution 0.680 for A and 0.107 for each of the rest.

Let’s take one step along the arc

At t = 0.5 our base sits at x_t = (0.780, 0.361, 0.361, 0.361), which reads as the distribution (0.609, 0.130, 0.130, 0.130).

One exponential-map step: $$x_{t+\Delta} = \exp_{x_t}\left(\frac{\Delta}{1-t}\,\log_{x_t}(x_1)\right), \qquad \Delta = 0.1$$

$$x_{0.6} = (0.824,\ 0.327,\ 0.327,\ 0.327) \implies p_{0.6} = (0.680,\ 0.107,\ 0.107,\ 0.107)$$

The point has length one exactly, because the step followed the sphere. A straight Euclidean step would leave it.

Subtract the part that points away from the sphere

Subtract the part that points away from the sphere

Subtract the part that points away from the sphere:

$$\boxed{v_\theta(t,x) = \tilde{v}_\theta(t,x) - \left\langle x,\ \tilde{v}_\theta(t,x) \right\rangle x}$$

Subtract the part that points away from the sphere

Subtract the part that points away from the sphere:

$$\boxed{v_\theta(t,x) = \tilde{v}_\theta(t,x) - \left\langle x,\ \tilde{v}_\theta(t,x) \right\rangle x}$$

The velocity we use equals the raw output minus the inner product of the current point with that output, times the current point.

Subtract the part that points away from the sphere

Subtract the part that points away from the sphere:

$$\boxed{v_\theta(t, x) = \tilde{v}_\theta(t, x) - \left\langle x,\ \tilde{v}_\theta(t, x) \right\rangle x}$$

The velocity we use equals the raw output minus the inner product of the current point with that output, times the current point.

At x = (0.780, 0.361, 0.361, 0.361), the raw output (0.3, −0.2, 0.5, 0.1) has inner product 0.379 with x. Subtracting the radial component gives (0.005, −0.337, 0.363, −0.037).

Subtract the part that points away from the sphere

Subtract the part that points away from the sphere:

$$\boxed{v_\theta(t, x) = \tilde{v}_\theta(t, x) - \left\langle x,\ \tilde{v}_\theta(t, x) \right\rangle x}$$

The velocity we use equals the raw output minus the inner product of the current point with that output, times the current point.

At x = (0.780, 0.361, 0.361, 0.361), the raw output (0.3, −0.2, 0.5, 0.1) has inner product 0.379 with x. Subtracting the radial component gives (0.005, −0.337, 0.363, −0.037).

We can use any architecture because we project its output onto the tangent space afterward.

Subtract the part that points away from the sphere

Subtract the part that points away from the sphere:

$$\boxed{v_\theta(t, x) = \tilde{v}_\theta(t, x) - \left\langle x,\ \tilde{v}_\theta(t, x) \right\rangle x}$$

The velocity we use equals the raw output minus the inner product of the current point with that output, times the current point.

At x = (0.780, 0.361, 0.361, 0.361), the raw output (0.3, −0.2, 0.5, 0.1) has inner product 0.379 with x. Subtracting the radial component gives (0.005, −0.337, 0.363, −0.037).

Removing the outward component keeps the motion on the sphere, so the squared coordinates still sum to one.

We regress the network onto that velocity

We regress the network onto that velocity

$$\mathcal{L}(\theta) = \mathbb{E}_{t,\,(x_0,x_1),\,x_t} \left\| v_\theta(t, x_t) - \frac{\log_{x_t}(x_1)}{1-t} \right\|^2$$

We regress the network onto that velocity

$$\mathcal{L}(\theta) = \mathbb{E}_{t,\,(x_0,x_1),\,x_t}\left\| v_\theta(t, x_t) - \frac{\log_{x_t}(x_1)}{1-t} \right\|^2$$

The loss is the expected squared distance between the model velocity at the point x_t and the logarithm map toward the data point divided by one minus t.

We regress the network onto that velocity

$$\mathcal{L}(\theta) = \mathbb{E}_{t,\,(x_0,x_1),\,x_t} \left\| v_\theta(t, x_t) - \frac{\log_{x_t}(x_1)}{1-t} \right\|^2$$

The loss is the expected squared distance between the model velocity at the point x_t and the logarithm map toward the data point divided by one minus t.

The expectation runs over the time t, the pair of source and data points, and the point x_t on the arc between them.

We regress the network onto that velocity

$$\mathcal{L}(\theta) = \mathbb{E}_{t,\,(x_0,x_1),\,x_t}\left\| v_\theta(t, x_t) - \frac{\log_{x_t}(x_1)}{1-t} \right\|^2$$

The loss is the expected squared distance between the model velocity at the point x_t and the logarithm map toward the data point divided by one minus t.

The expectation runs over the time t, the pair of source and data points, and the point x_t on the arc between them.

Regression replaces cross-entropy here, because the target is a vector.

Choosing the source distribution

Choosing the source distribution

The path is built from a pair (x_0, x_1), so the source distribution is free.

Choosing the source distribution

The path is built from a pair (x_0, x_1), so the source distribution is free.

For Dirichlet paths, we used the uniform Dirichlet source. For masked paths, we started from the mask token.

Choosing the source distribution

The path is built from a pair (x_0, x_1), so the source distribution is free.

For Dirichlet paths, we used the uniform Dirichlet source. For masked paths, we started from the mask token.

With a source trained to look like the data, the arcs start closer to where they end.

With independent pairing, two arcs can cross

With independent pairing, two arcs can cross

Two independently paired arcs can cross. At their intersection, we have two possible regression targets.

With independent pairing, two arcs can cross

Two independently paired arcs can cross. At their intersection, we have two possible regression targets.

Pair the samples by the optimal transport plan:

$$\boxed{\pi \;=\; \arg\min_{\pi'} \; \mathbb{E}_{(x_0,x_1)\sim\pi'}\big[\, d^2(x_0,x_1)\,\big]}$$

With independent pairing, two arcs can cross

Two independently paired arcs can cross. At their intersection, we have two possible regression targets.

Pair the samples by the optimal transport plan:

$$\pi \;=\; \arg\min_{\pi'} \; \mathbb{E}_{(x_0,x_1)\sim\pi'}\left[\, d^2(x_0, x_1) \,\right]$$

The coupling π is the joint distribution over source and data pairs that minimizes the expected squared geodesic distance, among all couplings with the given marginals.

With independent pairing, two arcs can cross

Two independently paired arcs can cross. At their intersection, we have two possible regression targets.

Pair the samples by the optimal transport plan:

$$\boxed{\pi \;=\; \arg\min_{\pi'} \; \mathbb{E}_{(x_0,x_1)\sim\pi'}\big[\, d^2(x_0,x_1)\,\big]}$$

The coupling π is the joint distribution over source and data pairs that minimizes the expected squared geodesic distance, among all couplings with the given marginals.

Under this plan, each intermediate point corresponds to one pair. The arcs do not cross and have the least kinetic energy.

With independent pairing, two arcs can cross

Two independently paired arcs can cross. At their intersection, we have two possible regression targets.

Pair the samples by the optimal transport plan:

$$\pi \;=\; \arg\min_{\pi'} \; \mathbb{E}_{(x_0,x_1)\sim\pi'}\left[\, d^2(x_0, x_1) \,\right]$$

The coupling π is the joint distribution over source and data pairs that minimizes the expected squared geodesic distance, among all couplings with the given marginals.

Under this plan, each intermediate point corresponds to one pair. The arcs do not cross and have the least kinetic energy.

In practice we solve the plan on each minibatch with the Sinkhorn algorithm.

Let’s pair three bases two ways

Let’s pair three bases two ways

The three sources favor A, C, and G. The targets are smoothed A, G, and C.

Let’s pair three bases two ways

The three sources favor A, C, and G. The targets are smoothed A, G, and C.

The squared geodesic costs are $$C = \begin{pmatrix} 0.076 & 0.963 & 0.963 \\ 0.963 & 0.963 & 0.076 \\ 0.963 & 0.076 & 0.963 \end{pmatrix}$$

Let’s pair three bases two ways

The three sources favor A, C, and G. The targets are smoothed A, G, and C.

The squared geodesic costs are $$C = \begin{pmatrix} 0.076 & 0.963 & 0.963 \\ 0.963 & 0.963 & 0.076 \\ 0.963 & 0.076 & 0.963 \end{pmatrix}$$

The cost matrix has 0.076 where a source and a target favor the same base, and 0.963 where they do not.

Let’s pair three bases two ways

The three sources favor A, C, and G. The targets are smoothed A, G, and C.

The squared geodesic costs are $$C = \begin{pmatrix} 0.076 & 0.963 & 0.963 \\ 0.963 & 0.963 & 0.076 \\ 0.963 & 0.076 & 0.963 \end{pmatrix}$$

The total costs are $$\boxed{\text{pairing in order : } 2.001, \qquad \text{optimal pairing : } 0.227}$$

Let’s pair three bases two ways

The three sources favor A, C, and G. The targets are smoothed A, G, and C.

The squared geodesic costs are $$C = \begin{pmatrix} 0.076 & 0.963 & 0.963 \\ 0.963 & 0.963 & 0.076 \\ 0.963 & 0.076 & 0.963 \end{pmatrix}$$

The total costs are $$\boxed{\text{pairing in order : } 2.001, \qquad \text{optimal pairing : } 0.227}$$

Pairing in the original order costs 2.001, while the optimal pairing costs 0.227.

Let’s pair three bases two ways

The three sources favor A, C, and G. The targets are smoothed A, G, and C.

The squared geodesic costs are $$C = \begin{pmatrix} 0.076 & 0.963 & 0.963 \\ 0.963 & 0.963 & 0.076 \\ 0.963 & 0.076 & 0.963 \end{pmatrix}$$

The total costs are $$\boxed{\text{pairing in order : } \; 2.001, \qquad \text{optimal pairing : } \; 0.227}$$

Pairing in the original order costs 2.001, while the optimal pairing costs 0.227.

Mean arc length falls from 0.746 to 0.275, so the velocities the network must fit are smaller and more consistent.

Compare the three state spaces we have seen

Compare the three state spaces we have seen

We used tokens with jump rates, then the simplex with a Dirichlet path, and now the sphere with great circles.

Compare the three state spaces we have seen

We used tokens with jump rates, then the simplex with a Dirichlet path, and now the sphere with great circles.

In all three, we choose a conditional path and write down its velocity.

Compare the three state spaces we have seen

We used tokens with jump rates, then the simplex with a Dirichlet path, and now the sphere with great circles.

In all three, we choose a conditional path and write down its velocity.

With jump rates and Dirichlet paths, we train a denoiser and recover the velocity by a posterior average over the data token.

Compare the three state spaces we have seen

We used tokens with jump rates, then the simplex with a Dirichlet path, and now the sphere with great circles.

In all three, we choose a conditional path and write down its velocity.

With jump rates and Dirichlet paths, we train a denoiser and recover the velocity by a posterior average over the data token.

In Fisher-Flow, we regress the velocity itself against the geodesic target and project the output onto the tangent space, so no denoiser appears.

The conditional paths differ only in their geometry

We used tokens with jump rates, then the simplex with a Dirichlet path, and now the sphere with great circles.

In all three, we choose a conditional path and write down its velocity.

With jump rates and Dirichlet paths, we train a denoiser and recover the velocity by a posterior average over the data token.

In Fisher-Flow, we regress the velocity itself against the geodesic target and project the output onto the tangent space, so no denoiser appears.

Let’s write Fisher-Flow for DNA

```python
def to_sphere(p):
    return p.clamp(min=1e-8).sqrt()            # simplex ->
sphere

def to_simplex(x):
    return x ** 2                               # sphere ->
simplex

def log_map(x, y):
    c = (x * y).sum(-1, keepdim=True).clamp(max=1 - 1e-6)
    v = y - c * x                              # orthogonal to
x
    return v * torch.arccos(c) / v.norm(dim=-1, keepdim=True)

def exp_map(x, w):
    n = w.norm(dim=-1, keepdim=True).clamp(min=1e-8)
    return torch.cos(n) * x + torch.sin(n) * (w / n)

def geodesic(x0, x1, t):
    return exp_map(x0, t * log_map(x0, x1))     # the
conditional path

def loss(model, x0, x1, t):
    xt = geodesic(x0, x1, t[:, None, None])
    u = log_map(xt, x1) / (1 - t[:, None, None])
    raw = model(xt, t)
    v = raw - (xt * raw).sum(-1, keepdim=True) * xt
```

We take square roots to map onto the sphere and square to return

- We map a probability vector over A, C, G, T to a unit vector and back.

```
def to_sphere(p):
    return p.clamp(min=1e-8).sqrt()           # simplex ->
sphere

def to_simplex(x):
    return x ** 2                             # sphere ->
simplex

def log_map(x, y):
    c = (x * y).sum(-1, keepdim=True).clamp(max=1 - 1e-6)
    v = y - c * x                             # orthogonal to
x
    return v * torch.arccos(c) / v.norm(dim=-1, keepdim=True)

def exp_map(x, w):
    n = w.norm(dim=-1, keepdim=True).clamp(min=1e-8)
    return torch.cos(n) * x + torch.sin(n) * (w / n)

def geodesic(x0, x1, t):
    return exp_map(x0, t * log_map(x0, x1))     # the
conditional path

def loss(model, x0, x1, t):
    xt = geodesic(x0, x1, t[:, None, None])
    u = log_map(xt, x1) / (1 - t[:, None, None])
    raw = model(xt, t)
    v = raw - (xt * raw).sum(-1, keepdim=True) * xt
```

The direction along the arc comes from the logarithm map

- We map a probability vector over A, C, G, T to a unit vector and back.
- The orthogonal component determines the direction, while the angle determines the length.

```
def to_sphere(p):
    return p.clamp(min=1e-8).sqrt()            # simplex ->
sphere

def to_simplex(x):
    return x ** 2                               # sphere ->
simplex

def log_map(x, y):
    c = (x * y).sum(-1, keepdim=True).clamp(max=1 - 1e-6)
    v = y - c * x                               # orthogonal to
x
    return v * torch.arccos(c) / v.norm(dim=-1, keepdim=True)

def exp_map(x, w):
    n = w.norm(dim=-1, keepdim=True).clamp(min=1e-8)
    return torch.cos(n) * x + torch.sin(n) * (w / n)

def geodesic(x0, x1, t):
    return exp_map(x0, t * log_map(x0, x1))     # the
conditional path

def loss(model, x0, x1, t):
    xt = geodesic(x0, x1, t[:, None, None])
    u = log_map(xt, x1) / (1 - t[:, None, None])
    raw = model(xt, t)
    v = raw - (xt * raw).sum(-1, keepdim=True) * xt
```

The exponential map is one step along the sphere

- We map a probability vector over A, C, G, T to a unit vector and back.
- The orthogonal component determines the direction, while the angle determines the length.
- We combine the current point weighted by cosine with the tangent direction weighted by sine. The result remains on the sphere.

```
def to_sphere(p):
    return p.clamp(min=1e-8).sqrt()             # simplex ->
sphere

def to_simplex(x):
    return x ** 2                                # sphere ->
simplex

def log_map(x, y):
    c = (x * y).sum(-1, keepdim=True).clamp(max=1 - 1e-6)
    v = y - c * x                                # orthogonal to
x
    return v * torch.arccos(c) / v.norm(dim=-1, keepdim=True)

def exp_map(x, w):
    n = w.norm(dim=-1, keepdim=True).clamp(min=1e-8)
    return torch.cos(n) * x + torch.sin(n) * (w / n)

def geodesic(x0, x1, t):
    return exp_map(x0, t * log_map(x0, x1))      # the
conditional path

def loss(model, x0, x1, t):
    xt = geodesic(x0, x1, t[:, None, None])
    u = log_map(xt, x1) / (1 - t[:, None, None])
    raw = model(xt, t)
    v = raw - (xt * raw).sum(-1, keepdim=True) * xt
```

The conditional path is the arc at fraction t

- We evaluate the exponential map at the fraction t of the full tangent vector.

```
def to_sphere(p):
    return p.clamp(min=1e-8).sqrt()            # simplex ->
sphere

def to_simplex(x):
    return x ** 2                               # sphere ->
simplex

def log_map(x, y):
    c = (x * y).sum(-1, keepdim=True).clamp(max=1 - 1e-6)
    v = y - c * x                              # orthogonal to
x
    return v * torch.arccos(c) / v.norm(dim=-1, keepdim=True)

def exp_map(x, w):
    n = w.norm(dim=-1, keepdim=True).clamp(min=1e-8)
    return torch.cos(n) * x + torch.sin(n) * (w / n)

def geodesic(x0, x1, t):
    return exp_map(x0, t * log_map(x0, x1))     # the
conditional path

def loss(model, x0, x1, t):
    xt = geodesic(x0, x1, t[:, None, None])
    u = log_map(xt, x1) / (1 - t[:, None, None])
    raw = model(xt, t)
    v = raw - (xt * raw).sum(-1, keepdim=True) * xt
```

The target is the remaining arc over the remaining time

- We evaluate the exponential map at the fraction t of the full tangent vector.

```
def to_sphere(p):
    return p.clamp(min=1e-8).sqrt()            # simplex ->
sphere

def to_simplex(x):
    return x ** 2                               # sphere ->
simplex

def log_map(x, y):
    c = (x * y).sum(-1, keepdim=True).clamp(max=1 - 1e-6)
    v = y - c * x                               # orthogonal to
x
    return v * torch.arccos(c) / v.norm(dim=-1, keepdim=True)

def exp_map(x, w):
    n = w.norm(dim=-1, keepdim=True).clamp(min=1e-8)
    return torch.cos(n) * x + torch.sin(n) * (w / n)

def geodesic(x0, x1, t):
    return exp_map(x0, t * log_map(x0, x1))     # the
conditional path

def loss(model, x0, x1, t):
    xt = geodesic(x0, x1, t[:, None, None])
    u = log_map(xt, x1) / (1 - t[:, None, None])
    raw = model(xt, t)
    v = raw - (xt * raw).sum(-1, keepdim=True) * xt
```

We project the raw network output onto the tangent space

- We evaluate the exponential map at the fraction t of the full tangent vector.
- We can use any architecture to predict the raw vector.

```
def to_sphere(p):
    return p.clamp(min=1e-8).sqrt()            # simplex ->
sphere

def to_simplex(x):
    return x ** 2                               # sphere ->
simplex

def log_map(x, y):
    c = (x * y).sum(-1, keepdim=True).clamp(max=1 - 1e-6)
    v = y - c * x                               # orthogonal to
x
    return v * torch.arccos(c) / v.norm(dim=-1, keepdim=True)

def exp_map(x, w):
    n = w.norm(dim=-1, keepdim=True).clamp(min=1e-8)
    return torch.cos(n) * x + torch.sin(n) * (w / n)

def geodesic(x0, x1, t):
    return exp_map(x0, t * log_map(x0, x1))     # the
conditional path

def loss(model, x0, x1, t):
    xt = geodesic(x0, x1, t[:, None, None])
    u = log_map(xt, x1) / (1 - t[:, None, None])
    raw = model(xt, t)
    v = raw - (xt * raw).sum(-1, keepdim=True) * xt
```

The loss is a plain squared error on the sphere

- We evaluate the exponential map at the fraction t of the full tangent vector.
- We can use any architecture to predict the raw vector.

```
def to_sphere(p):
    return p.clamp(min=1e-8).sqrt()            # simplex ->
sphere

def to_simplex(x):
    return x ** 2                               # sphere ->
simplex

def log_map(x, y):
    c = (x * y).sum(-1, keepdim=True).clamp(max=1 - 1e-6)
    v = y - c * x                               # orthogonal to
x
    return v * torch.arccos(c) / v.norm(dim=-1, keepdim=True)

def exp_map(x, w):
    n = w.norm(dim=-1, keepdim=True).clamp(min=1e-8)
    return torch.cos(n) * x + torch.sin(n) * (w / n)

def geodesic(x0, x1, t):
    return exp_map(x0, t * log_map(x0, x1))     # the
conditional path

def loss(model, x0, x1, t):
    xt = geodesic(x0, x1, t[:, None, None])
    u = log_map(xt, x1) / (1 - t[:, None, None])
    raw = model(xt, t)
    v = raw - (xt * raw).sum(-1, keepdim=True) * xt
```

Fisher-Flow: a quarter of a spherical path

Toy alphabet A and G. Follow the spherical path from the all-A corner to the all-G corner. Square roots of probabilities are the sphere coordinates, denoted by y.

Fisher-Flow: a quarter of a spherical path

Toy alphabet A and G. Follow the spherical path from the all-A corner to the all-G corner. Square roots of probabilities are the sphere coordinates, denoted by y.

$$y(t) = \Big(\cos(\pi t/2), \sin(\pi t/2)\Big), \qquad y(0.25) \approx (0.9239, 0.3827)$$

Fisher-Flow: a quarter of a spherical path

Toy alphabet A and G. Follow the spherical path from the all-A corner to the all-G corner. Square roots of probabilities are the sphere coordinates, denoted by y.

$$y(t) = \big(\cos(\pi t/2), \sin(\pi t/2)\big), \qquad y(0.25) \approx (0.9239, 0.3827)$$

A quarter of the journey rotates the square-root vector by 22.5 degrees.

Fisher-Flow: a quarter of a spherical path

Toy alphabet A and G. Follow the spherical path from the all-A corner to the all-G corner. Square roots of probabilities are the sphere coordinates, denoted by y.

$$y(t) = \big(\cos(\pi t/2), \sin(\pi t/2)\big), \qquad y(0.25) \approx (0.9239, 0.3827)$$

A quarter of the journey rotates the square-root vector by 22.5 degrees.

$$p(0.25) = y(0.25)^2 = \big(\cos^2(\pi/8), \sin^2(\pi/8)\big) \approx (0.8536, 0.1464)$$

Fisher-Flow: a quarter of a spherical path

Toy alphabet A and G. Follow the spherical path from the all-A corner to the all-G corner. Square roots of probabilities are the sphere coordinates, denoted by y.

$$y(t) = \big(\cos(\pi t/2), \sin(\pi t/2)\big), \qquad y(0.25) \approx (0.9239, 0.3827)$$

A quarter of the journey rotates the square-root vector by 22.5 degrees.

$$p(0.25) = y(0.25)^2 = \big(\cos^2(\pi/8), \sin^2(\pi/8)\big) \approx (0.8536, 0.1464)$$

Squaring the coordinates gives probabilities of 85.36% for A and 14.64% for G.

Fisher-Flow: a quarter of a spherical path

Toy alphabet A and G. Follow the spherical path from the all-A corner to the all-G corner. Square roots of probabilities are the sphere coordinates, denoted by y.

$$y(t) = \big(\cos(\pi t/2), \sin(\pi t/2)\big), \qquad y(0.25) \approx (0.9239, 0.3827)$$

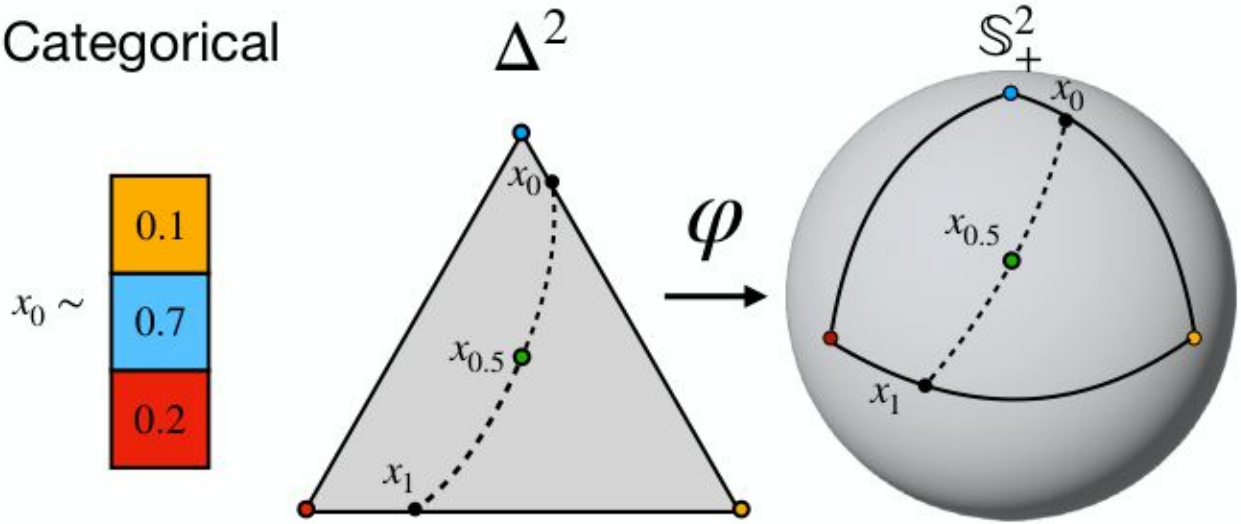
A quarter of the journey rotates the square-root vector by 22.5 degrees.

$$p(0.25) = y(0.25)^2 = \big(\cos^2(\pi/8), \sin^2(\pi/8)\big) \approx (0.8536, 0.1464)$$

Squaring the coordinates gives probabilities of 85.36% for A and 14.64% for G.

Equal progress in spherical distance produces a curved change in the probabilities.

Taking square roots maps a categorical distribution onto the sphere



A Fisher-Rao geodesic on the simplex becomes a great-circle arc on the sphere.

Davis,..,Bose. *NeurIPS* (2024)

Promoter design measured by regulatory activity

Table 1: MSE of the transcription profile conditioned on generated promoter DNA sequences over the test set. The last 3 MSE and PPL values are from 5 independent experiments. The remaining numbers are taken directly from Stark et al. [68].

Model	MSE (↓)	PPL (↓)
BIT DIFFUSION (BIT-ENCODING)	0.041	—
BIT DIFFUSION (ONE-HOT ENCODING)	0.040	—
D3PM-UNIFORM	0.038	—
DDSM	0.033	—
LANGUAGE MODEL	0.034 ± 0.001	2.247 ± 0.102
DIRICHLET FM	0.034 ± 0.004	$\mathbf{1.978 \pm 0.006}$
FISHER-FLOW (ours)	$\mathbf{0.029 \pm 0.001}$	$\mathbf{1.4 \pm 2.7}$

Davis,..,Bose. *NeurIPS* (2024)

The Dirichlet path reaches an error of 0.034 on this split

Table 1: MSE of the transcription profile conditioned on generated promoter DNA sequences over the test set. The last 3 MSE and PPL values are from 5 independent experiments. The remaining numbers are taken directly from Stark et al. [68].

Model	MSE (↓)	PPL (↓)
BIT DIFFUSION (BIT-ENCODING)	0.041	—
BIT DIFFUSION (ONE-HOT ENCODING)	0.040	—
D3PM-UNIFORM	0.038	—
DDSM	0.033	—
LANGUAGE MODEL	0.034 ± 0.001	2.247 ± 0.102
DIRICHLET FM	0.034 ± 0.004	$\mathbf{1.978 \pm 0.006}$
FISHER-FLOW (ours)	$\mathbf{0.029 \pm 0.001}$	$\mathbf{1.4 \pm 2.7}$

Davis,..,Bose. *NeurIPS* (2024)

Using the sphere improves the error to 0.029

Table 1: MSE of the transcription profile conditioned on generated promoter DNA sequences over the test set. The last 3 MSE and PPL values are from 5 independent experiments. The remaining numbers are taken directly from Stark et al. [68].

Model	MSE (↓)	PPL (↓)
BIT DIFFUSION (BIT-ENCODING)	0.041	—
BIT DIFFUSION (ONE-HOT ENCODING)	0.040	—
D3PM-UNIFORM	0.038	—
DDSM	0.033	—
LANGUAGE MODEL	0.034 ± 0.001	2.247 ± 0.102
DIRICHLET FM	0.034 ± 0.004	$\mathbf{1.978 \pm 0.006}$
FISHER-FLOW (ours)	$\mathbf{0.029 \pm 0.001}$	$\mathbf{1.4 \pm 2.7}$

Davis,..,Bose. *NeurIPS* (2024)

A language model reaches 0.034 with far more steps

Table 1: MSE of the transcription profile conditioned on generated promoter DNA sequences over the test set. The last 3 MSE and PPL values are from 5 independent experiments. The remaining numbers are taken directly from Stark et al. [68].

Model	MSE (↓)	PPL (↓)
BIT DIFFUSION (BIT-ENCODING)	0.041	—
BIT DIFFUSION (ONE-HOT ENCODING)	0.040	—
D3PM-UNIFORM	0.038	—
DDSM	0.033	—
LANGUAGE MODEL	0.034 ± 0.001	2.247 ± 0.102
DIRICHLET FM	0.034 ± 0.004	$\mathbf{1.978 \pm 0.006}$
FISHER-FLOW (ours)	$\mathbf{0.029 \pm 0.001}$	$\mathbf{1.4 \pm 2.7}$

Davis,..,Bose. *NeurIPS* (2024)

Fisher-Flow reports a perplexity of 1.4 with a standard error of 2.7, so read that number with care

Table 1: MSE of the transcription profile conditioned on generated promoter DNA sequences over the test set. The last 3 MSE and PPL values are from 5 independent experiments. The remaining numbers are taken directly from Stark et al. [68].

Model	MSE (↓)	PPL (↓)
BIT DIFFUSION (BIT-ENCODING)	0.041	—
BIT DIFFUSION (ONE-HOT ENCODING)	0.040	—
D3PM-UNIFORM	0.038	—
DDSM	0.033	—
LANGUAGE MODEL	0.034 ± 0.001	2.247 ± 0.102
DIRICHLET FM	0.034 ± 0.004	$\mathbf{1.978 \pm 0.006}$
FISHER-FLOW (ours)	$\mathbf{0.029 \pm 0.001}$	$\mathbf{1.4 \pm 2.7}$

Davis,..,Bose. *NeurIPS* (2024)

Enhancer generation measured by perplexity

Table 2: Perplexities (PPL) values for different methods for enhancer DNA generation. Lower PPL is better. Values are an average and standard error over 5 seeds.

Method	**Melanoma PPL** (↓)	**Fly Brain PPL** (↓)
Random Sequence	895.88	895.88
Language Model	2.22 ± 0.09	2.19 ± 0.10
DIRICHLET FM	2.25 ± 0.01	2.25 ± 0.02
FISHER-FLOW (ours)	$\mathbf{1.4 \pm 0.1}$	$\mathbf{1.4 \pm 0.66}$

Davis,..,Bose. *NeurIPS* (2024)

The Dirichlet path reaches 2.25 on both datasets

Table 2: Perplexities (PPL) values for different methods for enhancer DNA generation. Lower PPL is better. Values are an average and standard error over 5 seeds.

Method	**Melanoma PPL** (↓)	**Fly Brain PPL** (↓)
Random Sequence	895.88	895.88
Language Model	2.22 ± 0.09	2.19 ± 0.10
DIRICHLET FM	2.25 ± 0.01	2.25 ± 0.02
FISHER-FLOW (ours)	$\mathbf{1.4 \pm 0.1}$	$\mathbf{1.4 \pm 0.66}$

Davis,..,Bose. *NeurIPS* (2024)

The sphere reaches 1.4 on both

Table 2: Perplexities (PPL) values for different methods for enhancer DNA generation. Lower PPL is better. Values are an average and standard error over 5 seeds.

Method	**Melanoma PPL** (↓)	**Fly Brain PPL** (↓)
Random Sequence	895.88	895.88
Language Model	2.22 ± 0.09	2.19 ± 0.10
DIRICHLET FM	2.25 ± 0.01	2.25 ± 0.02
FISHER-FLOW (ours)	$\mathbf{1.4 \pm 0.1}$	$\mathbf{1.4 \pm 0.66}$

Davis,..,Bose. *NeurIPS* (2024)

A language model reaches 2.22 and 2.19

Table 2: Perplexities (PPL) values for different methods for enhancer DNA generation. Lower PPL is better. Values are an average and standard error over 5 seeds.

Method	**Melanoma PPL** (↓)	**Fly Brain PPL** (↓)
Random Sequence	895.88	895.88
Language Model	2.22 ± 0.09	2.19 ± 0.10
DIRICHLET FM	2.25 ± 0.01	2.25 ± 0.02
FISHER-FLOW (ours)	$\mathbf{1.4 \pm 0.1}$	$\mathbf{1.4 \pm 0.66}$

Davis,..,Bose. *NeurIPS* (2024)

The fly brain figure carries a standard error of 0.66, which is half the mean

Table 2: Perplexities (PPL) values for different methods for enhancer DNA generation. Lower PPL is better. Values are an average and standard error over 5 seeds.

Method	**Melanoma PPL (↓)**	**Fly Brain PPL (↓)**
Random Sequence	895.88	895.88
Language Model	2.22 ± 0.09	2.19 ± 0.10
DIRICHLET FM	2.25 ± 0.01	2.25 ± 0.02
FISHER-FLOW (ours)	$\mathbf{1.4 \pm 0.1}$	$\mathbf{1.4 \pm 0.66}$

Davis,..,Bose. *NeurIPS* (2024)

The appendix reports biological distances of 27.5 for Fisher-Flow and 7.3 for the Dirichlet path on melanoma.

Table 2: Perplexities (PPL) values for different methods for enhancer DNA generation. Lower PPL is better. Values are an average and standard error over 5 seeds.

Method	**Melanoma PPL** (↓)	**Fly Brain PPL** (↓)
Random Sequence	895.88	895.88
Language Model	2.22 ± 0.09	2.19 ± 0.10
DIRICHLET FM	2.25 ± 0.01	2.25 ± 0.02
FISHER-FLOW (ours)	$\mathbf{1.4 \pm 0.1}$	$\mathbf{1.4 \pm 0.66}$

Davis,..,Bose. *NeurIPS* (2024)

The appendix reports biological distances of 27.5 for Fisher-Flow and 7.3 for the Dirichlet path on melanoma. The authors switched the headline metric to perplexity.

Table 2: Perplexities (PPL) values for different methods for enhancer DNA generation. Lower PPL is better. Values are an average and standard error over 5 seeds.

Method	**Melanoma PPL** (↓)	**Fly Brain PPL** (↓)
Random Sequence	895.88	895.88
Language Model	2.22 ± 0.09	2.19 ± 0.10
DIRICHLET FM	2.25 ± 0.01	2.25 ± 0.02
FISHER-FLOW (ours)	$\mathbf{1.4 \pm 0.1}$	$\mathbf{1.4 \pm 0.66}$

Davis,..,Bose. *NeurIPS* (2024)

Summary: the simplex and the sphere

Summary: the simplex and the sphere

Straight lines on the simplex lose support:

$$\psi_t(x_0 \mid x_1) = (1-t)x_0 + tx_1 \implies \text{support } \{x_i \geq t\}$$

Summary: the simplex and the sphere

Straight lines on the simplex lose support:

$$\psi_t(x_0 \mid x_1) = (1-t)x_0 + tx_1 \implies \text{support } \{x_i \geq t\}$$

The straight-line path has support only where the data coordinate is at least t, so corners get ruled out at the times one over k.

Summary: the simplex and the sphere

Straight lines on the simplex lose support:

$$\psi_t(x_0 \mid x_1) = (1-t)x_0 + tx_1 \implies \text{support } \{x_i \geq t\}$$

Dirichlet paths keep it:

$$p_t(x \mid e_i) = \text{Dir}(x;\, \mathbf{1} + te_i), \qquad u_t(x \mid e_i) = C(x_i, t)\,(e_i - x)$$

Summary: the simplex and the sphere

Straight lines on the simplex lose support:

$$\psi_t(x_0 \mid x_1) = (1-t)x_0 + tx_1 \implies \text{support } \{x_i \geq t\}$$

Dirichlet paths keep it:

$$p_t(x \mid e_i) = \text{Dir}(x;\, \mathbf{1} + te_i), \qquad u_t(x \mid e_i) = C(x_i, t)\,(e_i - x)$$

With a Dirichlet path, every corner remains possible. The conditional velocity points toward the data corner at speed $C(x_i, t)$.

Summary: the simplex and the sphere

Straight lines on the simplex lose support:

$$\psi_t(x_0 \mid x_1) = (1-t)x_0 + tx_1 \implies \text{support } \{x_i \geq t\}$$

Dirichlet paths keep it:

$$p_t(x \mid e_i) = \mathrm{Dir}(x;\, \mathbf{1} + te_i), \qquad u_t(x \mid e_i) = C(x_i, t)\,(e_i - x)$$

Fisher-Rao becomes Euclidean on the sphere: $\langle u, v \rangle_p = \sum_i \frac{u^i v^i}{p^i}, \qquad \varphi(p) = \sqrt{p}$

Summary: the simplex and the sphere

Straight lines on the simplex lose support:

$$\psi_t(x_0 \mid x_1) = (1-t)x_0 + tx_1 \implies \text{support } \{x_i \geq t\}$$

Dirichlet paths keep it:

$$p_t(x \mid e_i) = \mathrm{Dir}(x;\, \mathbf{1} + te_i), \qquad u_t(x \mid e_i) = C(x_i, t)\,(e_i - x)$$

Fisher-Rao becomes Euclidean on the sphere: $\langle u, v \rangle_p = \sum_i \frac{u^i v^i}{p^i}, \qquad \varphi(p) = \sqrt{p}$

The Fisher-Rao inner product weights changes by inverse probability. Taking square roots converts it into the ordinary inner product on a sphere.

Summary: what we get from the relaxation

Summary: what we get from the relaxation

The velocity on the sphere:

$$u_t(x_t \mid x_1) = \frac{\log_{x_t}(x_1)}{1-t}$$

Summary: what we get from the relaxation

The velocity on the sphere: $$u_t(x_t \mid x_1) = \frac{\log_{x_t}(x_1)}{1-t}$$

The velocity is the logarithm map toward the data point divided by the time remaining, which has constant length.

Summary: what we get from the relaxation

The velocity on the sphere: $$u_t(x_t \mid x_1) = \frac{\log_{x_t}(x_1)}{1-t}$$

The posterior average again: $$\hat{v} = \sum_k u_t(x \mid e_k)\, p^{\theta}_{1|t}(e_k \mid x)$$

Summary: what we get from the relaxation

The velocity on the sphere: $$u_t(x_t \mid x_1) = \frac{\log_{x_t}(x_1)}{1-t}$$

The posterior average again: $$\hat{v} = \sum_k u_t(x \mid e_k)\, p^{\theta}_{1|t}(e_k \mid x)$$

The marginal velocity is the posterior-weighted average of the conditional velocities, as with jump rates.

Summary: what we get from the relaxation

The velocity on the sphere: $$u_t(x_t \mid x_1) = \frac{\log_{x_t}(x_1)}{1-t}$$

The posterior average again: $$\hat{v} = \sum_k u_t(x \mid e_k)\, p^{\theta}_{1|t}(e_k \mid x)$$

Because we sample by integrating an ODE, we can also use guidance, optimal-transport couplings, and one-step distillation.

Could we build the path by gradually cooling the noise we already use to sample tokens?

Gumbel-Softmax Flow Matching

Gumbel-Softmax Flow Matching with Straight-Through Guidance for Controllable Biological Sequence Generation

Sophia Tang[1,2], **Yinuo Zhang**[1,3], **Alexander Tong**[4,5], **Pranam Chatterjee**[1,6,7,†]

[1]Department of Biomedical Engineering, Duke University
[2]Management and Technology Program, University of Pennsylvania
[3]Center of Computational Biology, Duke-NUS Medical School
[4]Mila, Quebec AI Institute, [5]Université de Montréal
[6]Department of Computer Science, Duke University
[7]Department of Biostatistics and Bioinformatics, Duke University

[†]Corresponding author: pranam.chatterjee@duke.edu

Tang,..,Chatterjee. *ICLR Workshop* (2025)

So far, each geometry has had a path of its own

So far, each geometry has had a path of its own

In Dirichlet flow matching, we needed a Beta-function speed, computed numerically at every step.

So far, each geometry has had a path of its own

In Dirichlet flow matching, we needed a Beta-function speed, computed numerically at every step.

In Fisher-Flow, we needed exponential and logarithm maps on a sphere.

So far, each geometry has had a path of its own

In Dirichlet flow matching, we needed a Beta-function speed, computed numerically at every step.

In Fisher-Flow, we needed exponential and logarithm maps on a sphere.

In both, we started from a geometry and then looked for a path inside it.

The next path comes from the way we sample a token

The next path comes from the way we sample a token

With the Gumbel-max trick, we draw a categorical sample with noise and an argmax:

$$g_k \sim \text{Gumbel}(0, 1), \qquad \arg\max_k \left[\log \pi_k + g_k\right] \sim \text{Cat}(\pi)$$

The next path comes from the way we sample a token

With the Gumbel-max trick, we draw a categorical sample with noise and an argmax:

$$g_k \sim \text{Gumbel}(0, 1), \qquad \arg\max_k \left[\log \pi_k + g_k\right] \sim \text{Cat}(\pi)$$

Adding independent Gumbel noise g_k to each log probability and taking the argmax gives a sample from the categorical distribution π.

The next path comes from the way we sample a token

With the Gumbel-max trick, we draw a categorical sample with noise and an argmax:

$$g_k \sim \text{Gumbel}(0, 1), \qquad \arg\max_k \left[\log \pi_k + g_k\right] \sim \text{Cat}(\pi)$$

Adding independent Gumbel noise g_k to each log probability and taking the argmax gives a sample from the categorical distribution π.

A Gumbel variable is the negative log of an exponential clock.

The next path comes from the way we sample a token

With the Gumbel-max trick, we draw a categorical sample with noise and an argmax:

$$g_k \sim \text{Gumbel}(0, 1), \qquad \arg\max_k \left[\log \pi_k + g_k\right] \sim \text{Cat}(\pi)$$

Adding independent Gumbel noise g_k to each log probability and taking the argmax gives a sample from the categorical distribution π.

A Gumbel variable is the negative log of an exponential clock. We recover the exponential race used in discrete diffusion by expressing it in logarithms.

The next path comes from the way we sample a token

With the Gumbel-max trick, we draw a categorical sample with noise and an argmax:

$$g_k \sim \mathrm{Gumbel}(0, 1), \qquad \arg\max_k \left[\log \pi_k + g_k\right] \sim \mathrm{Cat}(\pi)$$

Adding independent Gumbel noise g_k to each log probability and taking the argmax gives a sample from the categorical distribution π.

A Gumbel variable is the negative log of an exponential clock. We recover the exponential race used in discrete diffusion by expressing it in logarithms.

The randomness sits in the noise, so the same noise with different logits gives a different letter, smoothly.

Replace the argmax with a softmax

Replace the argmax with a softmax

$$x = \text{softmax}\left(\frac{\log \pi + g}{\tau}\right), \qquad x_k = \frac{\exp\left((\log \pi_k + g_k)/\tau\right)}{\sum_j \exp\left((\log \pi_j + g_j)/\tau\right)}$$

Replace the argmax with a softmax

$$x = \text{softmax}\left(\frac{\log \pi + g}{\tau}\right), \qquad x_k = \frac{\exp\left((\log \pi_k + g_k)/\tau\right)}{\sum_j \exp\left((\log \pi_j + g_j)/\tau\right)}$$

The Gumbel-softmax sample is the softmax of the noisy log probabilities divided by a temperature τ, so the k-th coordinate is the exponential of that quantity divided by the sum over all letters.

Replace the argmax with a softmax

$$x = \text{softmax}\left(\frac{\log \pi + g}{\tau}\right), \qquad x_k = \frac{\exp\left((\log \pi_k + g_k)/\tau\right)}{\sum_j \exp\left((\log \pi_j + g_j)/\tau\right)}$$

The Gumbel-softmax sample is the softmax of the noisy log probabilities divided by a temperature τ, so the k-th coordinate is the exponential of that quantity divided by the sum over all letters.

As τ approaches zero, the vector approaches the one-hot argmax. At high temperatures, it approaches the uniform vector.

Replace the argmax with a softmax

$$x = \text{softmax}\left(\frac{\log \pi + g}{\tau}\right), \qquad x_k = \frac{\exp\left((\log \pi_k + g_k)/\tau\right)}{\sum_j \exp\left((\log \pi_j + g_j)/\tau\right)}$$

The Gumbel-softmax sample is the softmax of the noisy log probabilities divided by a temperature τ, so the k-th coordinate is the exponential of that quantity divided by the sum over all letters.

As τ approaches zero, the vector approaches the one-hot argmax. At high temperatures, it approaches the uniform vector.

We use temperature to control how concentrated the probability vector is.

Replace the argmax with a softmax

$$x = \text{softmax}\left(\frac{\log \pi + g}{\tau}\right), \qquad x_k = \frac{\exp\left((\log \pi_k + g_k)/\tau\right)}{\sum_j \exp\left((\log \pi_j + g_j)/\tau\right)}$$

The Gumbel-softmax sample is the softmax of the noisy log probabilities divided by a temperature τ, so the k-th coordinate is the exponential of that quantity divided by the sum over all letters.

As τ approaches zero, the vector approaches the one-hot argmax. At high temperatures, it approaches the uniform vector.

Cooling sharpens the same noisy preferences until one coordinate dominates.

Now let the temperature fall with time

Now let the temperature fall with time

In Gumbel-Softmax Flow Matching, the temperature cools on a fixed schedule:

$$\boxed{\tau(t) = \tau_{\max}\, e^{-\lambda t}, \qquad \tau_{\max} = 10, \quad \lambda = 3}$$

Now let the temperature fall with time

In Gumbel-Softmax Flow Matching, the temperature cools on a fixed schedule:

$$\tau(t) = \tau_{\max} e^{-\lambda t}, \qquad \tau_{\max} = 10, \quad \lambda = 3$$

The temperature at time t is τ_{max} times the exponential of minus λ t, with τ_{max} equal to 10 and λ equal to 3 in all their experiments.

Now let the temperature fall with time

In Gumbel-Softmax Flow Matching, the temperature cools on a fixed schedule:

$$\tau(t) = \tau_{\max} e^{-\lambda t}, \qquad \tau_{\max} = 10, \quad \lambda = 3$$

The temperature at time t is τ_{max} times the exponential of minus λ t, with τ_{max} equal to 10 and λ equal to 3 in all their experiments.

At t = 0 the temperature is 10, which is hot enough that every letter has nearly the same probability.

Now let the temperature fall with time

In Gumbel-Softmax Flow Matching, the temperature cools on a fixed schedule:

$$\tau(t) = \tau_{\max} e^{-\lambda t}, \qquad \tau_{\max} = 10, \quad \lambda = 3$$

The temperature at time t is τ_{max} times the exponential of minus λ t, with τ_{max} equal to 10 and λ equal to 3 in all their experiments.

At t = 0 the temperature is 10, which is hot enough that every letter has nearly the same probability.

At t = 1 the temperature is $10\, e^{-3} = 0.498$, which is cold enough to concentrate on one letter.

Now let the temperature fall with time

In Gumbel-Softmax Flow Matching, the temperature cools on a fixed schedule:

$$\tau(t) = \tau_{\max} e^{-\lambda t}, \qquad \tau_{\max} = 10, \quad \lambda = 3$$

The temperature at time t is τ_{max} times the exponential of minus λ t, with τ_{max} equal to 10 and λ equal to 3 in all their experiments.

At t = 0 the temperature is 10, which is hot enough that every letter has nearly the same probability.

At t = 1 the temperature is $10\, e^{-3} = 0.498$, which is cold enough to concentrate on one letter.

The temperature never reaches zero, so the point stays in the interior and the density stays defined.

Put the data token in the logits and cool

Put the data token in the logits and cool

Put the data token in the logits and cool:

$$\psi_t(x_1 = e_k) = \mathrm{softmax}\left(\frac{\delta_{\cdot k} + g/\beta}{\tau(t)}\right)$$

Put the data token in the logits and cool

Put the data token in the logits and cool:

$$\psi_t(x_1 = e_k) = \mathrm{softmax}\left(\frac{\delta_{\cdot k} + g/\beta}{\tau(t)}\right)$$

The point at time t is the softmax of the one-hot vector at the data token k, plus scaled Gumbel noise, divided by the temperature at time t.

Put the data token in the logits and cool

Put the data token in the logits and cool:

$$\psi_t(x_1 = e_k) = \mathrm{softmax}\left(\frac{\delta_{\cdot k} + g/\beta}{\tau(t)}\right)$$

The point at time t is the softmax of the one-hot vector at the data token k, plus scaled Gumbel noise, divided by the temperature at time t.

One Gumbel draw is fixed for the whole trajectory, so the path is a smooth curve from near the center to near the corner e_k.

Put the data token in the logits and cool

Put the data token in the logits and cool:

$$\psi_t(x_1 = e_k) = \mathrm{softmax}\left(\frac{\delta_{\cdot k} + g/\beta}{\tau(t)}\right)$$

The point at time t is the softmax of the one-hot vector at the data token k, plus scaled Gumbel noise, divided by the temperature at time t.

One Gumbel draw is fixed for the whole trajectory, so the path is a smooth curve from near the center to near the corner e_k.

The endpoints hold only approximately, since at t = 1 the largest coordinate is close to one.

Put the data token in the logits and cool

Put the data token in the logits and cool:

$$\psi_t(x_1 = e_k) = \mathrm{softmax}\left(\frac{\delta_{\cdot k} + g/\beta}{\tau(t)}\right)$$

The point at time t is the softmax of the one-hot vector at the data token k, plus scaled Gumbel noise, divided by the temperature at time t.

One Gumbel draw is fixed for the whole trajectory, so the path is a smooth curve from near the center to near the corner e_k.

The endpoints hold only approximately, since at t = 1 the largest coordinate is close to one.

We scale the noise with β. A larger β produces less dispersed paths.

Let’s anneal one DNA base toward G

Let’s anneal one DNA base toward G

Fix x_1 = G and one Gumbel draw. Evaluate the point at four times.

Let’s anneal one DNA base toward G

Fix x_1 = G and one Gumbel draw. Evaluate the point at four times.

$$
\begin{aligned}
t = 0.0 &: \ \tau = 10.00 \ \ (0.2438,\ 0.2331,\ 0.2805,\ 0.2426) \\
t = 0.5 &: \ \tau = 2.231 \ \ (0.2142,\ 0.1751,\ 0.4012,\ 0.2095) \\
t = 0.9 &: \ \tau = 0.672 \ \ (0.0955,\ 0.0489,\ 0.7669,\ 0.0887) \\
t = 1.0 &: \ \tau = 0.498 \ \ (0.0528,\ 0.0214,\ 0.8781,\ 0.0477)
\end{aligned}
$$

Let’s anneal one DNA base toward G

Fix x_1 = G and one Gumbel draw. Evaluate the point at four times.

$$
\begin{aligned}
t = 0.0 &: \ \tau = 10.00 \ \ (0.2438,\ 0.2331,\ 0.2805,\ 0.2426) \\
t = 0.5 &: \ \tau = 2.231 \ \ (0.2142,\ 0.1751,\ 0.4012,\ 0.2095) \\
t = 0.9 &: \ \tau = 0.672 \ \ (0.0955,\ 0.0489,\ 0.7669,\ 0.0887) \\
t = 1.0 &: \ \tau = 0.498 \ \ (0.0528,\ 0.0214,\ 0.8781,\ 0.0477)
\end{aligned}
$$

At time zero the four coordinates are near one quarter each, at t = 0.5 G has risen to 0.4012, at t = 0.9 it holds 0.7669, and at t = 1 it holds 0.8781.

Let’s anneal one DNA base toward G

Fix x_1 = G and one Gumbel draw. Evaluate the point at four times.

$$
\begin{aligned}
t = 0.0 &: \ \tau = 10.00 \ \ (0.2438,\ 0.2331,\ 0.2805,\ 0.2426) \\
t = 0.5 &: \ \tau = 2.231 \ \ (0.2142,\ 0.1751,\ 0.4012,\ 0.2095) \\
t = 0.9 &: \ \tau = 0.672 \ \ (0.0955,\ 0.0489,\ 0.7669,\ 0.0887) \\
t = 1.0 &: \ \tau = 0.498 \ \ (0.0528,\ 0.0214,\ 0.8781,\ 0.0477)
\end{aligned}
$$

At time zero the four coordinates are near one quarter each, at t = 0.5 G has risen to 0.4012, at t = 0.9 it holds 0.7669, and at t = 1 it holds 0.8781.

The final point assigns probability 0.878 to G. We select the final token with an argmax.

Let’s anneal one DNA base toward G

Fix x_1 = G and one Gumbel draw. Evaluate the point at four times.

$$
\begin{aligned}
t = 0.0 &: \ \tau = 10.00 \ \ (0.2438,\ 0.2331,\ 0.2805,\ 0.2426) \\
t = 0.5 &: \ \tau = 2.231 \ \ (0.2142,\ 0.1751,\ 0.4012,\ 0.2095) \\
t = 0.9 &: \ \tau = 0.672 \ \ (0.0955,\ 0.0489,\ 0.7669,\ 0.0887) \\
t = 1.0 &: \ \tau = 0.498 \ \ (0.0528,\ 0.0214,\ 0.8781,\ 0.0477)
\end{aligned}
$$

At time zero the four coordinates are near one quarter each, at t = 0.5 G has risen to 0.4012, at t = 0.9 it holds 0.7669, and at t = 1 it holds 0.8781.

The final point assigns probability 0.878 to G. We select the final token with an argmax.

The other three letters shrink at different speeds, because each carries its own Gumbel draw.

Differentiate that path in time

Differentiate that path in time

$$u_{t,i}(x \mid x_1 = e_k) = \frac{\lambda}{\tau(t)}\, x_i \sum_j x_j \Big[(\delta_{ik} + g_i/\beta) - (\delta_{jk} + g_j/\beta)\Big]$$

Differentiate that path in time

$$u_{t,i}(x \mid x_1 = e_k) = \frac{\lambda}{\tau(t)}\, x_i \sum_j x_j \Big[(\delta_{ik} + g_i/\beta) - (\delta_{jk} + g_j/\beta)\Big]$$

The i-th coordinate of the conditional velocity equals λ over τ(t), times the current mass x_i, times the sum over letters j of x_j times the difference between letter i's noisy logit and letter j's, each logit being the data indicator plus the Gumbel draw divided by β.

Differentiate that path in time

$$u_{t,i}(x \mid x_1 = e_k) = \frac{\lambda}{\tau(t)}\, x_i \sum_j x_j \Big[(\delta_{ik} + g_i/\beta) - (\delta_{jk} + g_j/\beta)\Big]$$

The i-th coordinate of the conditional velocity equals λ over τ(t), times the current mass x_i, times the sum over letters j of x_j times the difference between letter i's noisy logit and letter j's, each logit being the data indicator plus the Gumbel draw divided by β.

Applying the chain rule through the softmax Jacobian gives the difference between each logit and its probability-weighted mean. A coordinate grows when its logit exceeds that mean.

Differentiate that path in time

$$u_{t,i}(x \mid x_1 = e_k) = \frac{\lambda}{\tau(t)}\, x_i \sum_j x_j \Big[(\delta_{ik} + g_i/\beta) - (\delta_{jk} + g_j/\beta)\Big]$$

The i-th coordinate of the conditional velocity equals λ over τ(t), times the current mass x_i, times the sum over letters j of x_j times the difference between letter i's noisy logit and letter j's, each logit being the data indicator plus the Gumbel draw divided by β.

A letter gains probability when its noisy logit exceeds the current probability-weighted average.

The replicator equation

The replicator equation

The coordinates sum to zero because the two averages cancel, so the velocity is tangent to the simplex.

The replicator equation

The coordinates sum to zero because the two averages cancel, so the velocity is tangent to the simplex.

Evolutionary dynamics has the same equation, with the logits as fitness and the coordinates as population shares.

The replicator equation

The coordinates sum to zero because the two averages cancel, so the velocity is tangent to the simplex.

Evolutionary dynamics has the same equation, with the logits as fitness and the coordinates as population shares.

We retain g_i/β when differentiating the stated path. Equation 10 in the paper prints g_i without this scaling.

At inference we drop the noise and the velocity simplifies

At inference we drop the noise and the velocity simplifies

Setting g = 0 in the formula above:

$$\boxed{u_t(x \mid x_1 = e_k) = \frac{\lambda}{\tau(t)}\, x_k \left(e_k - x\right)}$$

At inference we drop the noise and the velocity simplifies

Setting g = 0 in the formula above:

$$u_t(x \mid x_1 = e_k) = \frac{\lambda}{\tau(t)}\, x_k \left(e_k - x\right)$$

With the noise set to zero, the conditional velocity equals λ over τ(t), times the current mass on the data token, times the direction from the current point toward the data corner.

At inference we drop the noise and the velocity simplifies

Setting g = 0 in the formula above:

$$u_t(x \mid x_1 = e_k) = \frac{\lambda}{\tau(t)}\, x_k \left(e_k - x\right)$$

With the noise set to zero, the conditional velocity equals λ over τ(t), times the current mass on the data token, times the direction from the current point toward the data corner.

The factor $x_k(1 - x_k)$ makes the speed small both near the data corner and when the data token has little probability.

At inference we drop the noise and the velocity simplifies

Setting g = 0 in the formula above:

$$u_t(x \mid x_1 = e_k) = \frac{\lambda}{\tau(t)}\, x_k \left(e_k - x\right)$$

With the noise set to zero, the conditional velocity equals λ over τ(t), times the current mass on the data token, times the direction from the current point toward the data corner.

The factor $x_k(1 - x_k)$ makes the speed small both near the data corner and when the data token has little probability.

The velocity vanishes at $x_k = 0$.

At inference we drop the noise and the velocity simplifies

Setting g = 0 in the formula above:

$$u_t(x \mid x_1 = e_k) = \frac{\lambda}{\tau(t)}\, x_k \left(e_k - x\right)$$

With the noise set to zero, the conditional velocity equals λ over τ(t), times the current mass on the data token, times the direction from the current point toward the data corner.

The factor $x_k(1 - x_k)$ makes the speed small both near the data corner and when the data token has little probability.

The velocity vanishes at $x_k = 0$. A token cannot regain probability once it has none.

At inference we drop the noise and the velocity simplifies

Setting g = 0 in the formula above:

$$u_t(x \mid x_1 = e_k) = \frac{\lambda}{\tau(t)}\, x_k \left(e_k - x\right)$$

With the noise set to zero, the conditional velocity equals λ over τ(t), times the current mass on the data token, times the direction from the current point toward the data corner.

The factor $x_k(1 - x_k)$ makes the speed small both near the data corner and when the data token has little probability.

The velocity vanishes at $x_k = 0$. A token cannot regain probability once it has none.

Because 1/τ(t) grows, the flow accelerates as t approaches 1.

Let’s take one step on our base

Let’s take one step on our base

At t = 0.5 the base sits at x = (0.2142, 0.1751, 0.4012, 0.2095) and $\lambda/\tau(t) = 1.345$.

Let’s take one step on our base

At t = 0.5 the base sits at x = (0.2142, 0.1751, 0.4012, 0.2095) and λ/τ(t) = 1.345.

$$u_t(x \mid \mathrm{G}) = 1.345 \times 0.401 \times (e_{\mathrm{G}} - x) = (-0.116,\ -0.094,\ 0.323,\ -0.113)$$

Let’s take one step on our base

At t = 0.5 the base sits at x = (0.2142, 0.1751, 0.4012, 0.2095) and λ/τ(t) = 1.345.

$$u_t(x \mid \mathrm{G}) = 1.345 \times 0.401 \times (e_{\mathrm{G}} - x) = (-0.116,\ -0.094,\ 0.323,\ -0.113)$$

The velocity toward G is 1.345 times 0.4012 times the direction to the G corner, which gives 0.323 for G, −0.116 for A, −0.094 for C, and −0.113 for T.

Let’s take one step on our base

At t = 0.5 the base sits at x = (0.2142, 0.1751, 0.4012, 0.2095) and λ/τ(t) = 1.345.

$$u_t(x \mid \mathrm{G}) = 1.345 \times 0.401 \times (e_{\mathrm{G}} - x) = (-0.116,\ -0.094,\ 0.323,\ -0.113)$$

The coordinates sum to zero, so an Euler step preserves the sum. Their sampler still projects onto the simplex, because a large step can drive a coordinate below zero.

Let’s take one step on our base

At t = 0.5 the base sits at x = (0.2142, 0.1751, 0.4012, 0.2095) and λ/τ(t) = 1.345.

$$u_t(x \mid \mathrm{G}) = 1.345 \times 0.401 \times (e_{\mathrm{G}} - x) = (-0.116,\ -0.094,\ 0.323,\ -0.113)$$

The coordinates sum to zero, so an Euler step preserves the sum. Their sampler still projects onto the simplex, because a large step can drive a coordinate below zero.

Keeping the Gumbel noise with β = 2 gives velocity (–0.136, –0.217, 0.500, –0.147), about 50% larger. Noise also transfers probability between the non-target letters.

Training is cross-entropy on the data token

Training is cross-entropy on the data token

The denoiser predicts the data token:

$$\boxed{\mathcal{L} = \mathbb{E}\big[-\log\langle x_\theta(x_t, t),\ x_1\rangle\big]}$$

Training is cross-entropy on the data token

The denoiser predicts the data token: $$\boxed{\mathcal{L} = \mathbb{E}\big[-\log\langle x_\theta(x_t, t),\, x_1\rangle\big]}$$

The loss is the expected negative logarithm of the inner product between the denoiser output and the one-hot data token, which is the probability it assigns to the true letter.

Training is cross-entropy on the data token

The denoiser predicts the data token: $$\boxed{\mathcal{L} = \mathbb{E}\big[-\log\langle x_\theta(x_t, t),\, x_1\rangle\big]}$$

The loss is the expected negative logarithm of the inner product between the denoiser output and the one-hot data token, which is the probability it assigns to the true letter.

The conditional velocity appears only at sampling time.

Average the conditional velocities over the letters

Average the conditional velocities over the letters

$$u_t^\theta(x_t) = \sum_{k=1}^{V} u_t(x \mid x_1 = e_k) \left\langle x_\theta(x_t, t),\, e_k \right\rangle$$

Average the conditional velocities over the letters

$$u_t^\theta(x_t) = \sum_{k=1}^{V} u_t(x \mid x_1 = e_k)\, \langle x_\theta(x_t, t),\, e_k \rangle$$

The marginal velocity is the sum over letters of the conditional velocity toward that letter, weighted by the probability the denoiser assigns to it.

Average the conditional velocities over the letters

$$u_t^\theta(x_t) = \sum_{k=1}^{V} u_t(x \mid x_1 = e_k)\, \langle x_\theta(x_t, t),\, e_k \rangle$$

The marginal velocity is the sum over letters of the conditional velocity toward that letter, weighted by the probability the denoiser assigns to it.

The same posterior average has now appeared for jump rates, for Dirichlet paths, for the sphere, and here.

Average the conditional velocities over the letters

$$u_t^\theta(x_t) = \sum_{k=1}^{V} u_t(x \mid x_1 = e_k)\, \langle x_\theta(x_t, t),\, e_k \rangle$$

The marginal velocity is the sum over letters of the conditional velocity toward that letter, weighted by the probability the denoiser assigns to it.

The same posterior average has now appeared for jump rates, for Dirichlet paths, for the sphere, and here.

Training this way has the same gradient as regressing the true velocity (proved in the paper).

Let’s run one Euler step late in the path

Let’s run one Euler step late in the path

At t = 0.8 the base sits at (0.138, 0.084, 0.647, 0.131) and the denoiser gives (0.05, 0.10, 0.70, 0.15).

Let’s run one Euler step late in the path

At t = 0.8 the base sits at (0.138, 0.084, 0.647, 0.131) and the denoiser gives (0.05, 0.10, 0.70, 0.15).

$$u_t^\theta(x_t) = (-0.200,\ -0.108,\ 0.454,\ -0.146), \qquad x_t + 0.01\, u_t^\theta = (0.136,\ 0.083,\ 0.651,\ 0.129)$$

Let’s run one Euler step late in the path

At t = 0.8 the base sits at (0.138, 0.084, 0.647, 0.131) and the denoiser gives (0.05, 0.10, 0.70, 0.15).

$$u_t^\theta(x_t) = (-0.200,\ -0.108,\ 0.454,\ -0.146), \qquad x_t + 0.01\, u_t^\theta = (0.136,\ 0.083,\ 0.651,\ 0.129)$$

The marginal velocity is −0.200 for A, −0.108 for C, 0.454 for G, and −0.146 for T. A step of size 0.01 increases G from 0.647 to 0.651.

Let’s run one Euler step late in the path

At t = 0.8 the base sits at (0.138, 0.084, 0.647, 0.131) and the denoiser gives (0.05, 0.10, 0.70, 0.15).

$$u_t^\theta(x_t) = (-0.200,\ -0.108,\ 0.454,\ -0.146), \qquad x_t + 0.01\, u_t^\theta = (0.136,\ 0.083,\ 0.651,\ 0.129)$$

T has the second-highest denoiser probability, 0.15. Its probability still decreases because the velocity also depends on the current probability of each letter.

Let’s run one Euler step late in the path

At t = 0.8 the base sits at (0.138, 0.084, 0.647, 0.131) and the denoiser gives (0.05, 0.10, 0.70, 0.15).

$$u_t^\theta(x_t) = (-0.200,\ -0.108,\ 0.454,\ -0.146), \qquad x_t + 0.01\, u_t^\theta = (0.136,\ 0.083,\ 0.651,\ 0.129)$$

T has the second-highest denoiser probability, 0.15. Its probability still decreases because the velocity also depends on the current probability of each letter.

The step keeps the coordinates summing to one.

Write the same path in log coordinates

Write the same path in log coordinates

They also write the path in log coordinates, where the density is the ExpConcrete distribution.

Write the same path in log coordinates

They also write the path in log coordinates, where the density is the ExpConcrete distribution.

$$\boxed{\nabla_{x_i} \log p_t(x \mid x_1 = e_k) = -\tau(t) + \tau(t)\, V \cdot \mathrm{softmax}\big(\delta_{ik} - \tau(t)\, x_i\big)}$$

Write the same path in log coordinates

They also write the path in log coordinates, where the density is the ExpConcrete distribution.

$$\nabla_{x_i} \log p_t(x \mid x_1 = e_k) = -\tau(t) + \tau(t)\, V \cdot \mathrm{softmax}\big(\delta_{ik} - \tau(t)\, x_i\big)$$

The score of the conditional path in log coordinates is minus τ(t) plus τ(t) times the alphabet size V, times the softmax of the data indicator minus τ(t) times the coordinate.

Write the same path in log coordinates

They also write the path in log coordinates, where the density is the ExpConcrete distribution.

$$\boxed{\nabla_{x_i} \log p_t(x \mid x_1 = e_k) = -\tau(t) + \tau(t)\, V \cdot \mathrm{softmax}\big(\delta_{ik} - \tau(t)\, x_i\big)}$$

The score of the conditional path in log coordinates is minus τ(t) plus τ(t) times the alphabet size V, times the softmax of the data indicator minus τ(t) times the coordinate.

The coordinates sum to zero, since the softmax sums to one and V τ cancels V τ.

Write the same path in log coordinates

They also write the path in log coordinates, where the density is the ExpConcrete distribution.

$$\nabla_{x_i} \log p_t(x \mid x_1 = e_k) = -\tau(t) + \tau(t)\, V \cdot \mathrm{softmax}\big(\delta_{ik} - \tau(t)\, x_i\big)$$

The score of the conditional path in log coordinates is minus τ(t) plus τ(t) times the alphabet size V, times the softmax of the data indicator minus τ(t) times the coordinate.

The coordinates sum to zero, since the softmax sums to one and V τ cancels V τ.

We use the sign derived in the proof. Theorem 2 in the paper prints the opposite overall sign.

Gumbel-Softmax Flow Matching: cooling one base

For one toy DNA position, fix the data-plus-Gumbel logits at (0, −1, 1, 0) for A, C, G, T. Keep this same noise draw while lowering the temperature.

Gumbel-Softmax Flow Matching: cooling one base

For one toy DNA position, fix the data-plus-Gumbel logits at (0, −1, 1, 0) for A, C, G, T. Keep this same noise draw while lowering the temperature.

$$p_{\tau=2} = \mathrm{softmax}(0, -0.5, 0.5, 0) \approx (0.2350, 0.1425, 0.3875, 0.2350)$$

Gumbel-Softmax Flow Matching: cooling one base

For one toy DNA position, fix the data-plus-Gumbel logits at (0, −1, 1, 0) for A, C, G, T. Keep this same noise draw while lowering the temperature.

$$p_{\tau=2} = \mathrm{softmax}(0, -0.5, 0.5, 0) \approx (0.2350, 0.1425, 0.3875, 0.2350)$$

At temperature 2, G has the largest coordinate, but substantial mass remains on the other bases.

Gumbel-Softmax Flow Matching: cooling one base

For one toy DNA position, fix the data-plus-Gumbel logits at (0, −1, 1, 0) for A, C, G, T. Keep this same noise draw while lowering the temperature.

$$p_{\tau=2} = \mathrm{softmax}(0, -0.5, 0.5, 0) \approx (0.2350, 0.1425, 0.3875, 0.2350)$$

At temperature 2, G has the largest coordinate, but substantial mass remains on the other bases.

$$p_{\tau=0.5} = \mathrm{softmax}(0, -2, 2, 0) \approx (0.1050, 0.0142, 0.7758, 0.1050)$$

Gumbel-Softmax Flow Matching: cooling one base

For one toy DNA position, fix the data-plus-Gumbel logits at (0, −1, 1, 0) for A, C, G, T. Keep this same noise draw while lowering the temperature.

$$p_{\tau=2} = \mathrm{softmax}(0, -0.5, 0.5, 0) \approx (0.2350, 0.1425, 0.3875, 0.2350)$$

At temperature 2, G has the largest coordinate, but substantial mass remains on the other bases.

$$p_{\tau=0.5} = \mathrm{softmax}(0, -2, 2, 0) \approx (0.1050, 0.0142, 0.7758, 0.1050)$$

At temperature 0.5, G holds about 77.6% of the mass. The logits and the noise have stayed fixed.

Gumbel-Softmax Flow Matching: cooling one base

For one toy DNA position, fix the data-plus-Gumbel logits at (0, −1, 1, 0) for A, C, G, T. Keep this same noise draw while lowering the temperature.

$$p_{\tau=2} = \mathrm{softmax}(0, -0.5, 0.5, 0) \approx (0.2350, 0.1425, 0.3875, 0.2350)$$

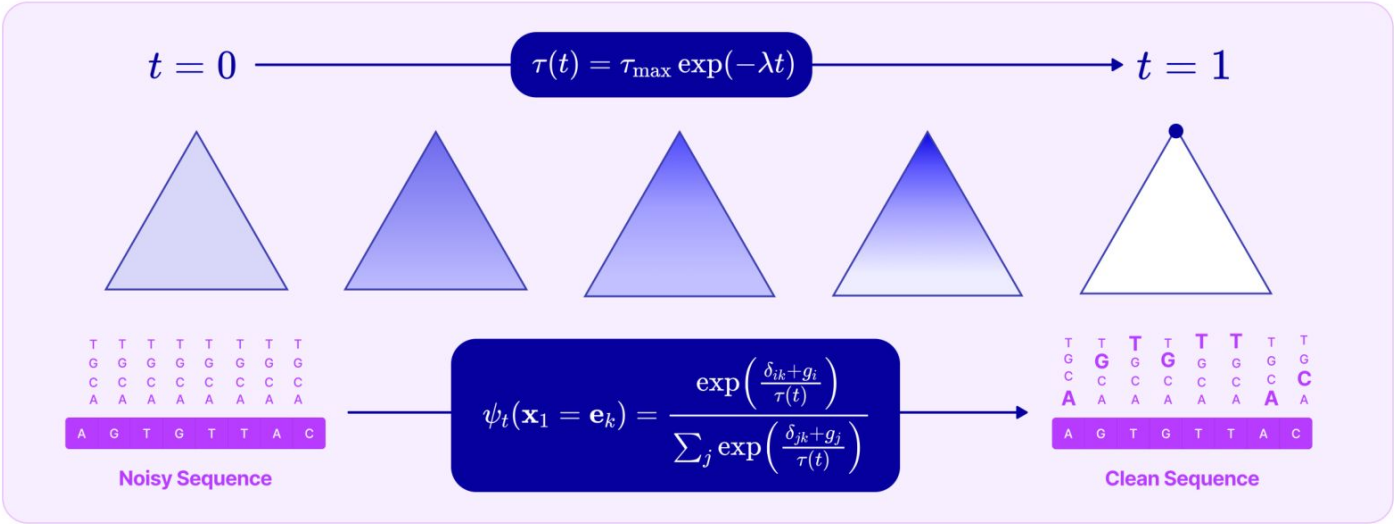
At temperature 2, G has the largest coordinate, but substantial mass remains on the other bases.

$$p_{\tau=0.5} = \mathrm{softmax}(0, -2, 2, 0) \approx (0.1050, 0.0142, 0.7758, 0.1050)$$

At temperature 0.5, G holds about 77.6% of the mass. The logits and the noise have stayed fixed.

Cooling magnifies the gaps already present in this trajectory.

Training on Gumbel-softmax samples



Gumbel-softmax transformations at temperature τ(t) produce the training points.

Tang,..,Chatterjee. *ICLR Workshop* (2025)

Here is promoter design in Table 1 of the paper

Model	MSE (↓)
Bit Diffusion (Bit Encoding)*	0.041
Bit Diffusion (One-Hot Encoding)*	0.040
D3PM-Uniform*	0.038
DDSM*	0.033
Language Model*	0.033
Dirichlet Flow Matching	0.029
Fisher Flow Matching	0.030
Gumbel-Softmax Flow Matching (Ours)	0.029

Table 1: **Evaluation of promoter DNA generation conditioned on transcription profile.** MSE was evaluated across all validation batches between the predicted signal of a conditionally generated sequence and the true sequence. Regulatory signals were predicted with a pretrained Sei model [25]. Numbers with * are from Stark et al. [4]

Tang,..,Chatterjee. *ICLR Workshop* (2025)

The Dirichlet path reaches an error of 0.029

Model	MSE (↓)
Bit Diffusion (Bit Encoding)*	0.041
Bit Diffusion (One-Hot Encoding)*	0.040
D3PM-Uniform*	0.038
DDSM*	0.033
Language Model*	0.033
Dirichlet Flow Matching	0.029
Fisher Flow Matching	0.030
Gumbel-Softmax Flow Matching (Ours)	0.029

Table 1: **Evaluation of promoter DNA generation conditioned on transcription profile.** MSE was evaluated across all validation batches between the predicted signal of a conditionally generated sequence and the true sequence. Regulatory signals were predicted with a pretrained Sei model [25]. Numbers with * are from Stark et al. [4]

Tang,..,Chatterjee. *ICLR Workshop* (2025)

The sphere reaches 0.030

Model	MSE (↓)
Bit Diffusion (Bit Encoding)*	0.041
Bit Diffusion (One-Hot Encoding)*	0.040
D3PM-Uniform*	0.038
DDSM*	0.033
Language Model*	0.033
Dirichlet Flow Matching	0.029
Fisher Flow Matching	0.030
Gumbel-Softmax Flow Matching (Ours)	0.029

Table 1: **Evaluation of promoter DNA generation conditioned on transcription profile.** MSE was evaluated across all validation batches between the predicted signal of a conditionally generated sequence and the true sequence. Regulatory signals were predicted with a pretrained Sei model [25]. Numbers with * are from Stark et al. [4]

Tang,..,Chatterjee. *ICLR Workshop* (2025)

The Gumbel-softmax path has error 0.029

Model	MSE (↓)
Bit Diffusion (Bit Encoding)*	0.041
Bit Diffusion (One-Hot Encoding)*	0.040
D3PM-Uniform*	0.038
DDSM*	0.033
Language Model*	0.033
Dirichlet Flow Matching	0.029
Fisher Flow Matching	0.030
Gumbel-Softmax Flow Matching (Ours)	0.029

Table 1: **Evaluation of promoter DNA generation conditioned on transcription profile.** MSE was evaluated across all validation batches between the predicted signal of a conditionally generated sequence and the true sequence. Regulatory signals were predicted with a pretrained Sei model [25]. Numbers with * are from Stark et al. [4]

Tang,..,Chatterjee. *ICLR Workshop* (2025)

The three geometries are not separated on this task. Each paper retrained the baselines, so these columns are not the same experiment as the earlier ones

Model	**MSE** (↓)
Bit Diffusion (Bit Encoding)*	0.041
Bit Diffusion (One-Hot Encoding)*	0.040
D3PM-Uniform*	0.038
DDSM*	0.033
Language Model*	0.033
Dirichlet Flow Matching	0.029
Fisher Flow Matching	0.030
Gumbel-Softmax Flow Matching (Ours)	0.029

Table 1: **Evaluation of promoter DNA generation conditioned on transcription profile.** MSE was evaluated across all validation batches between the predicted signal of a conditionally generated sequence and the true sequence. Regulatory signals were predicted with a pretrained Sei model [25]. Numbers with * are from Stark et al. [4]

Tang,..,Chatterjee. *ICLR Workshop* (2025)

Now let the alphabet grow, in Table 7 of the paper

Simplex Dimension K	20	40	60	80	100	120	140	160	512
Linear FM	0.013	0.046	0.070	0.100	0.114	0.112	0.156	0.146	0.479
Dirichlet FM	0.007	0.017	0.032	0.035	0.028	0.024	0.039	0.053	0.554
Fisher FM (Optimal Transport)	0.0004	0.007	0.007	0.007	0.008	0.043	0.013	0.013	0.036
Gumbel-Softmax FM (Ours)	0.029	0.027	0.025	0.027	0.030	0.029	0.035	0.038	0.048

Tang,..,Chatterjee. *ICLR Workshop* (2025)

At 512 letters, the Dirichlet path reaches a KL of 0.554 to the true distribution

Simplex Dimension K	20	40	60	80	100	120	140	160	512
Linear FM	0.013	0.046	0.070	0.100	0.114	0.112	0.156	0.146	0.479
Dirichlet FM	0.007	0.017	0.032	0.035	0.028	0.024	0.039	0.053	0.554
Fisher FM (Optimal Transport)	0.0004	0.007	0.007	0.007	0.008	0.043	0.013	0.013	0.036
Gumbel-Softmax FM (Ours)	0.029	0.027	0.025	0.027	0.030	0.029	0.035	0.038	0.048

Tang,..,Chatterjee. *ICLR Workshop* (2025)

The Gumbel-softmax path stays at 0.048, close to its value at 20 letters

Simplex Dimension K	20	40	60	80	100	120	140	160	512
Linear FM	0.013	0.046	0.070	0.100	0.114	0.112	0.156	0.146	0.479
Dirichlet FM	0.007	0.017	0.032	0.035	0.028	0.024	0.039	0.053	0.554
Fisher FM (Optimal Transport)	0.0004	0.007	0.007	0.007	0.008	0.043	0.013	0.013	0.036
Gumbel-Softmax FM (Ours)	0.029	0.027	0.025	0.027	0.030	0.029	0.035	0.038	0.048

Tang,..,Chatterjee. *ICLR Workshop* (2025)

The sphere with an optimal-transport coupling reaches 0.036

Simplex Dimension K	20	40	60	80	100	120	140	160	512
Linear FM	0.013	0.046	0.070	0.100	0.114	0.112	0.156	0.146	0.479
Dirichlet FM	0.007	0.017	0.032	0.035	0.028	0.024	0.039	0.053	0.554
Fisher FM (Optimal Transport)	0.0004	0.007	0.007	0.007	0.008	0.043	0.013	0.013	0.036
Gumbel-Softmax FM (Ours)	0.029	0.027	0.025	0.027	0.030	0.029	0.035	0.038	0.048

Tang,..,Chatterjee. *ICLR Workshop* (2025)

Below 40 letters the Dirichlet path is the better one, so the ranking depends on the alphabet

Simplex Dimension K	20	40	60	80	100	120	140	160	512
Linear FM	0.013	0.046	0.070	0.100	0.114	0.112	0.156	0.146	0.479
Dirichlet FM	0.007	0.017	0.032	0.035	0.028	0.024	0.039	0.053	0.554
Fisher FM (Optimal Transport)	0.0004	0.007	0.007	0.007	0.008	0.043	0.013	0.013	0.036
Gumbel-Softmax FM (Ours)	0.029	0.027	0.025	0.027	0.030	0.029	0.035	0.038	0.048

Tang,..,Chatterjee. *ICLR Workshop* (2025)